M04 — Prácticas integrales: patrones de arquitectura con .NET 8

Autor del curso: JAIME GALLO

Cómo utilizar el manual

Continúa exactamente la etapa **M03 3.4**, que contiene nueve proyectos y cincuenta pruebas. Construye cada etapa en su propia carpeta a partir de la anterior. Los bloques de este manual contienen el código completo que se crea o modifica; no copies una etapa terminada para eludir los pasos. Los 120 pasos mantienen la construcción de los catálogos, validadores, documentación y pruebas, e incorporan la implementación ejecutable de los patrones en las capas hexagonales.

Requisitos: Visual Studio con desarrollo ASP.NET, SDK .NET 8 y SQL Server Express o LocalDB. Las pruebas de integración usan el proveedor SQL Server real; las pruebas unitarias de lógica pueden emplear dobles de sus puertos. Ejecuta los comandos desde la raíz de la etapa indicada. Para usar terminal Bash sustituye las barras inversas de las rutas Windows por barras normales.

SQL Server para las pruebas. Conserva `tests/Compartido/SqlServerPruebas.cs` al copiar la etapa anterior. `BaseSqlServer.Crear()` toma la conexión de `ACERIA_TEST_SQLSERVER`, conecta a `master` y crea una base con nombre único por prueba; la cuenta necesita permiso para crear y eliminar esas bases. Al finalizar elimina solo la base que creó. Configura la variable antes de ejecutar `dotnet test`. Ejemplo PowerShell con LocalDB: `$env:ACERIA_TEST_SQLSERVER = "Server=(localdb)\MSSQLLocalDB;Database=master;Integrated Security=True;TrustServerCertificate=True"`. Para Express sustituye el servidor por `.\SQLEXPRESS` o el nombre de tu instancia. Si la variable no está definida, el helper necesita Docker y arranca SQL Server 2022 Express mediante `Testcontainers.MsSql` 3.10.0 y la imagen fija `mcr.microsoft.com/mssql/server:2022-CU14-ubuntu-22.04`. Una conexión errónea o un motor no disponible hacen fallar las pruebas; no se omiten ni se cambia a un proveedor en memoria.

Capas existentes: `AceriaApp.Api` es la presentación; `AceriaApp.Hexagonal.Aplicacion` contiene puertos y casos de uso; `AceriaApp.Hexagonal.Dominio` contiene `OrdenHexagonal`; `AceriaApp.Hexagonal.Infraestructura` implementa los adaptadores. La API y los tests de M03 se conservan. Los registros de los catálogos son ejemplos de reglas de diseño; un nombre escrito en un catálogo no registra ni implementa un servicio en el contenedor.

Convención de código: cada bloque indica el archivo y la operación. Crear exige un archivo nuevo; reemplazar exige un archivo existente; insertar exige localizar una sola vez la línea indicada. Los fragmentos de los retos se aplican únicamente en una copia optativa de la etapa terminada.

Preparar Docker Desktop en Windows para los tests SQL (opcional)

La **API se ejecuta en Windows con SQL Server Express LocalDB**. Docker solo es necesario si eliges la variante de pruebas SQL Server Express mediante Testcontainers; **no se despliega aquí la aplicación entera**. Trabaja con PowerShell y realiza estas comprobaciones antes del primer test que necesite SQL Server:

1. Instala Docker Desktop para Windows usando el motor **WSL 2** y contenedores **Linux**; abre Docker Desktop y espera a que indique que el motor está iniciado. Si ya utilizas LocalDB, puedes omitir Docker. La instalación, las redes y los despliegues completos de contenedores se estudiarán por separado.

2. En PowerShell ejecuta `docker version` y `docker compose version`. Comprueba el tipo del servidor con `docker info --format '{{.OSType}}'`: debe devolver `linux`. Si da un error de conexión, inicia Docker Desktop; no continúes.
3. Comprueba el motor con `docker run --rm hello-world`. La primera vez debe descargarse una imagen; no es todavía la prueba de SQL Server. Verifica espacio disponible y memoria suficiente para SQL Server Express.
4. **Opción Docker/Testcontainers:** desde la raíz de una etapa **ya completada**, ejecuta `Remove-Item Env:\ACERIA_TEST_SQLSERVER -ErrorAction SilentlyContinue`, seguido de `dotnet test .\AceriaApp.sln -c Release` (en la primera etapa de M02, usa el `.csproj` de tests si aún no existe la solución). El helper crea un contenedor temporal SQL Server Express 2022, bases aisladas y las elimina. La imagen de SQL Server se descarga automáticamente; **no hace falta un `compose.yml` de SQL**.
5. **Alternativa LocalDB sin Docker:** ejecuta `sqllocaldb start MSSQLLocalDB`, configura `$env:ACERIA_TEST_SQLSERVER = 'Server=(localdb)\MSSQLLocalDB;Database=master;Integrated Security=True;TrustServerCertificate=True'` y ejecuta de nuevo los tests. Ambas vías usan SQL Server real; no se permite sustituirlo por SQLite o InMemory.
6. **Diagnóstico:** si falla Docker, comprueba que Docker Desktop está abierto y en modo Linux; si falla SQL, revisa la salida del test, la cadena de conexión a `master` y los permisos de creación de bases. No consideres una compilación como prueba de persistencia ni ignores pruebas fallidas.

Resultado exigido: `dotnet test` debe terminar con todas las pruebas de la etapa en estado correcto. El comando `docker ps -a` sirve para observar los contenedores temporales durante la ejecución, no para atribuir automáticamente un PASS. Esta preparación no altera el proyecto AceriaApp ni exige Linux como sistema operativo del alumno.

Resultados de comprobación

Punto	Tests de catálogo nuevos	Tests funcionales nuevos	Tests de la etapa	Proyectos
Base M03 3.4	—	—	50	9
4.1	8	4	62	10
4.2	11	4	77	11
4.3	10	3	90	12
4.4	8	7	105	13
4.5	10	3	118	14

Los retos añaden una prueba al proyecto de su punto y a la solución completa, únicamente en la copia optativa. No se acumulan en las etapas obligatorias.

Punto 4.1 — Repository

Encapsular la persistencia sin esconder el contrato del dominio

M04 comienza donde terminó la arquitectura hexagonal de M03: los casos de uso se comunican con el exterior mediante puertos y un adaptador EF Core sobre SQL Server. En este punto estudiaremos Repository para comprender por qué el código de aplicación no debería construir consultas SQL ni depender del DbContext. La interfaz expresa las operaciones que necesita el negocio y la implementación concreta se ocupa de recuperar y guardar entidades sin romper sus reglas.

El alumno conservará el estado 3.4 y añadirá el catálogo didáctico para analizar repositorios, pero el aprendizaje principal se obtendrá al ejecutar un caso de uso con un doble del puerto y con el repositorio SQL real. La misma operación ha de respetar el contrato en ambos entornos. Una prueba de catálogo que clasifica nombres o métodos no acredita esa propiedad: necesitamos observar qué se recupera, qué se guarda y cómo se comporta la API ante un recurso ausente.

Carpeta de trabajo: `M04_PATRONES_ARQUITECTURA/PROYECTO/4.1-repository`.

Objetivo: completar los veinticuatro pasos, conservar los documentos heredados y comprobar el patrón con código y tests ejecutables. La etapa suma 62 pruebas en 10 proyectos.

PASO 4.1.1 — Crear la estructura del módulo 4 y del punto 4.1

La etapa 4.1 debe comenzar con una copia limpia del resultado 3.4, porque el nuevo patrón se incorpora sobre el repositorio SQL y las rutas hexagonales que ya existen. Verificar el inventario de proyectos antes de editar ayuda a atribuir cualquier error al cambio concreto y evita modificar la referencia histórica.

1. Abre la terminal integrada de Visual Studio y localiza `M03_ARQUITECTURA_HEXAGONAL/PROYECTO/3.4-implementacion-aspnet-core`; esa es la etapa completa que vas a continuar.
2. Crea `M04_PATRONES_ARQUITECTURA/PROYECTO/4.1-repository` y copia en ella el contenido de `3.4-implementacion-aspnet-core`; en Windows puedes usar `xcopy` con `/E /I /H /Y`.
3. Excluye `bin`, `obj` y `TestResults`; abre únicamente la copia de `4.1-repository` para que el punto anterior permanezca intacto.
4. Ejecuta `dir` dentro de `4.1-repository` y verifica `AceriaApp.sln`, `README.md`, `src`, `tests`, `docs` y `global.json`.

Verificación concreta. La copia contiene la solución y los 9 proyectos de la etapa anterior; no contiene todavía archivos nuevos de 4.1.

Diagnóstico si falla.

- Usar `copy` en lugar de `xcopy`: `copy` no copia carpetas de forma recursiva.
- Olvidar el flag `/E`: no se copian las subcarpetas vacías.
- Olvidar el flag `/I`: `xcopy` pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag `/Y`: `xcopy` pide confirmación para sobrescribir.
- Copiar desde el punto 3.3 en lugar del 3.4: se pierde el contenido del último punto del módulo 3.

PASO 4.1.2 — Abrir la solución copiada en Visual Studio 2026

Abrir la solución copiada permite trabajar en el punto 4.1 sobre la base del punto 3.4. La compilación verifica que la copia es correcta.

1. Entra en `4.1-repository` y abre `AceriaApp.sln` con Visual Studio; desde Windows puedes ejecutar `start AceriaApp.sln`.
2. Ejecuta `dotnet sln AceriaApp.sln list` y comprueba los 9 proyectos heredados; todos deben existir bajo esta etapa.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`; conserva la salida como comprobación inicial.
4. Si la API está en ejecución, detenla antes de compilar. Continúa cuando la compilación termine sin errores y las rutas no apunten a otra etapa.

Verificación concreta. `dotnet sln AceriaApp.sln list` muestra 9 proyectos y `dotnet build AceriaApp.sln -c Release` termina con `Build succeeded.`.

Diagnóstico si falla.

- Abrir la solución del punto 3.4 en lugar de la del punto 4.1: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener archivos DLL bloqueados por una API en ejecución: detén la aplicación antes de recompilar.

PASO 4.1.3 — Verificar que la carpeta `docs` contiene los documentos del punto 3.4

La etapa conserva los 8 documentos anteriores y añade únicamente su ficha de patrón y su ADR. Las modificaciones de código se escriben en la copia actual.

1. Expande `docs` dentro de la copia de `4.1-repository`.
2. Comprueba los 8 documentos heredados: los ADR-010 a ADR-013 y las fichas de los puntos anteriores.
3. Compara sus nombres y contenido con `docs` de `3.4-implementacion-aspnet-core`. No modifiques ni elimines los documentos anteriores.
4. Confirma que todavía no existen `docs/patron-repository.md` ni `docs/ADR-014-patron-repository.md`; se crearán en este punto.

Verificación concreta. `docs` contiene 8 archivos heredados idénticos; los dos documentos de este punto todavía no existen.

Diagnóstico si falla.

- Modificar alguno de los ADR o documentos existentes: rompe la conservación de la etapa anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 3.4.
- Crear la carpeta `docs` si ya existe: `mkdir` falla.

PASO 4.1.4 — Crear el documento del patrón Repository en `docs`

El documento del patrón Repository es el artefacto de `docs` específico del punto 4.1. No modifica los 8 documentos heredados del punto 3.4.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `docs`.
2. Seleccionar `Add` → `New Item` → `Text File` y nombrar el archivo `patron-repository.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que el archivo aparece en la carpeta `docs` sin subrayados rojos.

Archivo: `docs/patron-repository.md`. **Operación:** Crear.

```
# Patrón Repository — Punto 4.1

## Definición

Abstracción que media entre la capa de dominio o aplicación y la capa de persistencia.

## Responsabilidades

- Recuperar entidades del almacenamiento.
- Añadir entidades al almacenamiento.
- Actualizar o eliminar entidades existentes.

## Lo que no debe hacer

- Contener reglas de negocio.
- Validar invariantes.
- Transformar entidades en DTOs.
- Ejecutar consultas complejas con múltiples JOINs.
```

```
- Gestionar transacciones de múltiples repositorios.

## Repositorio específico frente a genérico

- Específico: interfaz por entidad.
- Genérico: interfaz común para varias entidades.

## Repositorio y puertos de salida

La interfaz del repositorio es el puerto de salida de persistencia; la clase concreta es su adaptador de
infraestructura.

## Repositorio y pruebas con dobles

Los repositorios se sustituyen por dobles en los tests de la aplicación. Este punto utiliza un repositorio en
memoria para verificar el caso de uso sin EF Core y el adaptador relacional para comprobar persistencia.

## Repositorio y unidad de trabajo

El repositorio gestiona una entidad. La unidad de trabajo coordina varias.

## Cuándo usar el patrón

- Dominio complejo: sí.
- Varios repositorios: sí.
- Pruebas unitarias: sí.
- CRUD simple: no.
- Sin lógica de dominio: no.

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Puerto ejecutable | IOrdenRepositorio, en la aplicación |
| Adaptador ejecutable | OrdenRepositorioSqlServer, en infraestructura |
| Operaciones ejecutables | ListarAsync, ObtenerAsync, AgregarAsync, GuardarAsync |
| Consultas complejas | Servicios específicos |
| Validación | Test de coherencia de repositorios |

## Correspondencia con el código ejecutable

El catálogo de este punto describe reglas de diseño y contiene ejemplos de contratos. Sus nombres no implican
servicios registrados. La implementación utiliza los proyectos `AceriaApp.Hexagonal.Aplicacion`,
`AceriaApp.Hexagonal.Dominio` y `AceriaApp.Hexagonal.Infraestructura`, compuestos desde `AceriaApp.Api`.
Comprueba los archivos y los tests funcionales que incorpora el paso de compilación de este punto.
```

Verificación concreta. El archivo `docs/patron-repository.md` existe y contiene las 10 secciones mostradas; abre su vista previa y comprueba todos los apartados.

Diagnóstico si falla.

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: rompe la conservación de la etapa anterior.
- Guardar el archivo en la raíz del punto en lugar de `docs`: rompe la estructura del repositorio.

PASO 4.1.5 — Crear la carpeta `Repository` en el proyecto API

La carpeta `Repository` albergará las clases de validación del patrón Repository. Se crea como carpeta nueva sin tocar las carpetas existentes.

1. En el panel Solution Explorer, hacer clic derecho sobre el proyecto `AceriaApp.Api`.
2. Seleccionar `Add` → `New Folder` y nombrar la carpeta `Repository`.
3. Verificar que la carpeta `Repository` aparece dentro de `AceriaApp.Api` sin afectar a las carpetas existentes.
4. Verificar que no se ha modificado ningún archivo existente del punto 3.4.

Verificación concreta. la carpeta `Repository` existe dentro de `AceriaApp.Api` y las carpetas existentes permanecen intactas.

Diagnóstico si falla.

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: rompe la conservación de la etapa anterior.
- Crear la carpeta con otro nombre: rompe la convención del curso.

PASO 4.1.6 — Crear la clase `Repositorio`

La clase `Repositorio` del catálogo describe capacidades de persistencia para poder analizar su contrato. No es el adaptador EF Core que ejecutan los casos de uso: identifica los dos tipos y su función antes de continuar.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `Repository`.
2. Seleccionar `Add` → `Class` y nombrar la clase `Repositorio.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/Repository/Repositorio.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Repository;

public class Repositorio
{
    public string Nombre { get; init; } = string.Empty;
    public string Entidad { get; init; } = string.Empty;
    public int NumeroDeMetodos { get; init; }
    public bool TieneReglasDeNegocio { get; init; }
    public bool GestionaTransacciones { get; init; }
}
```

Verificación concreta. la clase `Repositorio` compila sin errores y expone cinco propiedades.

Diagnóstico si falla.

- Olvidar el `namespace`: la clase no se resuelve desde otros archivos.
- No inicializar las propiedades de tipo `string` con `string.Empty`: el compilador emite advertencias.
- Usar `set` en lugar de `init`: permite modificar la instancia después de crearla.

PASO 4.1.7 — Crear la clase `ValidadorRepositorios`

`ValidadorRepositorios` permite detectar descripciones que incumplen las reglas del ejercicio. Para entender Repository en ejecución es necesario además ver el puerto `IOrdenRepositorio` y el adaptador `OrdenRepositorioSqlServer` ya construidos en M03.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `Repository`.
2. Seleccionar `Add` → `Class` y nombrar la clase `ValidadorRepositorios.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/Repository/ValidadorRepositorios.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Repository;

public class ValidadorRepositorios
{
    public ResultadoRepositorios Validar(IEnumerable<Repositorio> repositorios)
    {
        var errores = new List<string>();
        var lista = repositorios.ToList();
```

```
        if (lista.Count == 0)
        {
            errores.Add("No hay repositorios declarados.");
        }

        foreach (var repositorio in lista)
        {
            if (string.IsNullOrWhiteSpace(repositorio.Nombre))
            {
                errores.Add("Un repositorio no declara el nombre.");
            }

            if (string.IsNullOrWhiteSpace(repositorio.Entidad))
            {
                errores.Add($"El repositorio '{repositorio.Nombre}' no declara la entidad.");
            }

            if (repositorio.NumeroDeMetodos < 1)
            {
                errores.Add($"El repositorio '{repositorio.Nombre}' no declara métodos.");
            }

            if (repositorio.TieneReglasDeNegocio)
            {
                errores.Add($"El repositorio '{repositorio.Nombre}' contiene reglas de negocio.");
            }

            if (repositorio.GestionaTransacciones)
            {
                errores.Add($"El repositorio '{repositorio.Nombre}' gestiona transacciones.");
            }
        }

        return new ResultadoRepositorios(errores.Count == 0, errores);
    }
}

public record ResultadoRepositorios(bool EsValido, IReadOnlyList<string> Errores);
```

Verificación concreta. la clase `ValidadorRepositorios` compila sin errores y expone un método y un `record`.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<string>` y `IReadOnlyList<string>` no se resuelven.
- Olvidar el `using System.Linq;`: `ToList` no se resuelve.
- No comprobar el caso de lista vacía: el validador devuelve válido sin repositorios.

PASO 4.1.8 — Añadir los `using` necesarios en `ValidadorRepositorios`

Los tipos `List<string>` e `IReadOnlyList<string>` pertenecen a `System.Collections.Generic`. El método `ToList` pertenece a `System.Linq`. En proyectos con `ImplicitUsings` desactivado, hay que añadir los usings explícitos.

1. En el panel Solution Explorer, hacer doble clic en `ValidadorRepositorios.cs`.
2. Inserta las directivas indicadas antes del namespace del archivo; la indicación del archivo identifica la línea exacta.
3. Verificar que no hay subrayados rojos en `List<string>` ni `ToList`.
4. Pulsar `Ctrl+S` para guardar el archivo.

Archivo: `src/AceriaApp.Api/Repository/ValidadorRepositorios.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
namespace AceriaApp.Api.Repository;
```

```
using System.Collections.Generic;
using System.Linq;
```

Verificación concreta. `ValidadorRepositorios.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir solo `System.Collections.Generic` y olvidar `System.Linq`: `ToList` no se resuelve.
- Mezclar directivas using con declaraciones de tipos: coloca las directivas antes del namespace para mantener el orden del archivo.
- Olvidar los usings y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 4.1.9 — Crear la clase `CatalogoRepositorios`

`CatalogoRepositorios` reúne definiciones que la práctica puede comparar sin abrir SQL Server. Es una herramienta para razonar sobre opciones, no el registro de repositorios de la aplicación ni una garantía de persistencia.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `Repository`.
2. Seleccionar `Add` → `Class` y nombrar la clase `CatalogoRepositorios.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/Repository/CatalogoRepositorios.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Repository;

public class CatalogoRepositorios
{
    public static List<Repositorio> RepositoriosPorDefecto()
    {
        return new List<Repositorio>
        {
            new() { Nombre = "IOrdenRepositorio", Entidad = "Orden", NumeroDeMetodos = 5,
TieneReglasDeNegocio = false, GestionaTransacciones = false },
            new() { Nombre = "IInventarioRepositorio", Entidad = "Material", NumeroDeMetodos = 5,
TieneReglasDeNegocio = false, GestionaTransacciones = false },
            new() { Nombre = "IClienteRepositorio", Entidad = "Cliente", NumeroDeMetodos = 4,
TieneReglasDeNegocio = false, GestionaTransacciones = false }
        };
    }
}
```

Verificación concreta. la clase `CatalogoRepositorios` compila sin errores y expone un método estático.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<T>` no se resuelve.
- No inicializar las propiedades de los objetos: el compilador emite advertencias.
- Reutilizar una lista estática mutable: cada llamada del catálogo debe construir una lista independiente.

PASO 4.1.10 — Añadir el `using` necesario en `CatalogoRepositorios`

El tipo `List<T>` pertenece a `System.Collections.Generic`. En proyectos con `ImplicitUsings` desactivado, hay que añadir el using explícito.

1. En el panel Solution Explorer, hacer doble clic en `CatalogoRepositorios.cs`.
2. Inserta las directivas indicadas antes del namespace del archivo; la indicación del archivo identifica la línea exacta.

3. Verificar que no hay subrayados rojos en `List<T>`.
4. Pulsar `Ctrl+S` para guardar el archivo.

Archivo: `src/AceriaApp.Api/Repository/CatalogoRepositorios.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
namespace AceriaApp.Api.Repository;
```

```
using System.Collections.Generic;
```

Verificación concreta. `CatalogoRepositorios.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir `using System.Linq;` en lugar de `System.Collections.Generic`: no resuelve `List<T>`.
- Colocar el using después de una clase: las directivas deben preceder las declaraciones de tipos.
- Olvidar el using y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 4.1.11 — Crear el proyecto de tests del punto 4.1

El punto 4.1 añade un proyecto de tests específico para las clases del patrón Repository. Se crea como proyecto nuevo y conserva las pruebas previas; las actualizaciones explícitas de integración se muestran en su paso correspondiente.

1. En `tests` crea la carpeta `AceriaApp.Repository.Tests` y, dentro, el archivo `AceriaApp.Repository.Tests.csproj` con el XML completo de este paso. En Visual Studio puedes elegir la plantilla xUnit y ajustar el archivo generado.
2. Elimina el archivo de ejemplo `UnitTest1.cs` si lo creó la plantilla; los archivos de pruebas completos se escriben en sus pasos correspondientes.
3. Ejecuta `dotnet sln AceriaApp.sln add tests/AceriaApp.Repository.Tests/AceriaApp.Repository.Tests.csproj` incluso si la solución ya existía y después `dotnet sln AceriaApp.sln list`.
4. Comprueba que el proyecto aparece bajo `tests` y que `dotnet restore tests/AceriaApp.Repository.Tests/AceriaApp.Repository.Tests.csproj` termina correctamente.

Archivo: `tests/AceriaApp.Repository.Tests/AceriaApp.Repository.Tests.csproj`. **Operación:** Crear.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. La solución muestra 10 proyectos, incluido `AceriaApp.Repository.Tests`; el nuevo csproj fija .NET 8, SDK de tests 17.12.0, xUnit 2.9.2 y runner 2.8.2.

Diagnóstico si falla.

- Crear el proyecto dentro de `src` en lugar de `tests`: rompe la estructura.
- Modificar los proyectos de tests del punto 3.4: rompe la conservación de la etapa anterior.

- Usar `dotnet new mstest`: el curso usa xUnit.

PASO 4.1.12 — Añadir la referencia del proyecto de tests a la API

La referencia permite que el proyecto de tests acceda a las clases de `Repository` del proyecto API.

1. Ejecuta `dotnet add tests/AceriaApp.Repository.Tests/AceriaApp.Repository.Tests.csproj reference src/AceriaApp.Api/AceriaApp.Api.csproj` o deja el archivo exactamente como el bloque XML de este paso.
2. Comprueba que la referencia va de tests a API; la API nunca debe referenciar el proyecto de tests.
3. Abre `tests/AceriaApp.Repository.Tests/AceriaApp.Repository.Tests.csproj` y comprueba el único `ProjectReference` a `../../src/AceriaApp.Api/AceriaApp.Api.csproj`.
4. Ejecuta `dotnet build tests/AceriaApp.Repository.Tests/AceriaApp.Repository.Tests.csproj -c Release` y verifica que el proyecto resuelve la API y sus dependencias.

Archivo: `tests/AceriaApp.Repository.Tests/AceriaApp.Repository.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. el archivo `AceriaApp.Repository.Tests.csproj` contiene un `ProjectReference` al proyecto API.

Diagnóstico si falla.

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo `.csproj`: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

PASO 4.1.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

`FluentAssertions` proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

1. Ejecuta `dotnet add tests/AceriaApp.Repository.Tests/AceriaApp.Repository.Tests.csproj package FluentAssertions --version 6.12.2` o reemplaza el csproj por el XML completo indicado.
2. Ejecuta `dotnet list tests/AceriaApp.Repository.Tests/AceriaApp.Repository.Tests.csproj package` y localiza `FluentAssertions 6.12.2`.
3. Comprueba que el paquete se añade al nuevo proyecto de tests y que se mantienen la referencia a la API y las versiones de xUnit.
4. Guarda el archivo y ejecuta `dotnet build tests/AceriaApp.Repository.Tests/AceriaApp.Repository.Tests.csproj -c Release`; las aserciones `Should()` estarán disponibles cuando escribas las pruebas.

Archivo: `tests/AceriaApp.Repository.Tests/AceriaApp.Repository.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. El archivo de proyecto y `dotnet list package` muestran `FluentAssertions 6.12.2`; la compilación finaliza correctamente.

Diagnóstico si falla.

- Instalar una versión preliminar: el curso usa la versión fijada en el archivo de proyecto.
- Olvidar el paquete: `Should()` no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

PASO 4.1.14 — Crear el archivo `ValidadorRepositoriosTests.cs`

Los tests verifican que el validador acepta el catálogo por defecto y rechaza los repositorios que violan las responsabilidades del patrón.

1. En el panel Solution Explorer, expandir `tests` → `AceriaApp.Repository.Tests`.
2. Crear el archivo `ValidadorRepositoriosTests.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `tests/AceriaApp.Repository.Tests/ValidadorRepositoriosTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.Repository;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Repository.Tests;

public class ValidadorRepositoriosTests
{
    private readonly ValidadorRepositorios _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoRepositorios.RepositoriosPorDefecto());

        resultado.EsValido.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinRepositorios_DevuelveError()
    {
        var resultado = _validador.Validar(new List<Repositorio>());

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay repositorios"));
    }

    [Fact]
    public void Validar_SinEntidad_DevuelveError()
```

```
    {
        var repositorios = new List<Repositorio>
        {
            new() { Nombre = "IRepositorioX", Entidad = string.Empty, NumeroDeMetodos = 5 }
        };

        var resultado = _validador.Validar(repositorios);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara la entidad"));
    }

    [Fact]
    public void Validar_ConReglasDeNegocio_DevuelveError()
    {
        var repositorios = new List<Repositorio>
        {
            new() { Nombre = "IRepositorioX", Entidad = "Orden", NumeroDeMetodos = 5, TieneReglasDeNegocio =
true }
        };

        var resultado = _validador.Validar(repositorios);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("reglas de negocio"));
    }

    [Fact]
    public void Validar_ConTransacciones_DevuelveError()
    {
        var repositorios = new List<Repositorio>
        {
            new() { Nombre = "IRepositorioX", Entidad = "Orden", NumeroDeMetodos = 5, GestionaTransacciones =
true }
        };

        var resultado = _validador.Validar(repositorios);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("gestiona transacciones"));
    }
}
```

Verificación concreta. el archivo `ValidadorRepositoriosTests.cs` compila y contiene cinco métodos de test.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.Repository;`: las clases no se resuelven.
- Usar `Assert.True` en lugar de `Should().BeTrue()`: el curso usa FluentAssertions.
- No cubrir el caso de lista vacía: el mensaje de error no se verifica.

PASO 4.1.15 — Crear el archivo `CatalogoRepositoriosTests.cs`

Los tests verifican que el catálogo por defecto contiene los repositorios esperados y que ninguno contiene reglas de negocio ni gestiona transacciones.

1. En el panel Solution Explorer, hacer clic derecho sobre el proyecto `AceriaApp.Repository.Tests`.
2. Seleccionar `Add` → `Class` y nombrar la clase `CatalogoRepositoriosTests.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `tests/AceriaApp.Repository.Tests/CatalogoRepositoriosTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.Repository;
using FluentAssertions;
using Xunit;
```

```
namespace AceriaApp.Repository.Tests;

public class CatalogoRepositoriosTests
{
    [Fact]
    public void RepositoriosPorDefecto_ContieneTresRepositorios()
    {
        var repositorios = CatalogoRepositorios.RepositoriosPorDefecto();

        repositorios.Should().HaveCount(3);
    }

    [Fact]
    public void RepositoriosPorDefecto_NingunoTieneReglasDeNegocio()
    {
        var repositorios = CatalogoRepositorios.RepositoriosPorDefecto();

        repositorios.Should().OnlyContain(r => !r.TieneReglasDeNegocio);
    }

    [Fact]
    public void RepositoriosPorDefecto_NingunoGestionaTransacciones()
    {
        var repositorios = CatalogoRepositorios.RepositoriosPorDefecto();

        repositorios.Should().OnlyContain(r => !r.GestionaTransacciones);
    }
}
```

Verificación concreta. el archivo `CatalogoRepositoriosTests.cs` compila y contiene tres métodos de test.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.Repository;`: las clases no se resuelven.
- Olvidar el `using FluentAssertions;`: `OnlyContain` no se resuelve.
- No verificar las responsabilidades prohibidas: el catálogo podría violar el patrón.

PASO 4.1.16 — Probar el contrato de repositorio y compilar la solución

Las pruebas funcionales deben ejecutar el mismo caso de uso primero con un doble del puerto y después contra una base SQL Server aislada. La primera variante comprueba la aplicación sin infraestructura; la segunda confirma que el adaptador persiste y recupera los datos, incluida una transición del dominio leída desde otro DbContext.

1. Abre cada archivo de integración funcional indicado en este paso. Crea los nuevos y reemplaza únicamente los que se señalen; copia todo el código mostrado.
2. Recorre las referencias: el dominio define sus entidades, la aplicación usa puertos y la infraestructura implementa persistencia; la API compone el flujo.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`; comprueba que todos los proyectos nuevos estén incluidos en la solución.
4. Verifica `Build succeeded.` y `0 Error(s)`. Si hay advertencias, revisa su causa; en el paso siguiente ejecutarás las pruebas del comportamiento completo.

Integración funcional. Conserva las dependencias de los tests de catálogo y añade Testcontainers.MsSql, el helper compartido y WebApplicationFactory para las pruebas funcionales.

Archivo: `tests/AceriaApp.Repository.Tests/AceriaApp.Repository.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings><IsPackable>false</IsPackable><IsTestProject>true</IsTestProject></PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
```

```
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="Testcontainers.MsSql" Version="3.10.0" />
    <PackageReference Include="Microsoft.AspNetCore.Mvc.Testing" Version="8.0.21" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <Compile Include="../Compartido/SqlServerPruebas.cs" Link="SqlServerPruebas.cs" />
  </ItemGroup>
</Project>
```

Compila para .NET 8, igual que los proyectos heredados. Fija el cliente de contenedores para SQL Server; el proveedor EF Core SQL Server llega mediante la referencia a la API. Permite ejecutar peticiones HTTP contra el host real de la API.

Compila el helper heredado que crea y elimina únicamente la base aislada de cada prueba, sin añadir otro proyecto a la solución.

Integración funcional. Ejecuta el mismo puerto con un doble sin EF y con el adaptador EF Core sobre SQL Server. Comprueba normalización, capacidad, lectura entre contextos y persistencia de la transición de dominio. Son pruebas de caracterización del contrato real heredado de M3.

Archivo: `tests/AceriaApp.Repository.Tests/RepositoryFuncionalesTests.cs`. **Operación:** Crear.

```
using AceriaApp.Hexagonal.Aplicacion.CasosUso;
using AceriaApp.Hexagonal.Aplicacion.Puertos;
using AceriaApp.Hexagonal.Dominio;
using AceriaApp.Hexagonal.Infraestructura.Adaptadores;
using AceriaApp.Hexagonal.Infraestructura.Datos;
using AceriaApp.PruebasSqlServer;
using Microsoft.EntityFrameworkCore;
using Xunit;

namespace AceriaApp.Repository.Tests;

public sealed class RepositoryFuncionalesTests
{
    [Fact]
    public async Task Caso_de_uso_funciona_con_un_doble_del_puerto_sin_EF()
    {
        var puerto = new RepositorioEnMemoria();
        var caso = new OrdenesCasoUso(puerto);
        var orden = await caso.CrearAsync("  Axia  ", "Acero", 20);
        var recuperada = await caso.ObtenerAsync(orden.Id);
        Assert.Equal("Axia", recuperada!.Cliente);
        Assert.Equal(20, recuperada.Cantidad);
    }

    [Fact]
    public async Task Caso_de_uso_no_persiste_lotes_que_superan_la_capacidad()
    {
        var puerto = new RepositorioEnMemoria();
        var caso = new OrdenesCasoUso(puerto);
        await Assert.ThrowsAsync<ArgumentOutOfRangeException>(() => caso.CrearAsync("Axia", "Acero", 501));
        Assert.Empty(await caso.ListarAsync());
    }

    [Fact]
    public async Task Adaptador_relacional_cumple_contrato_crear_y_recuperar_en_otro_contexto()
    {
        await using var store = BaseSqlServer.Crear();
        var options = new
DbContextOptionsBuilder<HexagonalDbContext>().UseSqlServer(store.ConnectionString).Options;
        var orden = new OrdenHexagonal("Axia", "Acero", 20);
        await using (var db = new HexagonalDbContext(options))
        {
            Assert.Equal("Microsoft.EntityFrameworkCore.SqlServer", db.Database.ProviderName);
            await db.Database.EnsureCreatedAsync();
            await new OrdenRepositorioSqlServer(db).AgregarAsync(orden);
        }
        await using var lectura = new HexagonalDbContext(options);
        var puerto = new OrdenRepositorioSqlServer(lectura);
        var recuperada = await puerto.ObtenerAsync(orden.Id);
```

```
        Assert.Equal("Axia", recuperada!.Cliente);
        Assert.Equal(20, recuperada.Cantidad);
        Assert.Null(await puerto.ObtenerAsync(Guid.NewGuid()));
    }

    [Fact]
    public async Task Adaptador_persiste_la_transicion_de_dominio()
    {
        await using var store = BaseSqlServer.Crear();
        var options = new
DbContextOptionsBuilder<HexagonalDbContext>().UseSqlServer(store.ConnectionString).Options;
        Guid id;
        await using (var db = new HexagonalDbContext(options))
        {
            Assert.Equal("Microsoft.EntityFrameworkCore.SqlServer", db.Database.ProviderName);
            await db.Database.EnsureCreatedAsync();
            var caso = new OrdenesCasoUso(new OrdenRepositorioSqlServer(db));
            id = (await caso.CrearAsync("Axia", "Acero", 20)).Id;
            await caso.FinalizarAsync(id);
        }
        await using var lectura = new HexagonalDbContext(options);
        Assert.Equal("Finalizada", (await new OrdenRepositorioSqlServer(lectura).ObtenerAsync(id))!.Estado);
    }

    private sealed class RepositorioEnMemoria : IOrdenRepositorio
    {
        private readonly Dictionary<Guid, OrdenHexagonal> ordenes = new();
        public Task<IReadOnlyList<OrdenHexagonal>> ListarAsync(CancellationToken ct = default) =>
            Task.FromResult<IReadOnlyList<OrdenHexagonal>>(ordenes.Values.ToList());
        public Task<OrdenHexagonal?> ObtenerAsync(Guid id, CancellationToken ct = default) =>
            Task.FromResult(ordenes.GetValueOrDefault(id));
        public Task AgregarAsync(OrdenHexagonal orden, CancellationToken ct = default)
        { ordenes.Add(orden.Id, orden); return Task.CompletedTask; }
        public Task GuardarAsync(OrdenHexagonal orden, CancellationToken ct = default)
        { ordenes[orden.Id] = orden; return Task.CompletedTask; }
    }
}
```

El doble del puerto comprueba lógica de aplicación sin conectarse a una base. Una cantidad fuera de capacidad se rechaza antes de persistir. En `await using var store = BaseSqlServer.Crear();` se crea una base SQL Server exclusiva de esta prueba y la elimina al abandonar el ámbito.

Configura el mismo proveedor relacional que usa el adaptador en producción. Comprueba explícitamente el proveedor de la prueba de integración. Una conexión y un contexto independientes comprueban persistencia real.

Ejecuta la transición de dominio antes de recuperarla desde otro contexto.

Verificación concreta. La solución completa compila, incluidos el código de integración y sus pruebas. Las siguientes ejecuciones de tests comprobarán el comportamiento de los patrones.

Diagnóstico si falla.

- Tener la API en ejecución: el archivo `.dll` está bloqueado y la compilación falla.
- No restaurar paquetes: `dotnet build` restaura por defecto, pero si hay un error de red, hay que ejecutar `dotnet restore` primero.
- Usar una versión distinta del SDK: `dotnet --version` debe devolver `8.x.x`.

PASO 4.1.17 — Ejecutar todos los tests del punto 4.1

Ejecutar los tests del punto 4.1 verifica que las clases del patrón Repository funcionan correctamente.

1. Ejecuta `dotnet test tests/AceriaApp.Repository.Tests/AceriaApp.Repository.Tests.csproj -c Release` desde la raíz de esta etapa.
2. Comprueba un total de 12 pruebas: 8 de catálogo y 4 funcionales, con cero fallos y cero omisiones.

3. Si falla una prueba, abre su mensaje de aserción y el archivo indicado; corrige el comportamiento antes de continuar.
4. Repite el mismo comando tras la corrección y conserva el resultado de la ejecución válida.

Verificación concreta. El proyecto `AceriaApp.Repository.Tests` termina con `Failed: 0, Passed: 12, Skipped: 0, Total: 12`.

Diagnóstico si falla.

- Ejecutar otro proyecto: comprueba la ruta y el nombre de la asamblea de pruebas antes de interpretar los resultados.
- No compilar antes: `dotnet test` compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

PASO 4.1.18 — Ejecutar todos los tests de la solución

Ejecutar todos los tests de la solución verifica que los tests de los puntos anteriores y del punto 4.1 pasan conjuntamente.

1. Ejecuta `dotnet build AceriaApp.sln -c Release` para producir todas las asambleas de esta etapa.
2. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build --no-restore` para ejecutar todas las pruebas acumuladas.
3. Suma los resultados de los 6 proyectos de tests: deben ser 62 pruebas aprobadas, sin fallos ni omisiones.
4. Comprueba que las pruebas heredadas de M03 y los puntos anteriores de M04 siguen presentes; no borres tests para obtener un resultado verde.

Verificación concreta. La suma de las líneas de resultados es `Failed: 0`, `Passed: 62`, `Skipped: 0` y `Total: 62`. Dotnet muestra un resumen por proyecto, no necesariamente una única línea global.

Diagnóstico si falla.

- No haber compilado antes: `--no-build` falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
- Eliminar pruebas anteriores o cambiarlas sin una instrucción explícita: se pierde la cobertura de los contratos heredados.

PASO 4.1.19 — Crear el README del punto 4.1

El README del punto 4.1 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 3.4.

1. En la carpeta del punto `4.1-repository`, hacer clic derecho y seleccionar `Add` → `New Item` → `Text File`.
2. Nombrar el archivo `README.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que el README del punto 3.4 no se ha modificado.

Archivo: `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 4.1 — Repository

## Objetivo
Comprender el patrón Repository y añadir al proyecto las clases de validación de repositorios.

La práctica conecta además el patrón con los puertos, adaptadores y pruebas de integración que aparecen en
este manual.

## Estado del proyecto
- Solución `AceriaApp.sln` con 10 proyectos.
```

```
- Documento `docs/patron-repository.md` con las responsabilidades y las prohibiciones.
- Clases `Repositorio`, `ValidadorRepositorios` y `CatalogoRepositorios` en `src/AceriaApp.Api/Repository/`.
- 8 tests nuevos de catálogo y 4 funcionales en `tests/AceriaApp.Repository.Tests`.
- Los tests acumulados suman 62 pruebas: 8 de catálogo nuevas y 4 funcionales nuevas en este punto.

## Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio 2026.
2. Verificar que SQL Server Express está arrancado.
3. Configurar las conexiones SQL Server `AceriaApp` y `AceriaHexagonal` en el entorno local.
4. Ejecutar `dotnet run --project src/AceriaApp.Api/AceriaApp.Api.csproj --urls http://localhost:5000` desde
la raíz de esta etapa.
5. Navegar a `http://localhost:5000/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

## Qué se ha añadido respecto al punto anterior
- Documento `docs/patron-repository.md`.
- Carpeta `Repository` en el proyecto API con tres clases nuevas.
- Proyecto `tests/AceriaApp.Repository.Tests` con 8 tests de catálogo y 4 funcionales nuevos.
- Se conserva intacta la carpeta del punto 3.4.

## Documentación
- [ADR-010](docs/ADR-010-capas-aplicacion.md) a [ADR-013](docs/ADR-013-implementacion-aspnet-core.md)
(heredados).
- [Patrón Repository](docs/patron-repository.md).

## Verificación ejecutable

Los catálogos describen ejemplos de diseño. El comportamiento de las clases reales se comprueba además con
los tests funcionales del punto. Ejecuta `dotnet test AceriaApp.sln -c Release` y exige cero fallos y cero
omisiones.
```

Verificación concreta. El archivo `README.md` existe y contiene las 6 secciones mostradas; abre su vista previa y comprueba todos los apartados.

Diagnóstico si falla.

- Modificar el README del punto 3.4: rompe la conservación de la etapa anterior.
- Omitir la sección `Qué se ha añadido respecto al punto anterior`: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 3.4: rompe la separación.

PASO 4.1.20 — Crear el ADR-014 del punto 4.1

El ADR-014 documenta la decisión de adoptar el patrón Repository. Es un artefacto nuevo del punto 4.1 que no modifica los ADR existentes.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `docs`.
2. Seleccionar `Add` → `New Item` → `Text File` y nombrar el archivo `ADR-014-patron-repository.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que los ADR-010 a ADR-013 no se han modificado.

Archivo: `docs/ADR-014-patron-repository.md`. **Operación:** Crear.

```
# ADR-014: Adopción del patrón Repository

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha cerrado el módulo 3 con una arquitectura hexagonal completa. Es necesario definir la
abstracción de acceso a datos que usará la capa de aplicación.

## Decisión
Se adopta el patrón Repository con repositorios específicos por entidad. El contrato ejecutable
IOrdenRepositorio expone ListarAsync, ObtenerAsync, AgregarAsync y GuardarAsync. No contiene una operación
Delete. El catálogo estudia ejemplos de repositorios con cinco o cuatro métodos; esas cantidades no se
presentan como una inspección del contrato real. Los repositorios no contienen reglas de negocio, no validan
invariantes, no transforman entidades en DTOs, no ejecutan consultas complejas y no gestionan transacciones.
Las consultas complejas se delegan en servicios específicos. La coherencia de los ejemplos se valida con el
```

```
catálogo y el contrato real se comprueba con cuatro pruebas funcionales: doble sin EF, capacidad, lectura
desde otro contexto y persistencia de la transición.

## Alternativas consideradas
- Acceso directo a EF Core desde la aplicación: descartado por acoplar la aplicación a la tecnología.
- Repositorio genérico único: descartado por pérdida de expresividad.
- Sin repositorio, uso de DbContext directamente: descartado por dificultad de pruebas.

## Consecuencias
- Positivas: independencia de la tecnología, pruebas con dobles, separación de responsabilidades.
- Negativas: la interfaz del repositorio debe mantenerse simple.
- Mitigación: validación automática con el validador.
```

Verificación concreta. El archivo `docs/ADR-014-patron-repository.md` existe y contiene las 5 secciones mostradas; abre su vista previa y comprueba todos los apartados.

Diagnóstico si falla.

- Modificar los ADR anteriores: rompe la conservación de la etapa anterior.
- Omitir la sección `Alternativas consideradas`: el ADR pierde valor.
- Nombrar el archivo sin el prefijo `ADR-014`: rompe la convención.

PASO 4.1.21 — Verificar que los documentos heredados y la etapa 3.4 permanecen intactos

Las decisiones previas se conservan como registros históricos. Los nuevos ADR explican la evolución y las modificaciones de integración se aplican solo a la copia de la etapa actual.

1. Vuelve a la raíz del repositorio; las rutas siguientes se interpretan desde allí.
2. En CMD ejecuta `for %f in (M03_ARQUITECTURA_HEXAGONAL\PROYECTO\3.4-implementacion-aspnet-core\docs\*.md) do fc /L "%f" "M04_PATRONES_ARQUITECTURA\PROYECTO\4.1-repository\docs\%~nxf"`. En PowerShell puedes comparar `Get-FileHash` para cada par de documentos.
3. Comprueba 8 comparaciones y exige igualdad en todas; se conserva exactamente el contenido de los documentos heredados.
4. Si una comparación falla, restaura el documento de la etapa actual desde `3.4-implementacion-aspnet-core`. Verifica también que la carpeta del punto anterior no tiene modificaciones.

Verificación concreta. Los 8 documentos heredados coinciden byte a byte y el punto anterior permanece intacto.

Diagnóstico si falla.

- Usar `comp` en lugar de `fc`: `comp` compara binarios y no muestra diferencias de texto.
- Omitir `/L` cuando se desea una comparación de texto: indica el modo explícitamente y revisa cada resultado.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

PASO 4.1.22 — Verificar la estructura del punto 4.1

La verificación estructural confirma que el punto 4.1 añade archivos nuevos conservando los documentos heredados.

1. Escribir `tree /F M04_PATRONES_ARQUITECTURA\PROYECTO\4.1-repository` y pulsar Enter.
2. Verificar que la salida muestra `AceriaApp.sln`, `README.md`, `src\AceriaApp.Api\Repository` con las tres clases nuevas, `tests\AceriaApp.Repository.Tests` con los dos archivos de catálogo y el archivo de tests funcionales y `docs` con el documento del patrón y el ADR-014. Comprueba además todos los archivos de integración de las carpetas Patrones y los cambios de composición indicados en este punto.
3. Verificar que las carpetas existentes del punto 3.4 permanecen intactas.

4. Verificar que no existe ninguna carpeta `labs` ni ningún archivo `metaprompt.md` ni `verificacion.md`.

Verificación concreta. la estructura contiene los archivos nuevos y conserva los del punto 3.4.

Diagnóstico si falla.

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 3.4: viola la regla de no modificar.
- Omitir la carpeta `Repository`: los tests no encuentran las clases.

PASO 4.1.23 — Verificar el `.gitignore` del punto 4.1

El `.gitignore` se hereda del punto 3.4 sin modificaciones. La verificación confirma que no se ha tocado.

1. Verificar que el archivo `.gitignore` existe en la carpeta del punto 4.1 (copiado del punto 3.4).
2. Abrir el archivo `.gitignore` con Visual Studio 2026.
3. Verificar que contiene las líneas `**/bin/`, `**/obj/`, `.vs/`, `*.user`, `TestResults/` y `*.trx`.
4. Verificar que no se ha modificado respecto al punto 3.4. Pulsar `Ctrl+S` si no hay cambios y cerrar el archivo.

Verificación concreta. el `.gitignore` existe y contiene las seis líneas sin modificaciones.

Diagnóstico si falla.

- Modificar el `.gitignore` copiado del punto 3.4: rompe la conservación de la etapa anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.
- Crear el `.gitignore` con otro nombre: el archivo no se reconoce.

PASO 4.1.24 — Hacer el commit del punto 4.1 y verificar el estado final

El commit permite comparar esta etapa con la anterior. Si has realizado un commit por punto desde 1.1, el recorrido suma 14 commits de puntos; otros commits legítimos pueden aumentar el historial.

1. Desde la raíz del repositorio ejecuta `git status` y revisa que solo aparezcan los archivos de esta etapa y los entregables que has trabajado.
2. Ejecuta `git add .` después de verificar que `bin/`, `obj/`, `TestResults/` y credenciales están excluidos.
3. Ejecuta `git commit -m "Punto 4.1: Repository"` con `user.name` y `user.email` configurados.
4. Ejecuta `git log --oneline` y `git status`; comprueba que se registró el commit de este punto y que no quedan cambios pendientes de esta actividad.

Verificación concreta. El historial incluye el commit de 4.1 y `git status` no muestra cambios pendientes de esta actividad; no se exige un número rígido de commits a un repositorio con más trabajo.

Diagnóstico si falla.

- No tener configurado `user.name` y `user.email`: `git commit` falla.
- Añadir la carpeta `bin/` y `obj/` al commit: el `.gitignore` lo evita.
- Hacer commit antes de `git add .`: no hay cambios en el índice.

Reto de ampliación (optativo)

Trabaja en una copia independiente de la etapa obligatoria, por ejemplo `4.1-repository-reto`. Mantén intacta la carpeta obligatoria y todos sus documentos. Los tres fragmentos siguientes añaden un contrato, un cálculo y una prueba al proyecto del punto.

Reto: añadir un método `Resumen` a `ValidadorRepositorios` que devuelva el número total de repositorios, el número total de métodos y el número medio de métodos por repositorio, y un test que verifique el cálculo con el catálogo por defecto.

Paso 1: En `ValidadorRepositorios.cs`, añadir el siguiente `record` al final del archivo:

Archivo: `src/AceriaApp.Api/Repository/ValidadorRepositorios.cs`. **Operación:** Modificar.

Punto de inserción: después de este texto exacto, que debe aparecer una sola vez:

```
public record ResultadoRepositorios(bool EsValido, IReadOnlyList<string> Errores);
```

```
public record ResumenRepositorios(
    int Total,
    int TotalMetodos,
    double MediaMetodos);
```

Paso 2: Explicar el propósito del fragmento y las decisiones que contiene:

La declaración `public record ResumenRepositorios(` declara un contrato inmutable. Número total de repositorios. Número total de métodos. Media de métodos por repositorio.

Paso 3: En `ValidadorRepositorios.cs`, añadir el siguiente método estático dentro de la clase:

Archivo: `src/AceriaApp.Api/Repository/ValidadorRepositorios.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
}

public record ResultadoRepositorios(bool EsValido, IReadOnlyList<string> Errores);
```

Indenta el fragmento con cuatro espacios dentro de la clase; conserva el cierre de clase que aparece en el texto de inserción.

```
public static ResumenRepositorios Resumen(IEnumerable<Repositorio> repositorios)
{
    var lista = repositorios.ToList();
    var total = lista.Count;
    var metodos = lista.Sum(r => r.NumeroDeMetodos);
    var media = total == 0 ? 0 : (double)metodos / total;
    return new ResumenRepositorios(total, metodos, media);
}
```

Paso 4: Explicar el propósito del fragmento y las decisiones que contiene:

La declaración `public static ResumenRepositorios Resumen(IEnumerable<Repositorio> repos` declara el método. Materializa la lista. Cuenta los repositorios. Suma los métodos. Calcula la media con protección contra división por cero. Devuelve el resumen.

Paso 5: Guardar el archivo con `Ctrl+S`.

Paso 6: En `ValidadorRepositoriosTests.cs`, añadir el siguiente test:

Archivo: `tests/AceriaApp.Repository.Tests/ValidadorRepositoriosTests.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
}
```

Indenta el fragmento con cuatro espacios dentro de la clase; conserva el cierre de clase que aparece en el texto de inserción.

```
[Fact]
public void Resumen_ConCatalogoPorDefecto_CalculaMedia()
{
    var resumen = ValidadorRepositorios.Resumen(CatalogoRepositorios.RepositoriosPorDefecto());

    resumen.Total.Should().Be(3);
    resumen.TotalMetodos.Should().Be(14);
    resumen.MediaMetodos.Should().BeApproximately(4.67, 0.01);
}
```

Paso 7: Interpretar el test y explicar qué comportamiento acredita:

Marca el método como test de xUnit. Declara el test. Invoca el método. Verifica el total. Verifica el total de métodos. Verifica la media.

Paso 8: Ejecuta `dotnet test tests/AceriaApp.Repository.Tests/AceriaApp.Repository.Tests.csproj -c Release` en la copia optativa. Deben pasar 13 tests del proyecto, incluido el nuevo resumen. Ejecuta después `dotnet test AceriaApp.sln -c Release` y exige 63 pruebas aprobadas.

Paso 9: Comprueba que solo cambian `ValidadorRepositorios.cs` y `ValidadorRepositoriosTests.cs` en la copia optativa; la etapa obligatoria `4.1-repository` y el punto anterior deben permanecer idénticos.

Paso 10: Hacer commit con `git commit -m "Reto 4.1: método Resumen en ValidadorRepositorios"`.

Verificación: `Passed: 13` en el proyecto del punto y 63 pruebas aprobadas en la solución de la copia optativa, sin fallos ni omisiones.

Resultado verificable

Al finalizar este punto, el alumno debe tener:

- La carpeta `M04_PATRONES_ARQUITECTURA\PROYECTO/4.1-repository/` con la solución `AceriaApp.sln` que contiene 10 proyectos.
- El documento `docs/patron-repository.md` con las responsabilidades y las prohibiciones.
- El documento `docs/ADR-014-patron-repository.md` con la decisión sobre el patrón Repository.
- Las clases `Repositorio`, `ValidadorRepositorios` y `CatalogoRepositorios` en `src/AceriaApp.Api/Repository/`.
- El proyecto `tests/AceriaApp.Repository.Tests` añade 8 tests de catálogo y 4 funcionales; el reto añade uno más.
- `dotnet test AceriaApp.sln -c Release` ejecuta 62 pruebas en la etapa obligatoria y 63 en una copia con su reto; ninguna falla ni se omite.
- Los ADR-010 a ADR-013 del punto 3.4 sin modificar.
- Un commit revisado para el punto 4.1, con el historial previo conservado.

Continuidad del aprendizaje

El punto 4.1 ha definido el patrón Repository como abstracción entre la capa de aplicación y la capa de persistencia. El proyecto AceriaApp dispone ahora de un validador automático de repositorios que comprueba las responsabilidades y las prohibiciones del patrón, y 8 tests nuevos de catálogo más 4 funcionales. Esta base permite avanzar hacia el punto 4.2, donde se estudiará la unidad de trabajo como mecanismo para coordinar las operaciones de varios repositorios en una única transacción.

Punto 4.2 — Unit of Work

Entender quién confirma una operación y qué significa su transacción

EF Core dispone de una unidad de trabajo implícita: un DbContext sigue cambios y puede confirmarlos mediante SaveChangesAsync. Introducir un patrón Unit of Work sobre esta infraestructura solo resulta útil si aclara quién controla la transacción y evita confirmaciones dispersas. En AceriaApp el caso de uso debe mantener el límite de la operación dentro de una única base de datos, sin prometer atomicidad entre microservicios que todavía no participan.

La práctica separa el catálogo de unidades de trabajo de la implementación que utiliza el contexto real. Reconstruiremos los tipos y sus pruebas y comprobaremos qué ocurre cuando una regla de negocio rechaza un cambio antes de guardar. Los ensayos con SQL Server son los que muestran una confirmación efectiva; un test de coherencia del catálogo únicamente verifica las condiciones que le hemos programado.

Carpeta de trabajo: `M04_PATRONES_ARQUITECTURA/PROYECTO/4.2-unit-of-work`.

Objetivo: completar los veinticuatro pasos, conservar los documentos heredados y comprobar el patrón con código y tests ejecutables. La etapa suma 77 pruebas en 11 proyectos.

PASO 4.2.1 — Crear la carpeta del punto 4.2 a partir del punto 4.1

La copia preserva una etapa autónoma y reproducible. 4.1-repository aporta 62 tests aprobados y 10 proyectos antes de añadir este punto.

1. Abre la terminal integrada de Visual Studio y localiza `M04_PATRONES_ARQUITECTURA/PROYECTO/4.1-repository`; esa es la etapa completa que vas a continuar.
2. Crea `M04_PATRONES_ARQUITECTURA/PROYECTO/4.2-unit-of-work` y copia en ella el contenido de `4.1-repository`; en Windows puedes usar `xcopy` con `/E /I /H /Y`.
3. Excluye `bin`, `obj` y `TestResults`; abre únicamente la copia de `4.2-unit-of-work` para que el punto anterior permanezca intacto.
4. Ejecuta `dir` dentro de `4.2-unit-of-work` y verifica `AceriaApp.sln`, `README.md`, `src`, `tests`, `docs` y `global.json`.

Verificación concreta. La copia contiene la solución y los 10 proyectos de la etapa anterior; no contiene todavía archivos nuevos de 4.2.

Diagnóstico si falla.

- Usar `copy` en lugar de `xcopy`: `copy` no copia carpetas de forma recursiva.
- Olvidar el flag `/E`: no se copian las subcarpetas vacías.
- Olvidar el flag `/I`: `xcopy` pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag `/Y`: `xcopy` pide confirmación para sobrescribir.

PASO 4.2.2 — Abrir la solución copiada en Visual Studio 2026

Abrir la solución copiada permite trabajar en el punto 4.2 sobre la base del punto 4.1. La compilación verifica que la copia es correcta.

1. Entra en `4.2-unit-of-work` y abre `AceriaApp.sln` con Visual Studio; desde Windows puedes ejecutar `start AceriaApp.sln`.
2. Ejecuta `dotnet sln AceriaApp.sln list` y comprueba los 10 proyectos heredados; todos deben existir bajo esta etapa.

3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`; conserva la salida como comprobación inicial.
4. Si la API está en ejecución, detenla antes de compilar. Continúa cuando la compilación termine sin errores y las rutas no apunten a otra etapa.

Verificación concreta. `dotnet sln AceriaApp.sln list` muestra 10 proyectos y `dotnet build AceriaApp.sln -c Release` termina con `Build succeeded.`.

Diagnóstico si falla.

- Abrir la solución del punto 4.1 en lugar de la del punto 4.2: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener archivos DLL bloqueados por una API en ejecución: detén la aplicación antes de recompilar.

PASO 4.2.3 — Verificar que la carpeta `docs` contiene los documentos del punto 4.1

La etapa conserva los 10 documentos anteriores y añade únicamente su ficha de patrón y su ADR. Las modificaciones de código se escriben en la copia actual.

1. Expande `docs` dentro de la copia de `4.2-unit-of-work`.
2. Comprueba los 10 documentos heredados: los ADR-010 a ADR-014 y las fichas de los puntos anteriores.
3. Compara sus nombres y contenido con `docs` de `4.1-repository`. No modifiques ni elimines los documentos anteriores.
4. Confirma que todavía no existen `docs/patron-unit-of-work.md` ni `docs/ADR-015-patron-unit-of-work.md`; se crearán en este punto.

Verificación concreta. `docs` contiene 10 archivos heredados idénticos; los dos documentos de este punto todavía no existen.

Diagnóstico si falla.

- Modificar alguno de los ADR o documentos existentes: rompe la conservación de la etapa anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 4.1.
- Crear la carpeta `docs` si ya existe: `mkdir` falla.

PASO 4.2.4 — Crear el documento del patrón Unit of Work en `docs`

El documento del patrón Unit of Work es el artefacto de `docs` específico del punto 4.2. No modifica los 10 documentos existentes del punto 4.1.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `docs`.
2. Seleccionar `Add` → `New Item` → `Text File` y nombrar el archivo `patron-unit-of-work.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que el archivo aparece en la carpeta `docs` sin subrayados rojos.

Archivo: `docs/patron-unit-of-work.md`. **Operación:** Crear.

```
# Patrón Unit of Work — Punto 4.2

## Definición

Abstracción que coordina las operaciones de varios repositorios en una única transacción.

## Responsabilidades
```

```
- Mantener el seguimiento de cambios.
- Coordinar la persistencia.
- Exponer la operación de confirmación.

## Lo que no debe hacer

- Contener reglas de negocio.
- Conocer entidades concretas.
- Transformar entidades en DTOs.
- Ejecutar consultas complejas.
- Gestionar la lógica de los repositorios.

## Relación con Repository

El repositorio gestiona una entidad. La unidad de trabajo coordina varios repositorios.

## EF Core como unidad de trabajo

El `DbContext` implementa el patrón Unit of Work con `SaveChanges` y `SaveChangesAsync`.

## Unidad de trabajo personalizada

Envuelve el `DbContext` y expone solo las operaciones necesarias para la aplicación.

## Transacciones distribuidas

La unidad de trabajo se limita a la transacción local. La transacción distribuida requiere patrones
adicionales.

## Cuándo usar el patrón

- Operación con varias entidades: sí.
- Coordinación de repositorios: sí.
- Desacoplamiento de transacciones: sí.
- Operación con una entidad: un envoltorio adicional es opcional; resulta útil si se desea hacer explícito el
punto de confirmación o sustituirlo en pruebas.
- Sin requisito de atomicidad: no.

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Puerto de salida | IUnitOfWork |
| Adaptador de salida | EfUnitOfWork |
| Repositorio del nuevo flujo | OrdenTrabajoRepositorio prepara los cambios |
| Unidad de trabajo | Confirma los cambios |
| Validación | Test de coherencia |

## Correspondencia con el código ejecutable

El catálogo de este punto describe reglas de diseño y contiene ejemplos de contratos. Sus nombres no implican
servicios registrados. La implementación utiliza los proyectos `AceriaApp.Hexagonal.Aplicacion`,
`AceriaApp.Hexagonal.Dominio` y `AceriaApp.Hexagonal.Infraestructura`, compuestos desde `AceriaApp.Api`.
Comprueba los archivos y los tests funcionales que incorpora el paso de compilación de este punto.


## Variantes del catálogo y del contrato ejecutable

El catálogo de diseño de este punto incluye unidades que declaran confirmación y reversión explícitas. Sus
dos registros y el resumen de seis repositorios son datos del ejercicio de validación. La implementación
ejecutable utiliza `IUnitOfWork.CommitAsync` y `EfUnitOfWork`, con un repositorio para las órdenes; la prueba
del lote coordina dos entidades y la transacción pertenece al contexto. Una unidad de trabajo puede coordinar
varios repositorios que compartan ese contexto, pero este punto no implementa repositorios ficticios de
inventario o clientes.
```

Verificación concreta. El archivo `docs/patron-unit-of-work.md` existe y contiene las 11 secciones mostradas; abre su vista previa y comprueba todos los apartados.

Diagnóstico si falla.

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: rompe la conservación de la etapa anterior.

- Guardar el archivo en la raíz del punto en lugar de `docs`: rompe la estructura del repositorio.

PASO 4.2.5 — Crear la carpeta `UnitOfWork` en el proyecto API

La carpeta `UnitOfWork` albergará las clases de validación del patrón Unit of Work. Se crea como carpeta nueva sin tocar las carpetas existentes.

1. En el panel Solution Explorer, hacer clic derecho sobre el proyecto `AceriaApp.Api`.
2. Seleccionar `Add` → `New Folder` y nombrar la carpeta `UnitOfWork`.
3. Verificar que la carpeta `UnitOfWork` aparece dentro de `AceriaApp.Api` sin afectar a las carpetas existentes.
4. Verificar que no se ha modificado ningún archivo existente del punto 4.1.

Verificación concreta. la carpeta `UnitOfWork` existe dentro de `AceriaApp.Api` y las carpetas existentes permanecen intactas.

Diagnóstico si falla.

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: rompe la conservación de la etapa anterior.
- Crear la carpeta con otro nombre: rompe la convención del curso.

PASO 4.2.6 — Crear la clase `UnidadDeTrabajo`

`UnidadDeTrabajo` modela la responsabilidad de confirmar cambios. En EF Core esta responsabilidad ya se manifiesta en `SaveChangesAsync`; una abstracción adicional debe explicitar el límite transaccional, no introducir una segunda confirmación que deje estados parciales.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `UnitOfWork`.
2. Seleccionar `Add` → `Class` y nombrar la clase `UnidadDeTrabajo.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/UnitOfWork/UnidadDeTrabajo.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.UnitOfWork;

public class UnidadDeTrabajo
{
    public string Nombre { get; init; } = string.Empty;
    public int RepositoriosCoordinados { get; init; }
    public bool ExponeCommit { get; init; }
    public bool ExponeRollback { get; init; }
    public bool ContieneReglasDeNegocio { get; init; }
    public bool ConoceEntidadesConcretas { get; init; }
}
```

Verificación concreta. la clase `UnidadDeTrabajo` compila sin errores y expone seis propiedades.

Diagnóstico si falla.

- Olvidar el `namespace`: la clase no se resuelve desde otros archivos.
- No inicializar `Nombre` con `string.Empty`: el compilador emite advertencia de nulabilidad.
- Usar `set` en lugar de `init`: permite modificar la instancia después de crearla.

PASO 4.2.7 — Crear la clase `ValidadorUnidadesDeTrabajo`

El validador del catálogo estudia propiedades de una unidad de trabajo declarada. Sus pruebas no pueden demostrar que una transacción SQL se confirmó o se revirtió: esa evidencia exige comprobar el repositorio y la base real.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `UnitOfWork`.
2. Seleccionar `Add` → `Class` y nombrar la clase `ValidadorUnidadesDeTrabajo.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Alcance de este validador. `ExponeRollback` es una condición de la variante de diseño representada por los datos del catálogo. No es una exigencia universal del patrón ni una inspección de la interfaz real. El contrato ejecutable de esta práctica solo expone `IUnitOfWork.CommitAsync`; EF Core revierte el lote fallido y un contexto desechado descarta los cambios que no se confirmaron. El validador evalúa sus ejemplos de metadatos, mientras los tests funcionales comprueban esa implementación mínima.

Archivo: `src/AceriaApp.Api/UnitOfWork/ValidadorUnidadesDeTrabajo.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.UnitOfWork;

public class ValidadorUnidadesDeTrabajo
{
    public ResultadoUnidadesDeTrabajo Validar(IEnumerable<UnidadDeTrabajo> unidades)
    {
        var errores = new List<string>();
        var lista = unidades.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay unidades de trabajo declaradas.");
        }

        foreach (var unidad in lista)
        {
            if (string.IsNullOrWhiteSpace(unidad.Nombre))
            {
                errores.Add("Una unidad de trabajo no declara el nombre.");
            }

            if (unidad.RepositoriosCoordinados < 1)
            {
                errores.Add($"La unidad '{unidad.Nombre}' no coordina ningún repositorio.");
            }

            if (!unidad.ExponeCommit)
            {
                errores.Add($"La unidad '{unidad.Nombre}' no expone la operación de confirmación.");
            }

            if (!unidad.ExponeRollback)
            {
                errores.Add($"La unidad '{unidad.Nombre}' no expone la operación de reversión.");
            }

            if (unidad.ContieneReglasDeNegocio)
            {
                errores.Add($"La unidad '{unidad.Nombre}' contiene reglas de negocio.");
            }

            if (unidad.ConoceEntidadesConcretas)
            {
                errores.Add($"La unidad '{unidad.Nombre}' conoce entidades concretas.");
            }
        }

        return new ResultadoUnidadesDeTrabajo(errores.Count == 0, errores);
    }
}
```

```
public record ResultadoUnidadesDeTrabajo(bool EsValida, IReadOnlyList<string> Errores);
```

Verificación concreta. la clase `ValidadorUnidadesDeTrabajo` compila sin errores y expone un método y un `record`.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<string>` e `IReadOnlyList<string>` no se resuelven.
- Olvidar el `using System.Linq;`: `ToList` no se resuelve.
- No comprobar el caso de lista vacía: el validador devuelve válido sin unidades.

PASO 4.2.8 — Añadir los `using` necesarios en `ValidadorUnidadesDeTrabajo`

Los tipos `List<string>` e `IReadOnlyList<string>` pertenecen a `System.Collections.Generic`. El método `ToList` pertenece a `System.Linq`. En proyectos con `ImplicitUsings` desactivado, hay que añadir los usings explícitos.

1. En el panel Solution Explorer, hacer doble clic en `ValidadorUnidadesDeTrabajo.cs`.
2. Inserta las directivas indicadas antes del namespace del archivo; la indicación del archivo identifica la línea exacta.
3. Verificar que no hay subrayados rojos en `List<string>` ni `ToList`.
4. Pulsar `Ctrl+S` para guardar el archivo.

Archivo: `src/AceriaApp.Api/UnitOfWork/ValidadorUnidadesDeTrabajo.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
namespace AceriaApp.Api.UnitOfWork;
```

```
using System.Collections.Generic;
using System.Linq;
```

Verificación concreta. `ValidadorUnidadesDeTrabajo.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir solo `System.Collections.Generic` y olvidar `System.Linq`: `ToList` no se resuelve.
- Mezclar directivas using con declaraciones de tipos: coloca las directivas antes del namespace para mantener el orden del archivo.
- Olvidar los usings y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 4.2.9 — Crear la clase `CatalogoUnidadesDeTrabajo`

La clase `CatalogoUnidadesDeTrabajo` contiene las unidades de trabajo por defecto del proyecto. Se usan en los tests y en la validación automática.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `UnitOfWork`.
2. Seleccionar `Add` → `Class` y nombrar la clase `CatalogoUnidadesDeTrabajo.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/UnitOfWork/CatalogoUnidadesDeTrabajo.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.UnitOfWork;

public class CatalogoUnidadesDeTrabajo
```

```
{
    public static List<UnidadDeTrabajo> UnidadesPorDefecto()
    {
        return new List<UnidadDeTrabajo>
        {
            new()
            {
                Nombre = "UnitOfWorkEfCore",
                RepositoriosCoordinados = 3,
                ExponeCommit = true,
                ExponeRollback = true,
                ContieneReglasDeNegocio = false,
                ConoceEntidadesConcretas = false
            },
            new()
            {
                Nombre = "UnitOfWorkEnMemoria",
                RepositoriosCoordinados = 3,
                ExponeCommit = true,
                ExponeRollback = true,
                ContieneReglasDeNegocio = false,
                ConoceEntidadesConcretas = false
            }
        };
    }
}
```

Verificación concreta. la clase `CatalogoUnidadesDeTrabajo` compila sin errores y expone un método estático.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<T>` no se resuelve.
- No inicializar las propiedades de los objetos: el compilador emite advertencias.
- Reutilizar una lista estática mutable: cada llamada del catálogo debe construir una lista independiente.

PASO 4.2.10 — Añadir el `using` necesario en `CatalogoUnidadesDeTrabajo`

El tipo `List<T>` pertenece a `System.Collections.Generic`. En proyectos con `ImplicitUsings` desactivado, hay que añadir el using explícito.

1. En el panel Solution Explorer, hacer doble clic en `CatalogoUnidadesDeTrabajo.cs`.
2. Inserta las directivas indicadas antes del namespace del archivo; la indicación del archivo identifica la línea exacta.
3. Verificar que no hay subrayados rojos en `List<T>`.
4. Pulsar `Ctrl+S` para guardar el archivo.

Archivo: `src/AceriaApp.Api/UnitOfWork/CatalogoUnidadesDeTrabajo.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
namespace AceriaApp.Api.UnitOfWork;
```

```
using System.Collections.Generic;
```

Verificación concreta. `CatalogoUnidadesDeTrabajo.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir `using System.Linq;` en lugar de `System.Collections.Generic`: no resuelve `List<T>`.
- Colocar el using después de una clase: las directivas deben preceder las declaraciones de tipos.

- Olvidar el using y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 4.2.11 — Crear el proyecto de tests del punto 4.2

El punto 4.2 añade un proyecto de tests específico para las clases del patrón Unit of Work. Se crea como proyecto nuevo y conserva las pruebas previas; las actualizaciones explícitas de integración se muestran en su paso correspondiente.

1. En `tests` crea la carpeta `AceriaApp.UnitOfWork.Tests` y, dentro, el archivo `AceriaApp.UnitOfWork.Tests.csproj` con el XML completo de este paso. En Visual Studio puedes elegir la plantilla xUnit y ajustar el archivo generado.
2. Elimina el archivo de ejemplo `UnitTest1.cs` si lo creó la plantilla; los archivos de pruebas completos se escriben en sus pasos correspondientes.
3. Ejecuta `dotnet sln AceriaApp.sln add tests/AceriaApp.UnitOfWork.Tests/AceriaApp.UnitOfWork.Tests.csproj` incluso si la solución ya existía y después `dotnet sln AceriaApp.sln list`.
4. Comprueba que el proyecto aparece bajo `tests` y que `dotnet restore tests/AceriaApp.UnitOfWork.Tests/AceriaApp.UnitOfWork.Tests.csproj` termina correctamente.

Archivo: `tests/AceriaApp.UnitOfWork.Tests/AceriaApp.UnitOfWork.Tests.csproj`. **Operación:** Crear.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. La solución muestra 11 proyectos, incluido `AceriaApp.UnitOfWork.Tests`; el nuevo csproj fija .NET 8, SDK de tests 17.12.0, xUnit 2.9.2 y runner 2.8.2.

Diagnóstico si falla.

- Crear el proyecto dentro de `src` en lugar de `tests`: rompe la estructura.
- Modificar los proyectos de tests del punto 4.1: rompe la conservación de la etapa anterior.
- Usar `dotnet new mstest`: el curso usa xUnit.

PASO 4.2.12 — Añadir la referencia del proyecto de tests a la API

La referencia permite que el proyecto de tests acceda a las clases de `UnitOfWork` del proyecto API.

1. Ejecuta `dotnet add tests/AceriaApp.UnitOfWork.Tests/AceriaApp.UnitOfWork.Tests.csproj reference src/AceriaApp.Api/AceriaApp.Api.csproj` o deja el archivo exactamente como el bloque XML de este paso.
2. Comprueba que la referencia va de tests a API; la API nunca debe referenciar el proyecto de tests.
3. Abre `tests/AceriaApp.UnitOfWork.Tests/AceriaApp.UnitOfWork.Tests.csproj` y comprueba el único `ProjectReference` a `../../src/AceriaApp.Api/AceriaApp.Api.csproj`.
4. Ejecuta `dotnet build tests/AceriaApp.UnitOfWork.Tests/AceriaApp.UnitOfWork.Tests.csproj -c Release` y verifica que el proyecto resuelve la API y sus dependencias.

Archivo: `tests/AceriaApp.UnitOfWork.Tests/AceriaApp.UnitOfWork.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. el archivo `AceriaApp.UnitOfWork.Tests.csproj` contiene un `ProjectReference` al proyecto API.

Diagnóstico si falla.

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo `.csproj`: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

PASO 4.2.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

`FluentAssertions` proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

1. Ejecuta `dotnet add tests/AceriaApp.UnitOfWork.Tests/AceriaApp.UnitOfWork.Tests.csproj package FluentAssertions --version 6.12.2` o reemplaza el csproj por el XML completo indicado.
2. Ejecuta `dotnet list tests/AceriaApp.UnitOfWork.Tests/AceriaApp.UnitOfWork.Tests.csproj package` y localiza `FluentAssertions 6.12.2`.
3. Comprueba que el paquete se añade al nuevo proyecto de tests y que se mantienen la referencia a la API y las versiones de xUnit.
4. Guarda el archivo y ejecuta `dotnet build tests/AceriaApp.UnitOfWork.Tests/AceriaApp.UnitOfWork.Tests.csproj -c Release`; las aserciones `Should()` estarán disponibles cuando escribas las pruebas.

Archivo: `tests/AceriaApp.UnitOfWork.Tests/AceriaApp.UnitOfWork.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. El archivo de proyecto y `dotnet list package` muestran `FluentAssertions 6.12.2`; la compilación finaliza correctamente.

Diagnóstico si falla.

- Instalar una versión preliminar: el curso usa la versión fijada en el archivo de proyecto.
- Olvidar el paquete: `Should()` no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

PASO 4.2.14 — Crear el archivo `ValidadorUnidadesDeTrabajoTests.cs`

Los tests verifican que el validador acepta el catálogo por defecto y rechaza las unidades de trabajo que violan las responsabilidades del patrón.

1. En el panel Solution Explorer, expandir `tests` → `AceriaApp.UnitOfWork.Tests`.
2. Crear el archivo `ValidadorUnidadesDeTrabajoTests.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `tests/AceriaApp.UnitOfWork.Tests/ValidadorUnidadesDeTrabajoTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.UnitOfWork;
using FluentAssertions;
using Xunit;

namespace AceriaApp.UnitOfWork.Tests;

public class ValidadorUnidadesDeTrabajoTests
{
    private readonly ValidadorUnidadesDeTrabajo _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoUnidadesDeTrabajo.UnidadesPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinUnidades_DevuelveError()
    {
        var resultado = _validador.Validar(new List<UnidadDeTrabajo>());

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay unidades"));
    }

    [Fact]
    public void Validar_SinRepositoriosCoordinados_DevuelveError()
    {
        var unidades = new List<UnidadDeTrabajo>
        {
            new() { Nombre = "UoWX", RepositoriosCoordinados = 0, ExponeCommit = true, ExponeRollback =
true }
        };

        var resultado = _validador.Validar(unidades);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no coordina ningún repositorio"));
    }

    [Fact]
    public void Validar_SinCommit_DevuelveError()
    {
        var unidades = new List<UnidadDeTrabajo>
        {
```

```
            new() { Nombre = "UoWX", RepositoriosCoordinados = 1, ExponeCommit = false, ExponeRollback =
true }
        };

        var resultado = _validador.Validar(unidades);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("confirmación"));
    }

    [Fact]
    public void Validar_SinRollback_DevuelveError()
    {
        var unidades = new List<UnidadDeTrabajo>
        {
            new() { Nombre = "UoWX", RepositoriosCoordinados = 1, ExponeCommit = true, ExponeRollback =
false }
        };

        var resultado = _validador.Validar(unidades);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("reversión"));
    }

    [Fact]
    public void Validar_ConReglasDeNegocio_DevuelveError()
    {
        var unidades = new List<UnidadDeTrabajo>
        {
            new() { Nombre = "UoWX", RepositoriosCoordinados = 1, ExponeCommit = true, ExponeRollback = true,
ContieneReglasDeNegocio = true }
        };

        var resultado = _validador.Validar(unidades);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("reglas de negocio"));
    }

    [Fact]
    public void Validar_ConEntidadesConcretas_DevuelveError()
    {
        var unidades = new List<UnidadDeTrabajo>
        {
            new() { Nombre = "UoWX", RepositoriosCoordinados = 1, ExponeCommit = true, ExponeRollback = true,
ConoceEntidadesConcretas = true }
        };

        var resultado = _validador.Validar(unidades);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("entidades concretas"));
    }
}
```

Verificación concreta. el archivo `ValidadorUnidadesDeTrabajoTests.cs` compila y contiene siete métodos de test.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.UnitOfWork;`: las clases no se resuelven.
- Usar `Assert.True` en lugar de `Should().BeTrue()`: el curso usa FluentAssertions.
- No cubrir el caso de lista vacía: el mensaje de error no se verifica.

PASO 4.2.15 — Crear el archivo `CatalogoUnidadesDeTrabajoTests.cs`

Los tests verifican que el catálogo por defecto contiene las unidades esperadas y que ninguna viola las responsabilidades del patrón.

1. En el panel Solution Explorer, hacer clic derecho sobre el proyecto `AceriaApp.UnitOfWork.Tests`.

2. Seleccionar `Add` → `Class` y nombrar la clase `CatalogoUnidadesDeTrabajoTests.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `tests/AceriaApp.UnitOfWork.Tests/CatalogoUnidadesDeTrabajoTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.UnitOfWork;
using FluentAssertions;
using Xunit;

namespace AceriaApp.UnitOfWork.Tests;

public class CatalogoUnidadesDeTrabajoTests
{
    [Fact]
    public void UnidadesPorDefecto_ContieneDosUnidades()
    {
        var unidades = CatalogoUnidadesDeTrabajo.UnidadesPorDefecto();

        unidades.Should().HaveCount(2);
    }

    [Fact]
    public void UnidadesPorDefecto_TodasExponenCommitYRollback()
    {
        var unidades = CatalogoUnidadesDeTrabajo.UnidadesPorDefecto();

        unidades.Should().OnlyContain(u => u.ExponeCommit);
        unidades.Should().OnlyContain(u => u.ExponeRollback);
    }

    [Fact]
    public void UnidadesPorDefecto_NingunaContieneReglasDeNegocio()
    {
        var unidades = CatalogoUnidadesDeTrabajo.UnidadesPorDefecto();

        unidades.Should().OnlyContain(u => !u.ContieneReglasDeNegocio);
    }

    [Fact]
    public void UnidadesPorDefecto_NingunaConoceEntidadesConcretas()
    {
        var unidades = CatalogoUnidadesDeTrabajo.UnidadesPorDefecto();

        unidades.Should().OnlyContain(u => !u.ConoceEntidadesConcretas);
    }
}
```

Verificación concreta. el archivo `CatalogoUnidadesDeTrabajoTests.cs` compila y contiene cuatro métodos de test.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.UnitOfWork;`: las clases no se resuelven.
- Olvidar el `using FluentAssertions;`: `OnlyContain` no se resuelve.
- No verificar las responsabilidades prohibidas: el catálogo podría violar el patrón.

PASO 4.2.16 — Integrar la unidad de trabajo y compilar la solución

La integración muestra cómo se comparte el DbContext dentro del ámbito de la petición y quién ejecuta la confirmación. Comprueba un resultado correcto y un rechazo de dominio: no debe persistirse un cambio después de un fallo de validación.

1. Abre cada archivo de integración funcional indicado en este paso. Crea los nuevos y reemplaza únicamente los que se señalen; copia todo el código mostrado.
2. Recorre las referencias: el dominio define sus entidades, la aplicación usa puertos y la infraestructura implementa persistencia; la API compone el flujo.

3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`; comprueba que todos los proyectos nuevos estén incluidos en la solución.
4. Verifica `Build succeeded.` y `0 Error(s)`. Si hay advertencias, revisa su causa; en el paso siguiente ejecutarás las pruebas del comportamiento completo.

Integración funcional. Expone /api/v1/patrones/ordenes sin alterar los endpoints heredados. 4.2 usa el caso transaccional; 4.3 enruta por ISender; 4.4 traduce Result a 201/200 y ProblemDetails 400/404/409 con código estable.

Archivo: `src/AceriaApp.Api/Patrones/PatronesOrdenesEndpoints.cs`. **Operación:** Crear.

```
using AceriaApp.Api.Hexagonal;
using AceriaApp.Hexagonal.Aplicacion.Patrones;
namespace AceriaApp.Api.Patrones;

public static class PatronesOrdenesEndpoints
{
    public static IEndpointRouteBuilder MapPatronesOrdenes(this IEndpointRouteBuilder app)
    {
        var group = app.MapGroup("/api/v1/patrones/ordenes").WithTags("Patrones");
        group.MapGet("/{id:guid}", async (Guid id, OrdenesTransaccionales caso, CancellationToken ct) =>
        {
            var orden = await caso.ObtenerAsync(id, ct);
            return orden is null ? Results.NotFound() : Results.Ok(OrdenHexagonalDto.Desde(orden));
        }).WithName("ObtenerOrdenPatrones").Produces<OrdenHexagonalDto>(200).Produces(404);
        group.MapPost("", async (CrearOrdenHexagonalRequest body, OrdenesTransaccionales caso,
CancellationToken ct) =>
        {
            if (string.IsNullOrWhiteSpace(body.Cliente) || string.IsNullOrWhiteSpace(body.Material)
                || body.Cliente.Trim().Length > 100 || body.Material.Trim().Length > 100
                || body.Cantidad <= 0 || body.Cantidad > 500)
                return Results.Problem(title: "Datos de la orden inválidos.", statusCode: 400);
            var orden = await caso.CrearAsync(body.Cliente, body.Material, body.Cantidad, ct);
            return Results.CreatedAtRoute("ObtenerOrdenPatrones", new { id = orden.Id },
OrdenHexagonalDto.Desde(orden));
        }).WithName("CrearOrdenPatrones").Produces<OrdenHexagonalDto>(201).ProducesProblem(400);
        return app;
    }
}
```

Aísla la ruta de este módulo y conserva los endpoints existentes de M02 y M03. Devuelve 201 y una cabecera Location calculada con la ruta de lectura nombrada y el identificador creado.

Integración funcional. Integra únicamente registros y mapeo del endpoint de patrones. Conserva íntegros los endpoints, documentación y arranque de M2/M3; en 4.5 reemplaza registros repetidos por AddPatronesOrdenes().

Archivo: `src/AceriaApp.Api/Program.cs`. **Operación:** Reemplazar el contenido completo.

```
using AceriaApp.Api.Patrones;
using AceriaApp.Hexagonal.Aplicacion.Patrones;
using AceriaApp.Hexagonal.Infraestructura.Patrones;
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Validacion;
using FluentValidation;
using AceriaApp.Api.Documentacion;

using AceriaApp.Api.Hexagonal;
using AceriaApp.Hexagonal.Infraestructura;
using AceriaApp.Hexagonal.Infraestructura.Datos;
var builder=WebApplication.CreateBuilder(args);
// INICIO PASO 2.1.12: SQL Server Express LocalDB, no SQLite.
builder.Services.AddDbContext<AceriaDbContext>(o=>
    o.UseSqlServer(builder.Configuration.GetConnectionString("AceriaApp")));
// PASO 2.3.14: registrar validadores en el contenedor.
builder.Services.AddValidatorsFromAssemblyContaining<CrearOrdenValidator>(); // PASO 2.3.14
```

```
// PASO 2.4.6: registrar generador antes de Build.
builder.Services.AddDocumentacion(); // PASO 2.4.6

builder.Services.AddServiciosHexagonales(builder.Configuration);
builder.Services.AddScoped<IOrdenTrabajoRepositorio, OrdenTrabajoRepositorio>();
builder.Services.AddScoped<IUnitOfWork, EfUnitOfWork>();
builder.Services.AddScoped<OrdenesTransaccionales>();
var app=builder.Build();
// EnsureCreated se utiliza SOLO como ayuda didáctica en Development.
// Para un sistema productivo se utilizarían migraciones controladas.
// PASO 2.1.13: inicializar LocalDB solo en Development.
if(app.Environment.IsDevelopment()&&!app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope=app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<AceriaDbContext>().Database.EnsureCreatedAsync();
}

// INICIO PASO 2.1.14: colección de recursos, consulta real con EF Core.
app.MapGet("/api/v1/ordenes",async (AceriaDbContext db,CancellationToken ct)=>
{
    // PASO 2.1.15: lectura sin tracking y proyeccion a DTO.
    var ordenes=await db.Ordenes.AsNoTracking().OrderBy(x=>x.FechaCreacion).ToListAsync(ct);
    return Results.Ok(ordenes.Select(OrdenDto.Desde));
})
.WithName("ListarOrdenes").WithTags("Ordenes")
.WithSummary("Lista las órdenes de producción").Produces<IEnumerable<OrdenDto>>(200)
;
// INICIO PASO 2.1.17: recurso individual.
app.MapGet("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(x=>x.Id==id);
    // PASO 2.1.18: 200 para orden presente; 404 si no existe.
    IResult response=orden is null ? Results.NotFound() : Results.Ok(OrdenDto.Desde(orden));
    return response;
})
.WithName("ObtenerOrdenPorId").WithTags("Ordenes")
.WithSummary("Obtiene una orden por ID").Produces<OrdenDto>(200).Produces(404)
;


// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    // PASO 2.2.6: crear entidad y persistirla.
    var orden=new Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    // PASO 2.2.7: devolver 201 con Location.
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")
// PASO 2.3.15: filtro de POST.
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Crea una orden").Produces<OrdenDto>(201).ProducesValidationProblem(400).ProducesProblem(422)
;


// INICIO PASO 2.2.11: sustitución completa de los campos editables.
app.MapPut("/api/v1/ordenes/{id:guid}",async (Guid id,CrearOrdenRequest request,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cliente=request.Cliente;orden.Material=request.Material;
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("ReemplazarOrden").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
```

```
.WithSummary("Sustituye campos
editables").Produces(204).Produces(404).ProducesProblem(409).ProducesValidationProblem(400).ProducesProblem(4
22)
;


// INICIO PASO 2.2.14: cambiar SOLO la cantidad.
app.MapPatch("/api/v1/ordenes/{id:guid}/cantidad",
    async (Guid id,ActualizarCantidadRequest request,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("CambiarCantidad").WithTags("Ordenes")
// PASO 2.3.17: filtro de PATCH.
.AddEndpointFilter<ValidationFilter<ActualizarCantidadRequest>>()
.WithSummary("Modifica
cantidad").Produces<OrdenDto>(200).Produces(404).ProducesValidationProblem(400).ProducesProblem(409).Produces
Problem(422)
;


// INICIO PASO 2.2.16: eliminar recurso persistido.
app.MapDelete("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede eliminar una orden finalizada.",statusCode:409);
    db.Ordenes.Remove(orden);
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("EliminarOrden").WithTags("Ordenes")
.WithSummary("Elimina una orden pendiente").Produces(204).Produces(404).ProducesProblem(409)
;
// INICIO PASO 2.2.18: transición explícita del dominio.
app.MapPost("/api/v1/ordenes/{id:guid}/finalizar",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"La orden ya estaba finalizada.",statusCode:409);
    orden.Estado="Finalizada";
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("FinalizarOrden").WithTags("Ordenes")
.WithSummary("Finaliza una orden").Produces<OrdenDto>(200).Produces(404).ProducesProblem(409)
;

// PASO 2.4.8: habilitar Swagger solo en Development.
app.UseDocumentacion(); // PASO 2.4.8
// El contexto hexagonal usa su propia base para que EnsureCreated no dependa de
// las tablas heredadas del curso M02.
if (app.Environment.IsDevelopment() && !app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope = app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<HexagonalDbContext>()
        .Database.EnsureCreatedAsync();
}
app.MapHexagonalOrdenes();
app.MapPatronesOrdenes();
app.Run();
public partial class Program {} // permite WebApplicationFactory en tests
```

Permite mapear los endpoints del módulo desde la raíz de composición. Permite registrar las implementaciones de los puertos desde la API. Mantiene el registro SQL Server y los servicios que ya utilizaba M03.

Durante esta etapa se registra explícitamente el puerto de escritura con su adaptador. La unidad de trabajo comparte el contexto Scoped de la petición. Publica las rutas del nuevo flujo después de construir la aplicación y antes de ejecutarla.

Integración funcional. Separa el contrato de staging del repositorio autocontenido de M3. Agregar prepara una entidad; Obtener devuelve una entidad seguida por el mismo contexto. No confirma cambios.

Archivo: `src/AceriaApp.Hexagonal.Aplicacion/Patrones/IOrdenTrabajoRepositorio.cs`. **Operación:** Crear.

```
using AceriaApp.Hexagonal.Dominio;
namespace AceriaApp.Hexagonal.Aplicacion.Patrones;

/// <summary>Prepara cambios; la unidad de trabajo decide cuándo confirmarlos.</summary>
public interface IOrdenTrabajoRepositorio
{
    Task<OrdenHexagonal?> ObtenerAsync(Guid id, CancellationToken ct = default);
    Task AgregarAsync(OrdenHexagonal orden, CancellationToken ct = default);
}
```

Define el puerto que necesita el caso de uso; no depende de DbContext ni de SQL Server. Devuelve una entidad del dominio o null si no existe; el token permite propagar cancelación. Prepara una entidad en el contexto compartido. Este contrato no confirma cambios, porque esa decisión corresponde a la unidad de trabajo.

Integración funcional. Define en Aplicación el límite explícito de confirmación. Devuelve el número de entradas guardadas y propaga CancellationToken.

Archivo: `src/AceriaApp.Hexagonal.Aplicacion/Patrones/IUnitOfWork.cs`. **Operación:** Crear.

```
namespace AceriaApp.Hexagonal.Aplicacion.Patrones;

public interface IUnitOfWork
{
    Task<int> CommitAsync(CancellationToken ct = default);
}
```

La aplicación declara la frontera de persistencia y la infraestructura la implementa. Expone una única confirmación asincrónica del conjunto de cambios preparados; devuelve el número de entradas guardadas y propaga la cancelación.

Integración funcional. Coordina la regla de capacidad, el agregado al repositorio y un único commit. Es la variante inicial de 4.2 y permanece como referencia ejecutable al introducir Result en 4.4.

Archivo: `src/AceriaApp.Hexagonal.Aplicacion/Patrones/OrdenesTransaccionales.cs`. **Operación:** Crear.

```
using AceriaApp.Hexagonal.Dominio;
namespace AceriaApp.Hexagonal.Aplicacion.Patrones;

public sealed class OrdenesTransaccionales(IOrdenTrabajoRepositorio repositorio, IUnitOfWork trabajo)
{
    public async Task<OrdenHexagonal> CrearAsync(string cliente, string material, int cantidad,
CancellationToken ct = default)
    {
        if (cantidad > 500) throw new ArgumentOutOfRangeException(nameof(cantidad), "Capacidad máxima:
500.");
        var orden = new OrdenHexagonal(cliente, material, cantidad);
        await repositorio.AgregarAsync(orden, ct);
        await trabajo.CommitAsync(ct);
        return orden;
    }
```

```
    public Task<OrdenHexagonal?> ObtenerAsync(Guid id, CancellationToken ct = default) =>
        repositorio.ObtenerAsync(id, ct);
}
```

El caso de uso depende de dos puertos. No instancia repositorios ni conoce el proveedor de persistencia. La regla de capacidad se aplica antes de preparar cualquier escritura. El dominio construye la entidad y protege sus invariantes.

Prepara el cambio a través del repositorio; aún no hay confirmación. El caso de uso decide el momento de confirmar y propaga el mismo token de cancelación. La consulta se delega al puerto sin introducir consultas EF en la aplicación.

Integración funcional. Confirma todos los cambios seguidos por el contexto mediante un único SaveChangesAsync. EF Core usa su transacción automática en el proveedor relacional; el alcance se limita a esta base de datos, sin prometer atomicidad de servicios externos.

Archivo: `src/AceriaApp.Hexagonal.Infraestructura/Patrones/EfUnitOfWork.cs`. **Operación:** Crear.

```
using AceriaApp.Hexagonal.Aplicacion.Patrones;
using AceriaApp.Hexagonal.Infraestructura.Datos;
namespace AceriaApp.Hexagonal.Infraestructura.Patrones;

/// <summary>Un SaveChanges: transacción automática para los cambios de este contexto relacional.</summary>
public sealed class EfUnitOfWork(HexagonalDbContext db) : IUnitOfWork
{
    public Task<int> CommitAsync(CancellationToken ct = default) => db.SaveChangesAsync(ct);
}
```

La implementación de infraestructura recibe el mismo HexagonalDbContext que utiliza el repositorio del ámbito. Confirma todos los cambios pendientes con una sola llamada. El proveedor relacional usa una transacción automática cuando es necesaria; si falla el lote, se revierte. El token se transmite a EF Core.

Integración funcional. Implementa staging y lectura con seguimiento sobre HexagonalDbContext. No llama SaveChanges: varios cambios pueden confirmarse juntos mediante IUnitOfWork.

Archivo: `src/AceriaApp.Hexagonal.Infraestructura/Patrones/OrdenTrabajoRepositorio.cs`. **Operación:** Crear.

```
using AceriaApp.Hexagonal.Aplicacion.Patrones;
using AceriaApp.Hexagonal.Dominio;
using AceriaApp.Hexagonal.Infraestructura.Datos;
using Microsoft.EntityFrameworkCore;
namespace AceriaApp.Hexagonal.Infraestructura.Patrones;

public sealed class OrdenTrabajoRepositorio(HexagonalDbContext db) : IOrdenTrabajoRepositorio
{
    public Task<OrdenHexagonal?> ObtenerAsync(Guid id, CancellationToken ct = default) =>
        db.Ordenes.FirstOrDefaultAsync(x => x.Id == id, ct);
    public async Task AgregarAsync(OrdenHexagonal orden, CancellationToken ct = default) =>
        await db.Ordenes.AddAsync(orden, ct);
}
```

Recibe por constructor el contexto del ámbito actual y cumple el puerto de la aplicación. Consulta por identificador con seguimiento de EF Core; la entidad recuperada permanece asociada al contexto compartido. Añade la entidad al seguimiento sin llamar a SaveChanges. Un segundo contexto todavía no debe observarla antes del commit.

Integración funcional. Conserva las dependencias de los tests de catálogo y añade Testcontainers.MsSql, el helper compartido y WebApplicationFactory para las pruebas funcionales.

Archivo: `tests/AceriaApp.UnitOfWork.Tests/AceriaApp.UnitOfWork.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings><IsPackable>false</IsPackable><IsTestProject>true</IsTestProject></PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="Testcontainers.MsSql" Version="3.10.0" />
    <PackageReference Include="Microsoft.AspNetCore.Mvc.Testing" Version="8.0.21" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <Compile Include="../Compartido/SqlServerPruebas.cs" Link="SqlServerPruebas.cs" />
  </ItemGroup>
</Project>
```

Compila para .NET 8, igual que los proyectos heredados. Fija el cliente de contenedores para SQL Server; el proveedor EF Core SQL Server llega mediante la referencia a la API. Permite ejecutar peticiones HTTP contra el host real de la API.

Compila el helper heredado que crea y elimina únicamente la base aislada de cada prueba, sin añadir otro proyecto a la solución.

Integración funcional. Prueba staging, confirmación, abandono del ámbito y persistencia entre contextos. La prueba de atomicidad limita los lotes a un comando únicamente al inyectar el fallo y lo provoca antes del segundo INSERT, después de verificar uno ejecutado. WhenNeeded deja cero filas; el control negativo Never deja una. Así se demuestra rollback después de una escritura real, sin EF InMemory.

Archivo: `tests/AceriaApp.UnitOfWork.Tests/UnitOfWorkFuncionalesTests.cs`. **Operación:** Crear.

```
using System.Data.Common;
using Microsoft.EntityFrameworkCore.Diagnostics;
using AceriaApp.Hexagonal.Aplicacion.Patrones;
using AceriaApp.Hexagonal.Dominio;
using AceriaApp.Hexagonal.Infraestructura.Datos;
using AceriaApp.Hexagonal.Infraestructura.Patrones;
using AceriaApp.PruebasSqlServer;
using Microsoft.EntityFrameworkCore;
using Xunit;
namespace AceriaApp.UnitOfWork.Tests;

public sealed class UnitOfWorkFuncionalesTests
{
    [Fact]
    public async Task Agregar_solo_prepara_y_commit_persiste_todo()
    {
        await using var store = BaseSqlServer.Crear();
        await using var db = Contexto(store);
        await db.Database.EnsureCreatedAsync();
        var repo = new OrdenTrabajoRepositorio(db);
        await repo.AgregarAsync(new OrdenHexagonal("Axia", "Acero", 20));
        await repo.AgregarAsync(new OrdenHexagonal("Norte", "Chapa", 30));
        await using (var antes = Contexto(store)) Assert.Empty(await antes.Ordenes.ToListAsync());
        Assert.Equal(2, await new EfUnitOfWork(db).CommitAsync());
        await using var despues = Contexto(store);
        Assert.Equal(2, await despues.Ordenes.CountAsync());
    }

    [Fact]
    public async Task
Fallo_despues_de_una_escritura_revierte_el_lote_y_control_sin_transaccion_lo_demuestra()
    {
        foreach (var transacciones in new[] { AutoTransactionBehavior.WhenNeeded,
AutoTransactionBehavior.Never })
        {
            await using var store = BaseSqlServer.Crear();
            var fallo = new FalloSegundoInsert();
            await using var db = Contexto(store, fallo);
            await db.Database.EnsureCreatedAsync();
            db.Database.AutoTransactionBehavior = transacciones;
```

```
            var repo = new OrdenTrabajoRepositorio(db);
            await repo.AgregarAsync(new OrdenHexagonal("Axia", "Acero", 20));
            await repo.AgregarAsync(new OrdenHexagonal("Norte", "Chapa", 30));
            await Assert.ThrowsAsync<DbUpdateException>(() => new EfUnitOfWork(db).CommitAsync());
            Assert.Equal(1, fallo.InsertsEjecutados);
            await using var verificacion = Contexto(store);
            // Control negativo: Never deja la primera escritura. Producción usa WhenNeeded.
            var filasEsperadas = transacciones == AutoTransactionBehavior.Never ? 1 : 0;
            Assert.Equal(filasEsperadas, await verificacion.Ordenes.CountAsync());
        }
    }

    [Fact]
    public async Task Descartar_el_ambito_sin_commit_descarta_cambios()
    {
        await using var store = BaseSqlServer.Crear();
        await using (var db = Contexto(store))
        {
            await db.Database.EnsureCreatedAsync();
            await new OrdenTrabajoRepositorio(db).AgregarAsync(new OrdenHexagonal("Axia", "Acero", 20));
        }
        await using var verificacion = Contexto(store);
        Assert.Empty(await verificacion.Ordenes.ToListAsync());
    }

    [Fact]
    public async Task Caso_de_uso_confirma_y_permita_leer_desde_otro_contexto()
    {
        await using var store = BaseSqlServer.Crear();
        Guid id;
        await using (var db = Contexto(store))
        {
            await db.Database.EnsureCreatedAsync();
            var caso = new OrdenesTransaccionales(new OrdenTrabajoRepositorio(db), new EfUnitOfWork(db));
            id = (await caso.CrearAsync("Axia", "Acero", 20)).Id;
        }
        await using var verificacion = Contexto(store);
        Assert.Equal("Axia", (await verificacion.Ordenes.SingleAsync(x => x.Id == id)).Cliente);
    }

    private sealed class FalloSegundoInsert : DbCommandInterceptor
    {
        public int InsertsEjecutados { get; private set; }
        public override ValueTask<InterceptionResult<DbDataReader>> ReaderExecutingAsync(
            DbCommand command, CommandEventData data, InterceptionResult<DbDataReader> result,
CancellationToken ct = default)
        {
            if (command.CommandText.Contains("INSERT INTO", StringComparison.OrdinalIgnoreCase) &&
InsertsEjecutados == 1)
                throw new InvalidOperationException("Fallo de prueba después de ejecutar el primer INSERT.");
            return ValueTask.FromResult(result);
        }
        public override ValueTask<DbDataReader> ReaderExecutedAsync(
            DbCommand command, CommandExecutedEventData data, DbDataReader result, CancellationToken ct =
default)
        {
            if (command.CommandText.Contains("INSERT INTO", StringComparison.OrdinalIgnoreCase))
InsertsEjecutados++;
            return ValueTask.FromResult(result);
        }
    }

    private static HexagonalDbContext Contexto(BaseSqlServer store, IInterceptor? fallo = null)
    {
        var options = new DbContextOptionsBuilder<HexagonalDbContext>();
        if (fallo is null) options.UseSqlServer(store.ConnectionString);
        else options.UseSqlServer(store.ConnectionString, sql => sql.MaxBatchSize(1)).AddInterceptors(fallo);
        var db = new HexagonalDbContext(options.Options);
        Assert.Equal("Microsoft.EntityFrameworkCore.SqlServer", db.Database.ProviderName);
        return db;
    }
}
```

Un contexto independiente no ve cambios preparados; el commit debe persistir ambas órdenes. Ejecuta la misma secuencia con transacción automática y con el control negativo sin transacción. En `await using var db = Contexto(store, fallo);` se solo esta prueba separa los INSERT para provocar el error después de la primera escritura.

En `Assert.Equal(1, fallo.InsertsEjecutados);` se exige que una escritura se haya ejecutado antes de lanzar el fallo. Un contexto independiente consulta el estado confirmado; Never deja una fila y WhenNeeded ninguna. Desechar el contexto sin commit no escribe las entidades preparadas.

Detecta el INSERT aunque SQL Server anteponga instrucciones SET al comando. Desactiva la agrupación solamente cuando se inyecta el fallo; las demás pruebas conservan la configuración habitual. Comprueba que todos los contextos de la prueba usan SQL Server.

Verificación concreta. La solución completa compila, incluidos el código de integración y sus pruebas. Las siguientes ejecuciones de tests comprobarán el comportamiento de los patrones.

Diagnóstico si falla.

- Tener la API en ejecución: el archivo `.dll` está bloqueado y la compilación falla.
- No restaurar paquetes: `dotnet build` restaura por defecto, pero si hay un error de red, hay que ejecutar `dotnet restore` primero.
- Usar una versión distinta del SDK: `dotnet --version` debe devolver `8.x.x`.

PASO 4.2.17 — Ejecutar todos los tests del punto 4.2

Ejecutar los tests del punto 4.2 verifica que las clases del patrón Unit of Work funcionan correctamente.

1. Ejecuta `dotnet test tests/AceriaApp.UnitOfWork.Tests/AceriaApp.UnitOfWork.Tests.csproj -c Release` desde la raíz de esta etapa.
2. Comprueba un total de 15 pruebas: 11 de catálogo y 4 funcionales, con cero fallos y cero omisiones.
3. Si falla una prueba, abre su mensaje de aserción y el archivo indicado; corrige el comportamiento antes de continuar.
4. Repite el mismo comando tras la corrección y conserva el resultado de la ejecución válida.

Verificación concreta. El proyecto `AceriaApp.UnitOfWork.Tests` termina con `Failed: 0, Passed: 15, Skipped: 0, Total: 15`.

Diagnóstico si falla.

- Ejecutar otro proyecto: comprueba la ruta y el nombre de la asamblea de pruebas antes de interpretar los resultados.
- No compilar antes: `dotnet test` compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

PASO 4.2.18 — Ejecutar todos los tests de la solución

Ejecutar todos los tests de la solución verifica que los tests de los puntos anteriores y del punto 4.2 pasan conjuntamente.

1. Ejecuta `dotnet build AceriaApp.sln -c Release` para producir todas las asambleas de esta etapa.
2. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build --no-restore` para ejecutar todas las pruebas acumuladas.
3. Suma los resultados de los 7 proyectos de tests: deben ser 77 pruebas aprobadas, sin fallos ni omisiones.
4. Comprueba que las pruebas heredadas de M03 y los puntos anteriores de M04 siguen presentes; no borres tests para obtener un resultado verde.

Verificación concreta. La suma de las líneas de resultados es `Failed: 0`, `Passed: 77`, `Skipped: 0` y `Total: 77`. Dotnet muestra un resumen por proyecto, no necesariamente una única línea global.

Diagnóstico si falla.

- No haber compilado antes: `--no-build` falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
- Eliminar pruebas anteriores o cambiarlas sin una instrucción explícita: se pierde la cobertura de los contratos heredados.

PASO 4.2.19 — Crear el README del punto 4.2

El README del punto 4.2 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 4.1.

1. En la carpeta del punto `4.2-unit-of-work`, hacer clic derecho y seleccionar `Add` → `New Item` → `Text File`.
2. Nombrar el archivo `README.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que el README del punto 4.1 no se ha modificado.

Archivo: `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 4.2 — Unit of Work

## Objetivo
Comprender el patrón Unit of Work y añadir al proyecto las clases de validación de unidades de trabajo.

La práctica conecta además el patrón con los puertos, adaptadores y pruebas de integración que aparecen en
este manual.

## Estado del proyecto
- Solución `AceriaApp.sln` con 11 proyectos.
- Documento `docs/patron-unit-of-work.md` con las responsabilidades y las prohibiciones.
- Clases `UnidadDeTrabajo`, `ValidadorUnidadesDeTrabajo` y `CatalogoUnidadesDeTrabajo` en `src/AceriaApp.Api/
UnitOfWork/`.
- 11 tests nuevos de catálogo y 4 funcionales en `tests/AceriaApp.UnitOfWork.Tests`.
- Los tests acumulados suman 77 pruebas: 11 de catálogo nuevas y 4 funcionales nuevas en este punto.

## Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio 2026.
2. Verificar que SQL Server Express está arrancado.
3. Configurar las conexiones SQL Server `AceriaApp` y `AceriaHexagonal` en el entorno local.
4. Ejecutar `dotnet run --project src/AceriaApp.Api/AceriaApp.Api.csproj --urls http://localhost:5000` desde
la raíz de esta etapa.
5. Navegar a `http://localhost:5000/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

## Qué se ha añadido respecto al punto anterior
- Documento `docs/patron-unit-of-work.md`.
- Carpeta `UnitOfWork` en el proyecto API con tres clases nuevas.
- Proyecto `tests/AceriaApp.UnitOfWork.Tests` con 11 tests de catálogo y 4 funcionales nuevos.
- Se conserva intacta la carpeta del punto 4.1.

## Documentación
- [ADR-010](docs/ADR-010-capas-aplicacion.md) a [ADR-014](docs/ADR-014-patron-repository.md) (heredados).
- [Patrón Unit of Work](docs/patron-unit-of-work.md).

## Verificación ejecutable

Los catálogos describen ejemplos de diseño. El comportamiento de las clases reales se comprueba además con
los tests funcionales del punto. Ejecuta `dotnet test AceriaApp.sln -c Release` y exige cero fallos y cero
omisiones.
```

Verificación concreta. El archivo `README.md` existe y contiene las 6 secciones mostradas; abre su vista previa y comprueba todos los apartados.

Diagnóstico si falla.

- Modificar el README del punto 4.1: rompe la conservación de la etapa anterior.
- Omitir la sección `Qué se ha añadido respecto al punto anterior`: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 4.1: rompe la separación.

PASO 4.2.20 — Crear el ADR-015 del punto 4.2

El ADR-015 documenta la decisión de adoptar el patrón Unit of Work. Es un artefacto nuevo del punto 4.2 que no modifica los ADR existentes.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `docs`.
2. Seleccionar `Add` → `New Item` → `Text File` y nombrar el archivo `ADR-015-patron-unit-of-work.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que los ADR-010 a ADR-014 no se han modificado.

Archivo: `docs/ADR-015-patron-unit-of-work.md`. **Operación:** Crear.

```
# ADR-015: Adopción del patrón Unit of Work

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha adoptado el patrón Repository. Es necesario coordinar las operaciones de varios
repositorios en una única transacción.

## Decisión
Se adopta un contrato mínimo `IUnitOfWork` con `CommitAsync`. La implementación `EfUnitOfWork` delega en un
único `SaveChangesAsync` de `HexagonalDbContext`. El nuevo `OrdenTrabajoRepositorio` prepara cambios sin
confirmarlos y comparte contexto con la unidad de trabajo. EF Core utiliza su transacción automática
relacional cuando el lote lo requiere: una prueba provoca el fallo después de un INSERT y comprueba la
reversión; el control sin transacción deja una fila. No se publica un método Rollback en este contrato. Los
cambios no confirmados se descartan al desechar el contexto; una reversión tras un fallo de persistencia la
resuelve la transacción del proveedor. El repositorio heredado de M03 conserva su comportamiento para los
endpoints anteriores.

## Alternativas consideradas
- Gestión de transacciones en cada repositorio: descartado por duplicación.
- Uso directo del `DbContext` desde la aplicación: descartado por acoplamiento.
- Sin unidad de trabajo: descartado por riesgo de cambios parciales.

## Consecuencias
- Positivas: atomicidad, desacoplamiento de transacciones, coordinación de repositorios.
- Negativas: la unidad de trabajo añade una capa de indirección.
- Mitigación: la interfaz se limita a las operaciones necesarias.

## Variantes del catálogo y del contrato ejecutable

El catálogo de diseño de este punto incluye unidades que declaran confirmación y reversión explícitas. Sus
dos registros y el resumen de seis repositorios son datos del ejercicio de validación. La implementación
ejecutable utiliza `IUnitOfWork.CommitAsync` y `EfUnitOfWork`, con un repositorio para las órdenes; la prueba
del lote coordina dos entidades y la transacción pertenece al contexto. Una unidad de trabajo puede coordinar
varios repositorios que compartan ese contexto, pero este punto no implementa repositorios ficticios de
inventario o clientes.
```

Verificación concreta. El archivo `docs/ADR-015-patron-unit-of-work.md` existe y contiene las 6 secciones mostradas; abre su vista previa y comprueba todos los apartados.

Diagnóstico si falla.

- Modificar los ADR anteriores: rompe la conservación de la etapa anterior.
- Omitir la sección `Alternativas consideradas`: el ADR pierde valor.
- Nombrar el archivo sin el prefijo `ADR-015`: rompe la convención.

PASO 4.2.21 — Verificar que los documentos heredados y la etapa 4.1 permanecen intactos

Las decisiones previas se conservan como registros históricos. Los nuevos ADR explican la evolución y las modificaciones de integración se aplican solo a la copia de la etapa actual.

1. Vuelve a la raíz del repositorio; las rutas siguientes se interpretan desde allí.
2. En CMD ejecuta `for %f in (M04_PATRONES_ARQUITECTURA\PROYECTO\4.1-repository\docs\*.md) do fc /L "%f" "M04_PATRONES_ARQUITECTURA\PROYECTO\4.2-unit-of-work\docs\%~nxf"`. En PowerShell puedes comparar `Get-FileHash` para cada par de documentos.
3. Comprueba 10 comparaciones y exige igualdad en todas; se conserva exactamente el contenido de los documentos heredados.
4. Si una comparación falla, restaura el documento de la etapa actual desde `4.1-repository`. Verifica también que la carpeta del punto anterior no tiene modificaciones.

Verificación concreta. Los 10 documentos heredados coinciden byte a byte y el punto anterior permanece intacto.

Diagnóstico si falla.

- Usar `comp` en lugar de `fc`: `comp` compara binarios y no muestra diferencias de texto.
- Omitir `/L` cuando se desea una comparación de texto: indica el modo explícitamente y revisa cada resultado.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

PASO 4.2.22 — Verificar la estructura del punto 4.2

La verificación estructural confirma que el punto 4.2 añade archivos nuevos conservando los documentos heredados.

1. Escribir `tree /F M04_PATRONES_ARQUITECTURA\PROYECTO\4.2-unit-of-work` y pulsar Enter.
2. Verificar que la salida muestra `AceriaApp.sln`, `README.md`, `src\AceriaApp.Api\UnitOfWork` con las tres clases nuevas, `tests\AceriaApp.UnitOfWork.Tests` con los dos archivos de catálogo y el archivo de tests funcionales y `docs` con el documento del patrón y el ADR-015. Comprueba además todos los archivos de integración de las carpetas Patrones y los cambios de composición indicados en este punto.
3. Verificar que las carpetas existentes del punto 4.1 permanecen intactas.
4. Verificar que no existe ninguna carpeta `labs` ni ningún archivo `metaprompt.md` ni `verificacion.md`.

Verificación concreta. la estructura contiene los archivos nuevos y conserva los del punto 4.1.

Diagnóstico si falla.

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 4.1: viola la regla de no modificar.
- Omitir la carpeta `UnitOfWork`: los tests no encuentran las clases.

PASO 4.2.23 — Verificar el `.gitignore` del punto 4.2

El `.gitignore` se hereda del punto 4.1 sin modificaciones. La verificación confirma que no se ha tocado.

1. Verificar que el archivo `.gitignore` existe en la carpeta del punto 4.2 (copiado del punto 4.1).
2. Abrir el archivo `.gitignore` con Visual Studio 2026.
3. Verificar que contiene las líneas `**/bin/`, `**/obj/`, `.vs/`, `*.user`, `TestResults/` y `*.trx`.
4. Verificar que no se ha modificado respecto al punto 4.1. Pulsar `Ctrl+S` si no hay cambios y cerrar el archivo.

Verificación concreta. el `.gitignore` existe y contiene las seis líneas sin modificaciones.

Diagnóstico si falla.

- Modificar el `.gitignore` copiado del punto 4.1: rompe la conservación de la etapa anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.
- Crear el `.gitignore` con otro nombre: el archivo no se reconoce.

PASO 4.2.24 — Hacer el commit del punto 4.2 y verificar el estado final

El commit permite comparar esta etapa con la anterior. Si has realizado un commit por punto desde 1.1, el recorrido suma 15 commits de puntos; otros commits legítimos pueden aumentar el historial.

1. Desde la raíz del repositorio ejecuta `git status` y revisa que solo aparezcan los archivos de esta etapa y los entregables que has trabajado.
2. Ejecuta `git add .` después de verificar que `bin/`, `obj/`, `TestResults/` y credenciales están excluidos.
3. Ejecuta `git commit -m "Punto 4.2: Unit of Work"` con `user.name` y `user.email` configurados.
4. Ejecuta `git log --oneline` y `git status`; comprueba que se registró el commit de este punto y que no quedan cambios pendientes de esta actividad.

Verificación concreta. El historial incluye el commit de 4.2 y `git status` no muestra cambios pendientes de esta actividad; no se exige un número rígido de commits a un repositorio con más trabajo.

Diagnóstico si falla.

- No tener configurado `user.name` y `user.email`: `git commit` falla.
- Añadir la carpeta `bin/` y `obj/` al commit: el `.gitignore` lo evita.
- Hacer commit antes de `git add .`: no hay cambios en el índice.

Reto de ampliación (optativo)

Trabaja en una copia independiente de la etapa obligatoria, por ejemplo `4.2-unit-of-work-reto`. Mantén intacta la carpeta obligatoria y todos sus documentos. Los tres fragmentos siguientes añaden un contrato, un cálculo y una prueba al proyecto del punto.

Reto: añadir un método `Resumen` a `ValidadorUnidadesDeTrabajo` que devuelva el número total de unidades de trabajo, el número total de repositorios coordinados y la media de repositorios por unidad, y un test que verifique el cálculo con el catálogo por defecto.

Paso 1: En `ValidadorUnidadesDeTrabajo.cs`, añadir el siguiente `record` al final del archivo:

Archivo: `src/AceriaApp.Api/UnitOfWork/ValidadorUnidadesDeTrabajo.cs`. **Operación:** Modificar.

Punto de inserción: después de este texto exacto, que debe aparecer una sola vez:

```
public record ResultadoUnidadesDeTrabajo(bool EsValida, IReadOnlyList<string> Errores);
```

```
public record ResumenUnidades(
    int Total,
    int TotalRepositorios,
    double MediaRepositorios);
```

Paso 2: Explicar el propósito del fragmento y las decisiones que contiene:

La declaración `public record ResumenUnidades(` declara un contrato inmutable. Número total de unidades de trabajo. Número total de repositorios coordinados. Media de repositorios por unidad.

Paso 3: En `ValidadorUnidadesDeTrabajo.cs`, añadir el siguiente método estático dentro de la clase:

Archivo: `src/AceriaApp.Api/UnitOfWork/ValidadorUnidadesDeTrabajo.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
}

public record ResultadoUnidadesDeTrabajo(bool EsValida, IReadOnlyList<string> Errores);
```

Indenta el fragmento con cuatro espacios dentro de la clase; conserva el cierre de clase que aparece en el texto de inserción.

```
public static ResumenUnidades Resumen(IEnumerable<UnidadDeTrabajo> unidades)
{
    var lista = unidades.ToList();
    var total = lista.Count;
    var repositorios = lista.Sum(u => u.RepositoriosCoordinados);
    var media = total == 0 ? 0 : (double)repositorios / total;
    return new ResumenUnidades(total, repositorios, media);
}
```

Paso 4: Explicar el propósito del fragmento y las decisiones que contiene:

La declaración `public static ResumenUnidades Resumen(IEnumerable<UnidadDeTrabajo> unida` declara el método. Materializa la lista. Cuenta las unidades. Suma los repositorios. Calcula la media con protección contra división por cero. Devuelve el resumen.

Paso 5: Guardar el archivo con `Ctrl+S`.

Paso 6: En `ValidadorUnidadesDeTrabajoTests.cs`, añadir el siguiente test:

Archivo: `tests/AceriaApp.UnitOfWork.Tests/ValidadorUnidadesDeTrabajoTests.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
}
```

Indenta el fragmento con cuatro espacios dentro de la clase; conserva el cierre de clase que aparece en el texto de inserción.

```
[Fact]
public void Resumen_ConCatalogoPorDefecto_CalculaMedia()
{
    var resumen = ValidadorUnidadesDeTrabajo.Resumen(CatalogoUnidadesDeTrabajo.UnidadesPorDefecto());

    resumen.Total.Should().Be(2);
    resumen.TotalRepositorios.Should().Be(6);
    resumen.MediaRepositorios.Should().Be(3.0);
}
```

Paso 7: Interpretar el test y explicar qué comportamiento acredita:

Marca el método como test de xUnit. Declara el test. Invoca el método. Verifica el total. Verifica el total de repositorios. Verifica la media.

Paso 8: Ejecuta `dotnet test tests/AceriaApp.UnitOfWork.Tests/AceriaApp.UnitOfWork.Tests.csproj -c Release` en la copia optativa. Deben pasar 16 tests del proyecto, incluido el nuevo resumen. Ejecuta después `dotnet test AceriaApp.sln -c Release` y exige 78 pruebas aprobadas.

Paso 9: Comprueba que solo cambian `ValidadorUnidadesDeTrabajo.cs` y `ValidadorUnidadesDeTrabajoTests.cs` en la copia optativa; la etapa obligatoria `4.2-unit-of-work` y el punto anterior deben permanecer idénticos.

Paso 10: Hacer commit con `git commit -m "Reto 4.2: método Resumen en ValidadorUnidadesDeTrabajo"`.

Verificación: `Passed: 16` en el proyecto del punto y 78 pruebas aprobadas en la solución de la copia optativa, sin fallos ni omisiones.

Resultado verificable

Al finalizar este punto, el alumno debe tener:

- La carpeta `M04_PATRONES_ARQUITECTURA\PROYECTO/4.2-unit-of-work/` con la solución `AceriaApp.sln` que contiene 11 proyectos.
- El documento `docs/patron-unit-of-work.md` con las responsabilidades y las prohibiciones.
- El documento `docs/ADR-015-patron-unit-of-work.md` con la decisión sobre el patrón Unit of Work.
- Las clases `UnidadDeTrabajo`, `ValidadorUnidadesDeTrabajo` y `CatalogoUnidadesDeTrabajo` en `src/AceriaApp.Api/UnitOfWork/`.
- El proyecto `tests/AceriaApp.UnitOfWork.Tests` añade 11 tests de catálogo y 4 funcionales; el reto añade uno más.
- `dotnet test AceriaApp.sln -c Release` ejecuta 77 pruebas en la etapa obligatoria y 78 en una copia con su reto; ninguna falla ni se omite.
- Los ADR-010 a ADR-014 del punto 4.1 sin modificar.
- Un commit revisado para el punto 4.2, con el historial previo conservado.

Continuidad del aprendizaje

El punto 4.2 ha definido el patrón Unit of Work como abstracción para coordinar las operaciones de varios repositorios en una única transacción. El proyecto AceriaApp dispone ahora de un validador automático de unidades de trabajo que comprueba las responsabilidades y las prohibiciones del patrón, y 11 tests nuevos de catálogo más 4 funcionales. Esta base permite avanzar hacia el punto 4.3, donde se estudiará el patrón Mediator con MediatR como mecanismo para desacoplar los casos de uso de sus manejadores.

Punto 4.3 — Mediator

Coordinar solicitudes sin acoplar el emisor a todos los manejadores

El patrón Mediator permite enviar una petición a un manejador sin que el iniciador conozca su implementación concreta. Esta mediación tiene sentido en la capa de aplicación, donde se coordinan operaciones, y no debe convertir el dominio en dependiente de ASP.NET Core. En M04 utilizaremos MediatR para reconocer el registro, el descubrimiento del manejador y el trayecto que sigue una petición real, diferenciándolo de un simple catálogo de mensajes.

El alumno continuará desde la etapa 4.2 y conservará su persistencia. Las nuevas clases de análisis ayudarán a describir mensajes y reglas, pero la prueba decisiva será enviar un comando o consulta al mediador registrado y comprobar que el resultado y sus efectos son los previstos. Además, habrá que mantener separados los errores de configuración de DI de los errores esperados del negocio.

Carpeta de trabajo: `M04_PATRONES_ARQUITECTURA/PROYECTO/4.3-mediator`.

Objetivo: completar los veinticuatro pasos, conservar los documentos heredados y comprobar el patrón con código y tests ejecutables. La etapa suma 90 pruebas en 12 proyectos.

PASO 4.3.1 — Crear la carpeta del punto 4.3 a partir del punto 4.2

La copia preserva una etapa autónoma y reproducible. 4.2-unit-of-work aporta 77 tests aprobados y 11 proyectos antes de añadir este punto.

1. Abre la terminal integrada de Visual Studio y localiza `M04_PATRONES_ARQUITECTURA/PROYECTO/4.2-unit-of-work`; esa es la etapa completa que vas a continuar.
2. Crea `M04_PATRONES_ARQUITECTURA/PROYECTO/4.3-mediator` y copia en ella el contenido de `4.2-unit-of-work`; en Windows puedes usar `xcopy` con `/E /I /H /Y`.
3. Excluye `bin`, `obj` y `TestResults`; abre únicamente la copia de `4.3-mediator` para que el punto anterior permanezca intacto.
4. Ejecuta `dir` dentro de `4.3-mediator` y verifica `AceriaApp.sln`, `README.md`, `src`, `tests`, `docs` y `global.json`.

Verificación concreta. La copia contiene la solución y los 11 proyectos de la etapa anterior; no contiene todavía archivos nuevos de 4.3.

Diagnóstico si falla.

- Usar `copy` en lugar de `xcopy`: `copy` no copia carpetas de forma recursiva.
- Olvidar el flag `/E`: no se copian las subcarpetas vacías.
- Olvidar el flag `/I`: `xcopy` pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag `/Y`: `xcopy` pide confirmación para sobrescribir.

PASO 4.3.2 — Abrir la solución copiada en Visual Studio 2026

Abrir la solución copiada permite trabajar en el punto 4.3 sobre la base del punto 4.2. La compilación verifica que la copia es correcta.

1. Entra en `4.3-mediator` y abre `AceriaApp.sln` con Visual Studio; desde Windows puedes ejecutar `start AceriaApp.sln`.
2. Ejecuta `dotnet sln AceriaApp.sln list` y comprueba los 11 proyectos heredados; todos deben existir bajo esta etapa.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`; conserva la salida como comprobación inicial.
4. Si la API está en ejecución, detenla antes de compilar. Continúa cuando la compilación termine sin errores y las rutas no apunten a otra etapa.

Verificación concreta. `dotnet sln AceriaApp.sln list` muestra 11 proyectos y `dotnet build AceriaApp.sln -c Release` termina con `Build succeeded.`.

Diagnóstico si falla.

- Abrir la solución del punto 4.2 en lugar de la del punto 4.3: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener archivos DLL bloqueados por una API en ejecución: detén la aplicación antes de recompilar.

PASO 4.3.3 — Verificar que la carpeta `docs` contiene los documentos del punto 4.2

La etapa conserva los 12 documentos anteriores y añade únicamente su ficha de patrón y su ADR. Las modificaciones de código se escriben en la copia actual.

1. Expande `docs` dentro de la copia de `4.3-mediator`.
2. Comprueba los 12 documentos heredados: los ADR-010 a ADR-015 y las fichas de los puntos anteriores.

3. Compara sus nombres y contenido con `docs` de `4.2-unit-of-work`. No modifiques ni elimines los documentos anteriores.
4. Confirma que todavía no existen `docs/patron-mediator.md` ni `docs/ADR-016-patron-mediator.md`; se crearán en este punto.

Verificación concreta. `docs` contiene 12 archivos heredados idénticos; los dos documentos de este punto todavía no existen.

Diagnóstico si falla.

- Modificar alguno de los ADR o documentos existentes: rompe la conservación de la etapa anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 4.2.
- Crear la carpeta `docs` si ya existe: `mkdir` falla.

PASO 4.3.4 — Crear el documento del patrón Mediator en `docs`

El documento del patrón Mediator es el artefacto de `docs` específico del punto 4.3. No modifica los 12 documentos existentes del punto 4.2.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `docs`.
2. Seleccionar `Add` → `New Item` → `Text File` y nombrar el archivo `patron-mediator.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que el archivo aparece en la carpeta `docs` sin subrayados rojos.

Archivo: `docs/patron-mediator.md`. **Operación:** Crear.

```
# Patrón Mediator — Punto 4.3

## Definición

Patrón que reduce el acoplamiento entre los componentes al centralizar la comunicación en un objeto mediador.

## Mediator en la capa de aplicación

Los adaptadores de entrada envían comandos y consultas al mediador. El mediador los distribuye al manejador
correspondiente.

## Comandos

Mensajes que solicitan una acción que modifica el estado. Se nombran con un verbo en imperativo.

## Consultas

Mensajes que solicitan información sin modificar el estado. Se nombran con un sustantivo o con un verbo en
forma interrogativa.

## Notificaciones

Mensajes que informan de un evento ocurrido. Se nombran con un sustantivo en pasado. Pueden tener varios
manejadores.

## MediatR

Biblioteca de .NET que implementa el patrón Mediator. Ofrece `IRequest`, `IRequest<T>`, `INotification`,
`IRequestHandler` e `INotificationHandler`.

## Envío de comandos y consultas

En MediatR 12.4.1 se utiliza `Send`, que ya devuelve una tarea y se espera con `await`. Los endpoints
dependen de `ISender` para enviar comandos y consultas; no existe un método `SendAsync` en esta API.

## Publicación de notificaciones

MediatR 12.4.1 publica con `Publish`, que ya devuelve una tarea y se espera con `await`; no ofrece
`PublishAsync`. Las notificaciones se estudian en el catálogo de este punto y no se implementa aquí un
publicador de eventos de negocio.
```

```
## MediatR y validación

Un pipeline puede integrar FluentValidation como variante de diseño. El flujo ejecutable de este punto
registra `LoggingBehavior`, que registra inicio y fin sin datos del cliente y comprueba cancelación; las
comprobaciones de entrada permanecen en el endpoint y en el caso de uso. No se registra un pipeline de
FluentValidation para esta ruta.

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Comando ejecutable | CrearOrdenCommand |
| Consulta ejecutable | ObtenerOrdenQuery |
| Notificaciones | Ejemplos del catálogo; no se publican en este flujo |
| Manejadores | Implementan IRequestHandler |
| MediatR | Registrado con AddMediatR |
| Pipeline ejecutable | LoggingBehavior para observación y cancelación |

## Correspondencia con el código ejecutable

El catálogo de este punto describe reglas de diseño y contiene ejemplos de contratos. Sus nombres no implican
servicios registrados. La implementación utiliza los proyectos `AceriaApp.Hexagonal.Aplicacion`,
`AceriaApp.Hexagonal.Dominio` y `AceriaApp.Hexagonal.Infraestructura`, compuestos desde `AceriaApp.Api`.
Comprueba los archivos y los tests funcionales que incorpora el paso de compilación de este punto.
```

Verificación concreta. El archivo `docs/patron-mediator.md` existe y contiene las 11 secciones mostradas; abre su vista previa y comprueba todos los apartados.

Diagnóstico si falla.

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: rompe la conservación de la etapa anterior.
- Guardar el archivo en la raíz del punto en lugar de `docs`: rompe la estructura del repositorio.

PASO 4.3.5 — Añadir el paquete `MediatR` al proyecto API

MediatR solo aporta una capacidad de mediación cuando los mensajes y sus manejadores se registran correctamente. Verifica la versión NuGet y la compatibilidad con la solución antes de conectar el nuevo recorrido.

1. Abre `src/AceriaApp.Hexagonal.Aplicacion/AceriaApp.Hexagonal.Aplicacion.csproj`, donde vivirán los mensajes y manejadores reales.
2. Ejecuta `dotnet add src/AceriaApp.Hexagonal.Aplicacion/AceriaApp.Hexagonal.Aplicacion.csproj package MediatR --version 12.4.1`.
3. Ejecuta `dotnet list src/AceriaApp.Hexagonal.Aplicacion/AceriaApp.Hexagonal.Aplicacion.csproj package` y comprueba la versión `12.4.1`.
4. Guarda el archivo. La integración de servicios viene incluida en MediatR 12 y se registrará con el ensamblado de los manejadores durante la integración funcional.

Integración funcional. Instala MediatR 12.4.1 en la capa que define mensajes y handlers, y Logging.Abstractions 8.0.3 para el pipeline. El dominio permanece sin dependencias de infraestructura.

Archivo: `src/AceriaApp.Hexagonal.Aplicacion/AceriaApp.Hexagonal.Aplicacion.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings></PropertyGroup>
  <ItemGroup><ProjectReference Include="../AceriaApp.Hexagonal.Dominio/AceriaApp.Hexagonal.Dominio.csproj" /
></ItemGroup>
  <ItemGroup>
    <PackageReference Include="MediatR" Version="12.4.1" />
    <PackageReference Include="Microsoft.Extensions.Logging.Abstractions" Version="8.0.3" />
```

```
  </ItemGroup>
</Project>
```

Compila para .NET 8, igual que los proyectos heredados. Mantiene la referencia a la capa necesaria; los tests pueden alcanzar la API y sus referencias transitivas. Fija MediatR 12.4.1 en la aplicación, donde se definen mensajes, manejadores y pipeline.

Verificación concreta. La capa de aplicación referencia `MediatR 12.4.1`; la capa de dominio no contiene esa dependencia.

Diagnóstico si falla.

- Instalar una versión preliminar: el curso usa la versión fijada en el archivo de proyecto.
- Instalar `MediatR.Extensions.Microsoft.DependencyInjection` como paquete separado: a partir de MediatR 12 el paquete de integración está incluido.
- Añadir el paquete al proyecto de tests: los manejadores no se resuelven en producción.

PASO 4.3.6 — Crear la carpeta `Mediator` en el proyecto API

La carpeta `Mediator` albergará las clases de validación del patrón Mediator. Se crea como carpeta nueva sin tocar las carpetas existentes.

1. En el panel Solution Explorer, hacer clic derecho sobre el proyecto `AceriaApp.Api`.
2. Seleccionar `Add` → `New Folder` y nombrar la carpeta `Mediator`.
3. Verificar que la carpeta `Mediator` aparece dentro de `AceriaApp.Api` sin afectar a las carpetas existentes.
4. Verificar que no se ha modificado ningún archivo existente del punto 4.2.

Verificación concreta. la carpeta `Mediator` existe dentro de `AceriaApp.Api` y las carpetas existentes permanecen intactas.

Diagnóstico si falla.

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: rompe la conservación de la etapa anterior.
- Crear la carpeta con otro nombre: rompe la convención del curso.

PASO 4.3.7 — Crear la clase `Mensaje`

`Mensaje` describe los datos que permiten clasificar solicitudes dentro del catálogo de este punto. El mensaje enviado por `ISender` o `IMediator` es un contrato distinto que necesita su propio manejador y registro efectivo.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `Mediator`.
2. Seleccionar `Add` → `Class` y nombrar la clase `Mensaje.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/Mediator/Mensaje.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Mediator;

public class Mensaje
{
    public string Nombre { get; init; } = string.Empty;
    public string Tipo { get; init; } = string.Empty;
    public bool TieneManejador { get; init; }
```

```
    public bool DevuelveResultado { get; init; }
}
```

Verificación concreta. la clase `Mensaje` compila sin errores y expone cuatro propiedades.

Diagnóstico si falla.

- Olvidar el `namespace`: la clase no se resuelve desde otros archivos.
- No inicializar las propiedades de tipo `string` con `string.Empty`: el compilador emite advertencias.
- Usar `set` en lugar de `init`: permite modificar la instancia después de crearla.

PASO 4.3.8 — Crear la clase `ValidadorMensajes`

El catálogo puede rechazar descripciones ambiguas de mensajes o de sus responsabilidades. Esa validación no prueba que MediatR descubra el handler; se necesitará resolverlo y ejecutar una petición real.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `Mediator`.
2. Seleccionar `Add` → `Class` y nombrar la clase `ValidadorMensajes.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/Mediator/ValidadorMensajes.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Mediator;

public class ValidadorMensajes
{
    private static readonly string[] TiposPermitidos = { "Comando", "Consulta", "Notificacion" };

    public ResultadoMensajes Validar(IEnumerable<Mensaje> mensajes)
    {
        var errores = new List<string>();
        var lista = mensajes.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay mensajes declarados.");
        }

        foreach (var mensaje in lista)
        {
            if (string.IsNullOrWhiteSpace(mensaje.Nombre))
            {
                errores.Add("Un mensaje no declara el nombre.");
            }

            if (!TiposPermitidos.Contains(mensaje.Tipo))
            {
                errores.Add($"Tipo de mensaje no permitido en '{mensaje.Nombre}': '{mensaje.Tipo}'.");
            }

            if (!mensaje.TieneManejador)
            {
                errores.Add($"El mensaje '{mensaje.Nombre}' no tiene manejador registrado.");
            }

            if (mensaje.Tipo == "Notificacion" && mensaje.DevuelveResultado)
            {
                errores.Add($"La notificación '{mensaje.Nombre}' no puede devolver resultado.");
            }
        }

        return new ResultadoMensajes(errores.Count == 0, errores);
    }
}

public record ResultadoMensajes(bool EsValido, IReadOnlyList<string> Errores);
```

Verificación concreta. la clase `ValidadorMensajes` compila sin errores y expone un método y un `record`.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<string>` e `IReadOnlyList<string>` no se resuelven.
- Olvidar el `using System.Linq;`: `Contains` y `ToList` no se resuelven.
- No comprobar el caso de lista vacía: el validador devuelve válido sin mensajes.

PASO 4.3.9 — Añadir los `using` necesarios en `ValidadorMensajes`

Los tipos `List<string>` e `IReadOnlyList<string>` pertenecen a `System.Collections.Generic`. Los métodos `Contains` y `ToList` pertenecen a `System.Linq`. En proyectos con `ImplicitUsings` desactivado, hay que añadir los usings explícitos.

1. En el panel Solution Explorer, hacer doble clic en `ValidadorMensajes.cs`.
2. Inserta las directivas indicadas antes del namespace del archivo; la indicación del archivo identifica la línea exacta.
3. Verificar que no hay subrayados rojos en `List<string>`, `Contains` ni `ToList`.
4. Pulsar `Ctrl+S` para guardar el archivo.

Archivo: `src/AceriaApp.Api/Mediator/ValidadorMensajes.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
namespace AceriaApp.Api.Mediator;
```

```
using System.Collections.Generic;
using System.Linq;
```

Verificación concreta. `ValidadorMensajes.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir solo `System.Collections.Generic` y olvidar `System.Linq`: `Contains` y `ToList` no se resuelven.
- Mezclar directivas using con declaraciones de tipos: coloca las directivas antes del namespace para mantener el orden del archivo.
- Olvidar los usings y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 4.3.10 — Crear la clase `CatalogoMensajes`

La clase `CatalogoMensajes` contiene los mensajes por defecto del proyecto. Se usan en los tests y en la validación automática.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `Mediator`.
2. Seleccionar `Add` → `Class` y nombrar la clase `CatalogoMensajes.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/Mediator/CatalogoMensajes.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Mediator;

public class CatalogoMensajes
{
    public static List<Mensaje> MensajesPorDefecto()
    {
        return new List<Mensaje>
        {
            new() { Nombre = "CrearOrdenCommand", Tipo = "Comando", TieneManejador = true, DevuelveResultado
```

```
= true },
            new() { Nombre = "CancelarOrdenCommand", Tipo = "Comando", TieneManejador = true,
DevuelveResultado = false },
            new() { Nombre = "ObtenerOrdenQuery", Tipo = "Consulta", TieneManejador = true, DevuelveResultado
= true },
            new() { Nombre = "ListarOrdenesQuery", Tipo = "Consulta", TieneManejador = true,
DevuelveResultado = true },
            new() { Nombre = "OrdenCreadaNotification", Tipo = "Notificacion", TieneManejador = true,
DevuelveResultado = false }
        };
    }
}
```

Verificación concreta. la clase `CatalogoMensajes` compila sin errores y expone un método estático.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<T>` no se resuelve.
- No inicializar las propiedades de los objetos: el compilador emite advertencias.
- Reutilizar una lista estática mutable: cada llamada del catálogo debe construir una lista independiente.

PASO 4.3.11 — Añadir el `using` necesario en `CatalogoMensajes`

El tipo `List<T>` pertenece a `System.Collections.Generic`. En proyectos con `ImplicitUsings` desactivado, hay que añadir el using explícito.

1. En el panel Solution Explorer, hacer doble clic en `CatalogoMensajes.cs`.
2. Inserta las directivas indicadas antes del namespace del archivo; la indicación del archivo identifica la línea exacta.
3. Verificar que no hay subrayados rojos en `List<T>`.
4. Pulsar `Ctrl+S` para guardar el archivo.

Archivo: `src/AceriaApp.Api/Mediator/CatalogoMensajes.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
namespace AceriaApp.Api.Mediator;
```

```
using System.Collections.Generic;
```

Verificación concreta. `CatalogoMensajes.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir `using System.Linq;` en lugar de `System.Collections.Generic`: no resuelve `List<T>`.
- Colocar el using después de una clase: las directivas deben preceder las declaraciones de tipos.
- Olvidar el using y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 4.3.12 — Crear el proyecto de tests del punto 4.3

El punto 4.3 añade un proyecto de tests específico para las clases del patrón Mediator. Se crea como proyecto nuevo y conserva las pruebas previas; las actualizaciones explícitas de integración se muestran en su paso correspondiente.

1. En `tests` crea la carpeta `AceriaApp.Mediator.Tests` y, dentro, el archivo `AceriaApp.Mediator.Tests.csproj` con el XML completo de este paso. En Visual Studio puedes elegir la plantilla xUnit y ajustar el archivo generado.

2. Elimina el archivo de ejemplo `UnitTest1.cs` si lo creó la plantilla; los archivos de pruebas completos se escriben en sus pasos correspondientes.
3. Ejecuta `dotnet sln AceriaApp.sln add tests/AceriaApp.Mediator.Tests/AceriaApp.Mediator.Tests.csproj` incluso si la solución ya existía y después `dotnet sln AceriaApp.sln list`.
4. Comprueba que el proyecto aparece bajo `tests` y que `dotnet restore tests/AceriaApp.Mediator.Tests/AceriaApp.Mediator.Tests.csproj` termina correctamente.

Archivo: `tests/AceriaApp.Mediator.Tests/AceriaApp.Mediator.Tests.csproj`. **Operación:** Crear.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. La solución muestra 12 proyectos, incluido `AceriaApp.Mediator.Tests`; el nuevo csproj fija .NET 8, SDK de tests 17.12.0, xUnit 2.9.2 y runner 2.8.2.

Diagnóstico si falla.

- Crear el proyecto dentro de `src` en lugar de `tests`: rompe la estructura.
- Modificar los proyectos de tests del punto 4.2: rompe la conservación de la etapa anterior.
- Usar `dotnet new mstest`: el curso usa xUnit.

PASO 4.3.13 — Añadir la referencia del proyecto de tests a la API

La referencia permite que el proyecto de tests acceda a las clases de `Mediator` del proyecto API.

1. Ejecuta `dotnet add tests/AceriaApp.Mediator.Tests/AceriaApp.Mediator.Tests.csproj reference src/AceriaApp.Api/AceriaApp.Api.csproj` o deja el archivo exactamente como el bloque XML de este paso.
2. Comprueba que la referencia va de tests a API; la API nunca debe referenciar el proyecto de tests.
3. Abre `tests/AceriaApp.Mediator.Tests/AceriaApp.Mediator.Tests.csproj` y comprueba el único `ProjectReference` a `../../src/AceriaApp.Api/AceriaApp.Api.csproj`.
4. Ejecuta `dotnet build tests/AceriaApp.Mediator.Tests/AceriaApp.Mediator.Tests.csproj -c Release` y verifica que el proyecto resuelve la API y sus dependencias.

Archivo: `tests/AceriaApp.Mediator.Tests/AceriaApp.Mediator.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
```

```
  </ItemGroup>
</Project>
```

Verificación concreta. el archivo `AceriaApp.Mediator.Tests.csproj` contiene un `ProjectReference` al proyecto API.

Diagnóstico si falla.

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo `.csproj`: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

PASO 4.3.14 — Añadir el paquete `FluentAssertions` al proyecto de tests

`FluentAssertions` proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

1. Ejecuta `dotnet add tests/AceriaApp.Mediator.Tests/AceriaApp.Mediator.Tests.csproj package FluentAssertions --version 6.12.2` o reemplaza el csproj por el XML completo indicado.
2. Ejecuta `dotnet list tests/AceriaApp.Mediator.Tests/AceriaApp.Mediator.Tests.csproj package` y localiza `FluentAssertions 6.12.2`.
3. Comprueba que el paquete se añade al nuevo proyecto de tests y que se mantienen la referencia a la API y las versiones de xUnit.
4. Guarda el archivo y ejecuta `dotnet build tests/AceriaApp.Mediator.Tests/AceriaApp.Mediator.Tests.csproj -c Release`; las aserciones `Should()` estarán disponibles cuando escribas las pruebas.

Archivo: `tests/AceriaApp.Mediator.Tests/AceriaApp.Mediator.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. El archivo de proyecto y `dotnet list package` muestran `FluentAssertions 6.12.2`; la compilación finaliza correctamente.

Diagnóstico si falla.

- Instalar una versión preliminar: el curso usa la versión fijada en el archivo de proyecto.
- Olvidar el paquete: `Should()` no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

PASO 4.3.15 — Crear el archivo `ValidadorMensajesTests.cs`

Los tests verifican que el validador acepta el catálogo por defecto y rechaza los mensajes que violan las reglas del patrón.

1. En el panel Solution Explorer, expandir `tests` → `AceriaApp.Mediator.Tests`.
2. Crear el archivo `ValidadorMensajesTests.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `tests/AceriaApp.Mediator.Tests/ValidadorMensajesTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.Mediator;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Mediator.Tests;

public class ValidadorMensajesTests
{
    private readonly ValidadorMensajes _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoMensajes.MensajesPorDefecto());

        resultado.EsValido.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinMensajes_DevuelveError()
    {
        var resultado = _validador.Validar(new List<Mensaje>());

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay mensajes"));
    }

    [Fact]
    public void Validar_TipoNoPermitido_DevuelveError()
    {
        var mensajes = new List<Mensaje>
        {
            new() { Nombre = "MensajeX", Tipo = "TipoInventado", TieneManejador = true }
        };

        var resultado = _validador.Validar(mensajes);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Tipo de mensaje no permitido"));
    }

    [Fact]
    public void Validar_SinManejador_DevuelveError()
    {
        var mensajes = new List<Mensaje>
        {
            new() { Nombre = "MensajeX", Tipo = "Comando", TieneManejador = false }
        };

        var resultado = _validador.Validar(mensajes);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tiene manejador"));
    }

    [Fact]
    public void Validar_NotificacionConResultado_DevuelveError()
    {
        var mensajes = new List<Mensaje>
```

```
        {
            new() { Nombre = "NotificacionX", Tipo = "Notificacion", TieneManejador = true, DevuelveResultado
= true }
        };

        var resultado = _validador.Validar(mensajes);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no puede devolver resultado"));
    }
}
```

Verificación concreta. el archivo `ValidadorMensajesTests.cs` compila y contiene cinco métodos de test.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.Mediator;`: las clases no se resuelven.
- Usar `Assert.True` en lugar de `Should().BeTrue()`: el curso usa FluentAssertions.
- No cubrir el caso de notificación con resultado: la regla específica no se verifica.

PASO 4.3.16 — Crear el archivo `CatalogoMensajesTests.cs`

Los tests verifican que el catálogo por defecto contiene los mensajes esperados y la distribución correcta por tipo.

1. En el panel Solution Explorer, hacer clic derecho sobre el proyecto `AceriaApp.Mediator.Tests`.
2. Seleccionar `Add` → `Class` y nombrar la clase `CatalogoMensajesTests.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `tests/AceriaApp.Mediator.Tests/CatalogoMensajesTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.Mediator;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Mediator.Tests;

public class CatalogoMensajesTests
{
    [Fact]
    public void MensajesPorDefecto_ContieneCincoMensajes()
    {
        var mensajes = CatalogoMensajes.MensajesPorDefecto();

        mensajes.Should().HaveCount(5);
    }

    [Fact]
    public void MensajesPorDefecto_ContieneDosComandos()
    {
        var mensajes = CatalogoMensajes.MensajesPorDefecto();

        mensajes.Count(m => m.Tipo == "Comando").Should().Be(2);
    }

    [Fact]
    public void MensajesPorDefecto_ContieneDosConsultas()
    {
        var mensajes = CatalogoMensajes.MensajesPorDefecto();

        mensajes.Count(m => m.Tipo == "Consulta").Should().Be(2);
    }

    [Fact]
    public void MensajesPorDefecto_ContieneUnaNotificacion()
    {
        var mensajes = CatalogoMensajes.MensajesPorDefecto();
```

```
        mensajes.Count(m => m.Tipo == "Notificacion").Should().Be(1);
    }

    [Fact]
    public void MensajesPorDefecto_TodosTienenManejador()
    {
        var mensajes = CatalogoMensajes.MensajesPorDefecto();

        mensajes.Should().OnlyContain(m => m.TieneManejador);
    }
}
```

Verificación concreta. el archivo `CatalogoMensajesTests.cs` compila y contiene cinco métodos de test.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.Mediator;`: las clases no se resuelven.
- Olvidar el `using System.Linq;`: `Count` y `OnlyContain` no se resuelven.
- No verificar la distribución por tipo: el catálogo podría estar mal balanceado.

PASO 4.3.17 — Conectar MediatR y compilar la solución

El resultado de MediatR debe comprobarse desde el host real y también con el caso de uso aislado cuando corresponda. Una llamada `Send` con un manejador mal registrado debe fallar explícitamente; no se atribuirá su éxito a un catálogo correcto.

1. Abre cada archivo de integración funcional indicado en este paso. Crea los nuevos y reemplaza únicamente los que se señalen; copia todo el código mostrado.
2. Recorre las referencias: el dominio define sus entidades, la aplicación usa puertos y la infraestructura implementa persistencia; la API compone el flujo.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`; comprueba que todos los proyectos nuevos estén incluidos en la solución.
4. Verifica `Build succeeded.` y `0 Error(s)`. Si hay advertencias, revisa su causa; en el paso siguiente ejecutarás las pruebas del comportamiento completo.

Integración funcional. Expone /api/v1/patrones/ordenes sin alterar los endpoints heredados. 4.2 usa el caso transaccional; 4.3 enruta por ISender; 4.4 traduce Result a 201/200 y ProblemDetails 400/404/409 con código estable.

Archivo: `src/AceriaApp.Api/Patrones/PatronesOrdenesEndpoints.cs`. **Operación:** Reemplazar el contenido completo.

```
using MediatR;
using AceriaApp.Api.Hexagonal;
using AceriaApp.Hexagonal.Aplicacion.Patrones;
namespace AceriaApp.Api.Patrones;

public static class PatronesOrdenesEndpoints
{
    public static IEndpointRouteBuilder MapPatronesOrdenes(this IEndpointRouteBuilder app)
    {
        var group = app.MapGroup("/api/v1/patrones/ordenes").WithTags("Patrones");
        group.MapGet("/{id:guid}", async (Guid id, ISender sender, CancellationToken ct) =>
        {
            var orden = await sender.Send(new ObtenerOrdenQuery(id), ct);
            return orden is null ? Results.NotFound() : Results.Ok(OrdenHexagonalDto.Desde(orden));
        }).WithName("ObtenerOrdenPatrones").Produces<OrdenHexagonalDto>(200).Produces(404);
        group.MapPost("", async (CrearOrdenHexagonalRequest body, ISender sender, CancellationToken ct) =>
        {
            if (string.IsNullOrWhiteSpace(body.Cliente) || string.IsNullOrWhiteSpace(body.Material)
                || body.Cliente.Trim().Length > 100 || body.Material.Trim().Length > 100
                || body.Cantidad <= 0 || body.Cantidad > 500)
                return Results.Problem(title: "Datos de la orden inválidos.", statusCode: 400);
            var orden = await sender.Send(new CrearOrdenCommand(body.Cliente, body.Material, body.Cantidad),
ct);
```

```
            return Results.CreatedAtRoute("ObtenerOrdenPatrones", new { id = orden.Id },
OrdenHexagonalDto.Desde(orden));
        }).WithName("CrearOrdenPatrones").Produces<OrdenHexagonalDto>(201).ProducesProblem(400);
        return app;
    }
}
```

Aísla la ruta de este módulo y conserva los endpoints existentes de M02 y M03. La API solo construye la consulta y la envía; el contenedor selecciona su manejador. Traduce la petición HTTP en un comando de aplicación y propaga la cancelación.

Devuelve 201 y una cabecera Location calculada con la ruta de lectura nombrada y el identificador creado.

Integración funcional. Integra únicamente registros y mapeo del endpoint de patrones. Conserva íntegros los endpoints, documentación y arranque de M2/M3; en 4.5 reemplaza registros repetidos por AddPatronesOrdenes().

Archivo: `src/AceriaApp.Api/Program.cs`. **Operación:** Reemplazar el contenido completo.

```
using AceriaApp.Api.Patrones;
using AceriaApp.Hexagonal.Aplicacion.Patrones;
using AceriaApp.Hexagonal.Infraestructura.Patrones;
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Validacion;
using FluentValidation;
using AceriaApp.Api.Documentacion;

using AceriaApp.Api.Hexagonal;
using AceriaApp.Hexagonal.Infraestructura;
using AceriaApp.Hexagonal.Infraestructura.Datos;
var builder=WebApplication.CreateBuilder(args);
// INICIO PASO 2.1.12: SQL Server Express LocalDB, no SQLite.
builder.Services.AddDbContext<AceriaDbContext>(o=>
    o.UseSqlServer(builder.Configuration.GetConnectionString("AceriaApp")));
// PASO 2.3.14: registrar validadores en el contenedor.
builder.Services.AddValidatorsFromAssemblyContaining<CrearOrdenValidator>(); // PASO 2.3.14
// PASO 2.4.6: registrar generador antes de Build.
builder.Services.AddDocumentacion(); // PASO 2.4.6

builder.Services.AddServiciosHexagonales(builder.Configuration);
builder.Services.AddScoped<IOrdenTrabajoRepositorio, OrdenTrabajoRepositorio>();
builder.Services.AddScoped<IUnitOfWork, EfUnitOfWork>();
builder.Services.AddScoped<OrdenesTransaccionales>();
builder.Services.AddMediatR(cfg =>
{
    cfg.RegisterServicesFromAssemblyContaining<CrearOrdenCommand>();
    cfg.Lifetime = ServiceLifetime.Scoped;
    cfg.AddOpenBehavior(typeof(LoggingBehavior<,>));
});
var app=builder.Build();
// EnsureCreated se utiliza SOLO como ayuda didáctica en Development.
// Para un sistema productivo se utilizarían migraciones controladas.
// PASO 2.1.13: inicializar LocalDB solo en Development.
if(app.Environment.IsDevelopment()&&!app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope=app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<AceriaDbContext>().Database.EnsureCreatedAsync();
}

// INICIO PASO 2.1.14: colección de recursos, consulta real con EF Core.
app.MapGet("/api/v1/ordenes",async (AceriaDbContext db,CancellationToken ct)=>
{
    // PASO 2.1.15: lectura sin tracking y proyeccion a DTO.
    var ordenes=await db.Ordenes.AsNoTracking().OrderBy(x=>x.FechaCreacion).ToListAsync(ct);
    return Results.Ok(ordenes.Select(OrdenDto.Desde));
})
.WithName("ListarOrdenes").WithTags("Ordenes")
.WithSummary("Lista las órdenes de producción").Produces<IEnumerable<OrdenDto>>(200)
```

```
;
// INICIO PASO 2.1.17: recurso individual.
app.MapGet("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(x=>x.Id==id);
    // PASO 2.1.18: 200 para orden presente; 404 si no existe.
    IResult response=orden is null ? Results.NotFound() : Results.Ok(OrdenDto.Desde(orden));
    return response;
})
.WithName("ObtenerOrdenPorId").WithTags("Ordenes")
.WithSummary("Obtiene una orden por ID").Produces<OrdenDto>(200).Produces(404)
;


// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    // PASO 2.2.6: crear entidad y persistirla.
    var orden=new Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    // PASO 2.2.7: devolver 201 con Location.
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")
// PASO 2.3.15: filtro de POST.
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Crea una orden").Produces<OrdenDto>(201).ProducesValidationProblem(400).ProducesProblem(422)
;


// INICIO PASO 2.2.11: sustitución completa de los campos editables.
app.MapPut("/api/v1/ordenes/{id:guid}",async (Guid id,CrearOrdenRequest request,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cliente=request.Cliente;orden.Material=request.Material;
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("ReemplazarOrden").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Sustituye campos
editables").Produces(204).Produces(404).ProducesProblem(409).ProducesValidationProblem(400).ProducesProblem(4
22)
;


// INICIO PASO 2.2.14: cambiar SOLO la cantidad.
app.MapPatch("/api/v1/ordenes/{id:guid}/cantidad",
    async (Guid id,ActualizarCantidadRequest request,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("CambiarCantidad").WithTags("Ordenes")
// PASO 2.3.17: filtro de PATCH.
.AddEndpointFilter<ValidationFilter<ActualizarCantidadRequest>>()
.WithSummary("Modifica
cantidad").Produces<OrdenDto>(200).Produces(404).ProducesValidationProblem(400).ProducesProblem(409).Produces
Problem(422)
```

```
;


// INICIO PASO 2.2.16: eliminar recurso persistido.
app.MapDelete("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede eliminar una orden finalizada.",statusCode:409);
    db.Ordenes.Remove(orden);
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("EliminarOrden").WithTags("Ordenes")
.WithSummary("Elimina una orden pendiente").Produces(204).Produces(404).ProducesProblem(409)
;
// INICIO PASO 2.2.18: transición explícita del dominio.
app.MapPost("/api/v1/ordenes/{id:guid}/finalizar",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"La orden ya estaba finalizada.",statusCode:409);
    orden.Estado="Finalizada";
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("FinalizarOrden").WithTags("Ordenes")
.WithSummary("Finaliza una orden").Produces<OrdenDto>(200).Produces(404).ProducesProblem(409)
;

// PASO 2.4.8: habilitar Swagger solo en Development.
app.UseDocumentacion(); // PASO 2.4.8
// El contexto hexagonal usa su propia base para que EnsureCreated no dependa de
// las tablas heredadas del curso M02.
if (app.Environment.IsDevelopment() && !app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope = app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<HexagonalDbContext>()
        .Database.EnsureCreatedAsync();
}
app.MapHexagonalOrdenes();
app.MapPatronesOrdenes();
app.Run();
public partial class Program {} // permite WebApplicationFactory en tests
```

Permite mapear los endpoints del módulo desde la raíz de composición. Permite registrar las implementaciones de los puertos desde la API. Mantiene el registro SQL Server y los servicios que ya utilizaba M03.

Durante esta etapa se registra explícitamente el puerto de escritura con su adaptador. La unidad de trabajo comparte el contexto Scoped de la petición. Conecta MediatR con los manejadores reales del ensamblado de aplicación.

Registra el logging como comportamiento transversal del mediador. Publica las rutas del nuevo flujo después de construir la aplicación y antes de ejecutarla.

Integración funcional. Implementa IPipelineBehavior con logging de inicio y fin del tipo de petición. Propaga cancelación y excepciones; nunca registra datos del cliente ni hace casts de TResponse.

Archivo: `src/AceriaApp.Hexagonal.Aplicacion/Patrones/LoggingBehavior.cs`. **Operación:** Crear.

```
using MediatR;
using Microsoft.Extensions.Logging;
namespace AceriaApp.Hexagonal.Aplicacion.Patrones;

/// <summary>Observa el flujo sin registrar payloads ni alterar tipos de respuesta.</summary>
public sealed class LoggingBehavior<TRequest, TResponse>(ILogger<LoggingBehavior<TRequest, TResponse>>
logger)
    : IPipelineBehavior<TRequest, TResponse> where TRequest : notnull
```

```
{
    public async Task<TResponse> Handle(TRequest request, RequestHandlerDelegate<TResponse> next,
CancellationToken ct)
    {
        ct.ThrowIfCancellationRequested();
        logger.LogInformation("Inicio {Solicitud}", typeof(TRequest).Name);
        var response = await next();
        logger.LogInformation("Fin {Solicitud}", typeof(TRequest).Name);
        return response;
    }
}
```

El comportamiento genérico envuelve solicitudes y respuestas sin acoplarse a un comando concreto. Recibe la solicitud, el siguiente delegado del pipeline y el token de cancelación. Interrumpe el flujo antes del manejador cuando el llamador ya canceló; no se inicia una escritura.

Registra solo el nombre del tipo. No serializa el contenido del comando ni datos del cliente. Invoca exactamente una vez el siguiente comportamiento o el manejador, preservando el tipo de respuesta de MediatR 12.4.1. Registra la finalización normal después de obtener la respuesta. Una excepción se propaga y no produce este mensaje de finalización.

Entrega la respuesta original sin transformarla.

Integración funcional. Define command y query tipados y handlers que invocan el caso de uso real. En 4.4 los mensajes pasan a `Result<OrdenHexagonal>` y se añade FinalizarOrdenCommand, conservando el mismo flujo MediatR.

Archivo: `src/AceriaApp.Hexagonal.Aplicacion/Patrones/MensajesOrden.cs`. **Operación:** Crear.

```
using AceriaApp.Hexagonal.Dominio;
using MediatR;
namespace AceriaApp.Hexagonal.Aplicacion.Patrones;

public sealed record CrearOrdenCommand(string Cliente, string Material, int Cantidad) :
IRequest<OrdenHexagonal>;
public sealed record ObtenerOrdenQuery(Guid Id) : IRequest<OrdenHexagonal?>;

public sealed class CrearOrdenHandler(OrdenesTransaccionales caso) : IRequestHandler<CrearOrdenCommand,
OrdenHexagonal>
{
    public Task<OrdenHexagonal> Handle(CrearOrdenCommand request, CancellationToken ct) =>
        caso.CrearAsync(request.Cliente, request.Material, request.Cantidad, ct);
}
public sealed class ObtenerOrdenHandler(OrdenesTransaccionales caso) : IRequestHandler<ObtenerOrdenQuery,
OrdenHexagonal?>
{
    public Task<OrdenHexagonal?> Handle(ObtenerOrdenQuery request, CancellationToken ct) =>
caso.ObtenerAsync(request.Id, ct);
}
```

El comando transporta los datos de entrada e implementa IRequest con su tipo de respuesta; MediatR utiliza ese contrato para seleccionar el manejador. La consulta solicita una orden por identificador y declara el resultado que espera el emisor. El manejador recibe el caso de uso por constructor; no recibe DbContext ni construye infraestructura.

Delega la operación completa y transmite la cancelación. La regla de negocio y la confirmación siguen en el caso de uso. MediatR asocia esta clase al contrato de consulta; su único trabajo es delegar la lectura.

Integración funcional. Conserva las dependencias de los tests de catálogo y añade Testcontainers.MsSql, el helper compartido y WebApplicationFactory para las pruebas funcionales.

Archivo: `tests/AceriaApp.Mediator.Tests/AceriaApp.Mediator.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings><IsPackable>false</IsPackable><IsTestProject>true</IsTestProject></PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="Testcontainers.MsSql" Version="3.10.0" />
    <PackageReference Include="Microsoft.AspNetCore.Mvc.Testing" Version="8.0.21" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <Compile Include="../Compartido/SqlServerPruebas.cs" Link="SqlServerPruebas.cs" />
  </ItemGroup>
</Project>
```

Compila para .NET 8, igual que los proyectos heredados. Fija el cliente de contenedores para SQL Server; el proveedor EF Core SQL Server llega mediante la referencia a la API. Permite ejecutar peticiones HTTP contra el host real de la API.

Compila el helper heredado que crea y elimina únicamente la base aislada de cada prueba, sin añadir otro proyecto a la solución.

Integración funcional. Verifica persistencia y consulta mediante ISender, ejecución observable del pipeline sin payloads y cancelación sin efectos persistidos. En 4.4 adapta la lectura a Result.Valor y registra OrdenesConResultado.

Archivo: `tests/AceriaApp.Mediator.Tests/MediatorFuncionalesTests.cs`. **Operación:** Crear.

```
using AceriaApp.Hexagonal.Aplicacion.Patrones;
using AceriaApp.Hexagonal.Infraestructura.Datos;
using AceriaApp.Hexagonal.Infraestructura.Patrones;
using MediatR;
using AceriaApp.PruebasSqlServer;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;
using Xunit;
namespace AceriaApp.Mediator.Tests;

public sealed class MediatorFuncionalesTests
{
    [Fact]
    public async Task Command_y_query_ejecutan_el_caso_de_uso_y_devuelven_datos_persistidos()
    {
        await using var fixture = await Entorno.CrearAsync();
        using var scope = fixture.Services.CreateScope();
        var sender = scope.ServiceProvider.GetRequiredService<ISender>();
        await sender.Send(new CrearOrdenCommand("Axia", "Acero", 20));
        await using var db = fixture.Contexto();
        var id = (await db.Ordenes.SingleAsync()).Id;
        var recuperada = await sender.Send(new ObtenerOrdenQuery(id));
        Assert.Equal("Axia", recuperada!.Cliente);
        Assert.Equal(20, recuperada.Cantidad);
    }

    [Fact]
    public async Task Pipeline_registra_inicio_y_fin_sin_datos_de_cliente()
    {
        await using var fixture = await Entorno.CrearAsync();
        using var scope = fixture.Services.CreateScope();
        await scope.ServiceProvider.GetRequiredService<ISender>()
            .Send(new CrearOrdenCommand("Cliente confidencial", "Acero", 20));
        Assert.Contains(fixture.Logs.Mensajes, m => m.Contains("Inicio CrearOrdenCommand"));
        Assert.Contains(fixture.Logs.Mensajes, m => m.Contains("Fin CrearOrdenCommand"));
        Assert.DoesNotContain(fixture.Logs.Mensajes, m => m.Contains("Cliente confidencial"));
    }

    [Fact]
    public async Task Cancelacion_del_command_no_persiste_la_orden()
    {
        await using var fixture = await Entorno.CrearAsync();
```

```
        using var scope = fixture.Services.CreateScope();
        using var cancelacion = new CancellationTokenSource();
        cancelacion.Cancel();
        await Assert.ThrowsAnyAsync<OperationCanceledException>(() => scope.ServiceProvider
            .GetRequiredService<ISender>().Send(new CrearOrdenCommand("Axia", "Acero", 20),
cancelacion.Token));
        await using var db = fixture.Contexto();
        Assert.Empty(await db.Ordenes.ToListAsync());
    }

    private sealed class Entorno(BaseSqlServer store, ServiceProvider services, RegistroLogs logs) :
IAsyncDisposable
    {
        public ServiceProvider Services => services;
        public RegistroLogs Logs => logs;
        public HexagonalDbContext Contexto() => new(new
DbContextOptionsBuilder<HexagonalDbContext>().UseSqlServer(store.ConnectionString).Options);
        public static async Task<Entorno> CrearAsync()
        {
            var store = BaseSqlServer.Crear();
            var logs = new RegistroLogs();
            var services = new ServiceCollection();
            services.AddLogging(builder => builder.AddProvider(logs));
            services.AddDbContext<HexagonalDbContext>(o => o.UseSqlServer(store.ConnectionString));
            services.AddScoped<IOrdenTrabajoRepositorio, OrdenTrabajoRepositorio>();
            services.AddScoped<IUnitOfWork, EfUnitOfWork>();
            services.AddScoped<OrdenesTransaccionales>();
            services.AddMediatR(cfg => {
                cfg.RegisterServicesFromAssemblyContaining<CrearOrdenCommand>();
                cfg.Lifetime = ServiceLifetime.Scoped;
                cfg.AddOpenBehavior(typeof(LoggingBehavior<,>));
            });
            var fixture = new Entorno(store, services.BuildServiceProvider(new ServiceProviderOptions
{ ValidateScopes = true }), logs);
            try
            {
                await using var db = fixture.Contexto();
                Assert.Equal("Microsoft.EntityFrameworkCore.SqlServer", db.Database.ProviderName);
                await db.Database.EnsureCreatedAsync();
            }
            catch { await fixture.DisposeAsync(); throw; }
            return fixture;
        }
        public async ValueTask DisposeAsync() { await services.DisposeAsync(); await store.DisposeAsync(); }
    }

    private sealed class RegistroLogs : ILoggerProvider
    {
        public List<string> Mensajes { get; } = new();
        public ILogger CreateLogger(string name) => new Captura(Mensajes);
        public void Dispose() { }
        private sealed class Captura(List<string> mensajes) : ILogger
        {
            public IDisposable? BeginScope<TState>(TState state) where TState : notnull => null;
            public bool IsEnabled(LogLevel level) => true;
            public void Log<TState>(LogLevel level, EventId id, TState state, Exception? error, Func<TState,
Exception?, string> formatter) =>
                mensajes.Add(formatter(state, error));
        }
    }
}
```

El mediador ejecuta el caso de uso y una lectura independiente verifica la orden persistida. El pipeline registra inicio y fin sin incluir el nombre del cliente. Una cancelación previa impide persistir una orden.

En `var store = BaseSqlServer.Crear();` se asigna una base SQL Server aislada al entorno de cada prueba. Los ámbitos de DI reciben contextos SQL Server sobre la misma base de esa prueba. Comprueba el proveedor antes de crear las tablas.

En `catch { await fixture.DisposeAsync(); throw; }` se si falla la inicialización, libera también la base temporal. Libera los servicios antes de eliminar la base que les pertenece.

Verificación concreta. La solución completa compila, incluidos el código de integración y sus pruebas. Las siguientes ejecuciones de tests comprobarán el comportamiento de los patrones.

Diagnóstico si falla.

- Tener la API en ejecución: el archivo `.dll` está bloqueado y la compilación falla.
- No restaurar paquetes: `dotnet build` restaura por defecto, pero si hay un error de red, hay que ejecutar `dotnet restore` primero.
- Usar una versión distinta del SDK: `dotnet --version` debe devolver `8.x.x`.

PASO 4.3.18 — Ejecutar todos los tests del punto 4.3

Ejecutar los tests del punto 4.3 verifica que las clases del patrón Mediator funcionan correctamente.

1. Ejecuta `dotnet test tests/AceriaApp.Mediator.Tests/AceriaApp.Mediator.Tests.csproj -c Release` desde la raíz de esta etapa.
2. Comprueba un total de 13 pruebas: 10 de catálogo y 3 funcionales, con cero fallos y cero omisiones.
3. Si falla una prueba, abre su mensaje de aserción y el archivo indicado; corrige el comportamiento antes de continuar.
4. Repite el mismo comando tras la corrección y conserva el resultado de la ejecución válida.

Verificación concreta. El proyecto `AceriaApp.Mediator.Tests` termina con `Failed: 0, Passed: 13, Skipped: 0, Total: 13`.

Diagnóstico si falla.

- Ejecutar otro proyecto: comprueba la ruta y el nombre de la asamblea de pruebas antes de interpretar los resultados.
- No compilar antes: `dotnet test` compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

PASO 4.3.19 — Ejecutar todos los tests de la solución

Ejecutar todos los tests de la solución verifica que los tests de los puntos anteriores y del punto 4.3 pasan conjuntamente.

1. Ejecuta `dotnet build AceriaApp.sln -c Release` para producir todas las asambleas de esta etapa.
2. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build --no-restore` para ejecutar todas las pruebas acumuladas.
3. Suma los resultados de los 8 proyectos de tests: deben ser 90 pruebas aprobadas, sin fallos ni omisiones.
4. Comprueba que las pruebas heredadas de M03 y los puntos anteriores de M04 siguen presentes; no borres tests para obtener un resultado verde.

Verificación concreta. La suma de las líneas de resultados es `Failed: 0`, `Passed: 90`, `Skipped: 0` y `Total: 90`. Dotnet muestra un resumen por proyecto, no necesariamente una única línea global.

Diagnóstico si falla.

- No haber compilado antes: `--no-build` falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
- Eliminar pruebas anteriores o cambiarlas sin una instrucción explícita: se pierde la cobertura de los contratos heredados.

PASO 4.3.20 — Crear el README del punto 4.3

El README del punto 4.3 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 4.2.

1. En la carpeta del punto `4.3-mediator`, hacer clic derecho y seleccionar `Add` → `New Item` → `Text File`.
2. Nombrar el archivo `README.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que el README del punto 4.2 no se ha modificado.

Archivo: `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 4.3 — Mediator (MediatR)

## Objetivo
Comprender el patrón Mediator y añadir al proyecto las clases de validación de mensajes MediatR.

La práctica conecta además el patrón con los puertos, adaptadores y pruebas de integración que aparecen en
este manual.

## Estado del proyecto
- Solución `AceriaApp.sln` con 12 proyectos.
- Documento `docs/patron-mediator.md` con las definiciones y las reglas.
- Clases `Mensaje`, `ValidadorMensajes` y `CatalogoMensajes` en `src/AceriaApp.Api/Mediator/`.
- 10 tests nuevos de catálogo y 3 funcionales en `tests/AceriaApp.Mediator.Tests`.
- Los tests acumulados suman 90 pruebas: 10 de catálogo nuevas y 3 funcionales nuevas en este punto.

## Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio 2026.
2. Verificar que SQL Server Express está arrancado.
3. Configurar las conexiones SQL Server `AceriaApp` y `AceriaHexagonal` en el entorno local.
4. Ejecutar `dotnet run --project src/AceriaApp.Api/AceriaApp.Api.csproj --urls http://localhost:5000` desde
la raíz de esta etapa.
5. Navegar a `http://localhost:5000/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

## Qué se ha añadido respecto al punto anterior
- Documento `docs/patron-mediator.md`.
- Carpeta `Mediator` en el proyecto API con tres clases nuevas.
- Proyecto `tests/AceriaApp.Mediator.Tests` con 10 tests de catálogo y 3 funcionales nuevos.
- Se conserva intacta la carpeta del punto 4.2.

## Documentación
- [ADR-010](docs/ADR-010-capas-aplicacion.md) a [ADR-015](docs/ADR-015-patron-unit-of-work.md) (heredados).
- [Patrón Mediator](docs/patron-mediator.md).

## Verificación ejecutable

Los catálogos describen ejemplos de diseño. El comportamiento de las clases reales se comprueba además con
los tests funcionales del punto. Ejecuta `dotnet test AceriaApp.sln -c Release` y exige cero fallos y cero
omisiones.
```

Verificación concreta. El archivo `README.md` existe y contiene las 6 secciones mostradas; abre su vista previa y comprueba todos los apartados.

Diagnóstico si falla.

- Modificar el README del punto 4.2: rompe la conservación de la etapa anterior.
- Omitir la sección `Qué se ha añadido respecto al punto anterior`: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 4.2: rompe la separación.

PASO 4.3.21 — Crear el ADR-016 del punto 4.3

El ADR-016 documenta la decisión de adoptar el patrón Mediator con MediatR. Es un artefacto nuevo del punto 4.3 que no modifica los ADR existentes.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `docs`.

2. Seleccionar `Add` → `New Item` → `Text File` y nombrar el archivo `ADR-016-patron-mediator.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que los ADR-010 a ADR-015 no se han modificado.

Archivo: `docs/ADR-016-patron-mediator.md`. **Operación:** Crear.

```
# ADR-016: Adopción del patrón Mediator con MediatR

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha adoptado los patrones Repository y Unit of Work. Es necesario desacoplar los
adaptadores de entrada de los casos de uso.

## Decisión
Se adopta MediatR 12.4.1 en la capa de aplicación. `CrearOrdenCommand` y `ObtenerOrdenQuery` son records que
implementan `IRequest<T>` y sus manejadores implementan `IRequestHandler<TRequest,TResponse>`. Los endpoints
envían ambos mediante `ISender.Send`; los manejadores delegan en `OrdenesTransaccionales`. `AddMediatR`
descubre los manejadores del ensamblado de aplicación y registra `LoggingBehavior`. Este pipeline observa el
inicio y el fin y respeta la cancelación. Los contratos `INotification` e `INotificationHandler` se estudian
como variante; este flujo no publica notificaciones ni incluye un pipeline de FluentValidation.

## Alternativas consideradas
- Inyección directa de los servicios de aplicación en los adaptadores: descartado por acoplamiento.
- Uso de un bus de mensajes externo: descartado por simplicidad del proyecto.

## Consecuencias
- Positivas: bajo acoplamiento, adaptadores simples, un único punto de entrada.
- Negativas: la indirección del mediador puede dificultar el seguimiento.
- Mitigación: la validación automática y el registro claro de los manejadores.
```

Verificación concreta. El archivo `docs/ADR-016-patron-mediator.md` existe y contiene las 5 secciones mostradas; abre su vista previa y comprueba todos los apartados.

Diagnóstico si falla.

- Modificar los ADR anteriores: rompe la conservación de la etapa anterior.
- Omitir la sección `Alternativas consideradas`: el ADR pierde valor.
- Nombrar el archivo sin el prefijo `ADR-016`: rompe la convención.

PASO 4.3.22 — Verificar que los documentos heredados y la etapa 4.2 permanecen intactos

Las decisiones previas se conservan como registros históricos. Los nuevos ADR explican la evolución y las modificaciones de integración se aplican solo a la copia de la etapa actual.

1. Vuelve a la raíz del repositorio; las rutas siguientes se interpretan desde allí.
2. En CMD ejecuta `for %f in (M04_PATRONES_ARQUITECTURA\PROYECTO\4.2-unit-of-work\docs\*.md) do fc /L "%f" "M04_PATRONES_ARQUITECTURA\PROYECTO\4.3-mediator\docs\%~nxf"`. En PowerShell puedes comparar `Get-FileHash` para cada par de documentos.
3. Comprueba 12 comparaciones y exige igualdad en todas; se conserva exactamente el contenido de los documentos heredados.
4. Si una comparación falla, restaura el documento de la etapa actual desde `4.2-unit-of-work`. Verifica también que la carpeta del punto anterior no tiene modificaciones.

Verificación concreta. Los 12 documentos heredados coinciden byte a byte y el punto anterior permanece intacto.

Diagnóstico si falla.

- Usar `comp` en lugar de `fc`: `comp` compara binarios y no muestra diferencias de texto.

- Omitir `/L` cuando se desea una comparación de texto: indica el modo explícitamente y revisa cada resultado.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

PASO 4.3.23 — Verificar la estructura del punto 4.3

La verificación estructural confirma que el punto 4.3 añade archivos nuevos conservando los documentos heredados.

1. Escribir `tree /F M04_PATRONES_ARQUITECTURA\PROYECTO\4.3-mediator` y pulsar Enter.
2. Verificar que la salida muestra `AceriaApp.sln`, `README.md`, `src\AceriaApp.Api\Mediator` con las tres clases nuevas, `tests\AceriaApp.Mediator.Tests` con los dos archivos de catálogo y el archivo de tests funcionales y `docs` con el documento del patrón y el ADR-016. Comprueba además todos los archivos de integración de las carpetas Patrones y los cambios de composición indicados en este punto.
3. Verificar que las carpetas existentes del punto 4.2 permanecen intactas.
4. Verificar que no existe ninguna carpeta `labs` ni ningún archivo `metaprompt.md` ni `verificacion.md`.

Verificación concreta. la estructura contiene los archivos nuevos y conserva los del punto 4.2.

Diagnóstico si falla.

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 4.2: viola la regla de no modificar.
- Omitir la carpeta `Mediator`: los tests no encuentran las clases.

PASO 4.3.24 — Hacer el commit del punto 4.3 y verificar el estado final

El commit permite comparar esta etapa con la anterior. Si has realizado un commit por punto desde 1.1, el recorrido suma 16 commits de puntos; otros commits legítimos pueden aumentar el historial.

1. Desde la raíz del repositorio ejecuta `git status` y revisa que solo aparezcan los archivos de esta etapa y los entregables que has trabajado.
2. Ejecuta `git add .` después de verificar que `bin/`, `obj/`, `TestResults/` y credenciales están excluidos.
3. Ejecuta `git commit -m "Punto 4.3: Mediator"` con `user.name` y `user.email` configurados.
4. Ejecuta `git log --oneline` y `git status`; comprueba que se registró el commit de este punto y que no quedan cambios pendientes de esta actividad.

Verificación concreta. El historial incluye el commit de 4.3 y `git status` no muestra cambios pendientes de esta actividad; no se exige un número rígido de commits a un repositorio con más trabajo.

Diagnóstico si falla.

- No tener configurado `user.name` y `user.email`: `git commit` falla.
- Añadir la carpeta `bin/` y `obj/` al commit: el `.gitignore` lo evita.
- Hacer commit antes de `git add .`: no hay cambios en el índice.

Reto de ampliación (optativo)

Trabaja en una copia independiente de la etapa obligatoria, por ejemplo `4.3-mediator-reto`. Mantén intacta la carpeta obligatoria y todos sus documentos. Los tres fragmentos siguientes añaden un contrato, un cálculo y una prueba al proyecto del punto.

Reto: añadir un método `Resumen` a `ValidadorMensajes` que devuelva el número total de mensajes, el número de comandos, el número de consultas y el número de notificaciones, y un test que verifique el cálculo con el catálogo por defecto.

Paso 1: En `ValidadorMensajes.cs`, añadir el siguiente `record` al final del archivo:

Archivo: `src/AceriaApp.Api/Mediator/ValidadorMensajes.cs`. **Operación:** Modificar.

Punto de inserción: después de este texto exacto, que debe aparecer una sola vez:

```
public record ResultadoMensajes(bool EsValido, IReadOnlyList<string> Errores);
```

```
public record ResumenMensajes(
    int Total,
    int Comandos,
    int Consultas,
    int Notificaciones);
```

Paso 2: Explicar el propósito del fragmento y las decisiones que contiene:

La declaración `public record ResumenMensajes(` declara un contrato inmutable. Número total de mensajes. Número de comandos. Número de consultas. Número de notificaciones.

Paso 3: En `ValidadorMensajes.cs`, añadir el siguiente método estático dentro de la clase:

Archivo: `src/AceriaApp.Api/Mediator/ValidadorMensajes.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
}

public record ResultadoMensajes(bool EsValido, IReadOnlyList<string> Errores);
```

Indenta el fragmento con cuatro espacios dentro de la clase; conserva el cierre de clase que aparece en el texto de inserción.

```
public static ResumenMensajes Resumen(IEnumerable<Mensaje> mensajes)
{
    var lista = mensajes.ToList();
    var comandos = lista.Count(m => m.Tipo == "Comando");
    var consultas = lista.Count(m => m.Tipo == "Consulta");
    var notificaciones = lista.Count(m => m.Tipo == "Notificacion");
    return new ResumenMensajes(lista.Count, comandos, consultas, notificaciones);
}
```

Paso 4: Explicar el propósito del fragmento y las decisiones que contiene:

La declaración `public static ResumenMensajes Resumen(IEnumerable<Mensaje> mensajes)` declara el método. Materializa la lista. Cuenta los comandos. Cuenta las consultas. Cuenta las notificaciones. Devuelve el resumen.

Paso 5: Guardar el archivo con `Ctrl+S`.

Paso 6: En `ValidadorMensajesTests.cs`, añadir el siguiente test:

Archivo: `tests/AceriaApp.Mediator.Tests/ValidadorMensajesTests.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
}
```

Indenta el fragmento con cuatro espacios dentro de la clase; conserva el cierre de clase que aparece en el texto de inserción.

```
[Fact]
public void Resumen_ConCatalogoPorDefecto_CuentaPorTipo()
{
```

```
    var resumen = ValidadorMensajes.Resumen(CatalogoMensajes.MensajesPorDefecto());

    resumen.Total.Should().Be(5);
    resumen.Comandos.Should().Be(2);
    resumen.Consultas.Should().Be(2);
    resumen.Notificaciones.Should().Be(1);
}
```

Paso 7: Interpretar el test y explicar qué comportamiento acredita:

Marca el método como test de xUnit. Declara el test. Invoca el método. Verifica el total. Verifica los comandos. Verifica las consultas. Verifica las notificaciones.

Paso 8: Ejecuta `dotnet test tests/AceriaApp.Mediator.Tests/AceriaApp.Mediator.Tests.csproj -c Release` en la copia optativa. Deben pasar 14 tests del proyecto, incluido el nuevo resumen. Ejecuta después `dotnet test AceriaApp.sln -c Release` y exige 91 pruebas aprobadas.

Paso 9: Comprueba que solo cambian `ValidadorMensajes.cs` y `ValidadorMensajesTests.cs` en la copia optativa; la etapa obligatoria `4.3-mediator` y el punto anterior deben permanecer idénticos.

Paso 10: Hacer commit con `git commit -m "Reto 4.3: método Resumen en ValidadorMensajes"`.

Verificación: `Passed: 14` en el proyecto del punto y 91 pruebas aprobadas en la solución de la copia optativa, sin fallos ni omisiones.

Resultado verificable

Al finalizar este punto, el alumno debe tener:

- La carpeta `M04_PATRONES_ARQUITECTURA\PROYECTO/4.3-mediator/` con la solución `AceriaApp.sln` que contiene 12 proyectos.
- El documento `docs/patron-mediator.md` con las definiciones y las reglas del patrón.
- El documento `docs/ADR-016-patron-mediator.md` con la decisión sobre el patrón Mediator.
- Las clases `Mensaje`, `ValidadorMensajes` y `CatalogoMensajes` en `src/AceriaApp.Api/Mediator/`.
- El proyecto `tests/AceriaApp.Mediator.Tests` añade 10 tests de catálogo y 3 funcionales; el reto añade uno más.
- `dotnet test AceriaApp.sln -c Release` ejecuta 90 pruebas en la etapa obligatoria y 91 en una copia con su reto; ninguna falla ni se omite.
- Los ADR-010 a ADR-015 del punto 4.2 sin modificar.
- Un commit revisado para el punto 4.3, con el historial previo conservado.

Continuidad del aprendizaje

El punto 4.3 ha definido el patrón Mediator como mecanismo para desacoplar los adaptadores de entrada de los casos de uso. El proyecto AceriaApp dispone ahora de un validador automático de mensajes MediatR que comprueba los tipos permitidos, la presencia de manejador y las reglas específicas de cada tipo, y 10 tests nuevos de catálogo más 3 funcionales. Esta base permite avanzar hacia el punto 4.4, donde se estudiará el Result Pattern como alternativa al uso de excepciones para representar errores en los casos de uso.

Punto 4.4 — Result Pattern

Expresar fallos conocidos sin convertirlos en excepciones indiscriminadas

Una aplicación necesita explicar cuándo no encuentra un recurso, cuándo una petición es inválida y cuándo existe un conflicto de negocio. El patrón Result puede representar esas condiciones de manera explícita, evitando utilizar excepciones para cada rama normal del flujo. En este punto distinguiremos el resultado del caso de uso de la traducción que hace el adaptador HTTP hacia códigos y documentos de error.

En AceriaApp aplicaremos la separación al recorrido real de las órdenes. El catálogo de resultados es un ejercicio para razonar sobre sus estados; la integración verdadera se observa cuando un endpoint devuelve la respuesta prevista y no oculta una avería de SQL como si fuera una validación. Revisaremos casos satisfactorios y negativos, manteniendo el contrato heredado para no introducir regresiones al añadir el patrón.

Carpeta de trabajo: `M04_PATRONES_ARQUITECTURA/PROYECTO/4.4-result-pattern`.

Objetivo: completar los veinticuatro pasos, conservar los documentos heredados y comprobar el patrón con código y tests ejecutables. La etapa suma 105 pruebas en 13 proyectos.

PASO 4.4.1 — Crear la carpeta del punto 4.4 a partir del punto 4.3

La copia preserva una etapa autónoma y reproducible. 4.3-mediator aporta 90 tests aprobados y 12 proyectos antes de añadir este punto.

1. Abre la terminal integrada de Visual Studio y localiza `M04_PATRONES_ARQUITECTURA/PROYECTO/4.3-mediator`; esa es la etapa completa que vas a continuar.
2. Crea `M04_PATRONES_ARQUITECTURA/PROYECTO/4.4-result-pattern` y copia en ella el contenido de `4.3-mediator`; en Windows puedes usar `xcopy` con `/E /I /H /Y`.
3. Excluye `bin`, `obj` y `TestResults`; abre únicamente la copia de `4.4-result-pattern` para que el punto anterior permanezca intacto.
4. Ejecuta `dir` dentro de `4.4-result-pattern` y verifica `AceriaApp.sln`, `README.md`, `src`, `tests`, `docs` y `global.json`.

Verificación concreta. La copia contiene la solución y los 12 proyectos de la etapa anterior; no contiene todavía archivos nuevos de 4.4.

Diagnóstico si falla.

- Usar `copy` en lugar de `xcopy`: `copy` no copia carpetas de forma recursiva.
- Olvidar el flag `/E`: no se copian las subcarpetas vacías.
- Olvidar el flag `/I`: `xcopy` pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag `/Y`: `xcopy` pide confirmación para sobrescribir.

PASO 4.4.2 — Abrir la solución copiada en Visual Studio 2026

Abrir la solución copiada permite trabajar en el punto 4.4 sobre la base del punto 4.3. La compilación verifica que la copia es correcta.

1. Entra en `4.4-result-pattern` y abre `AceriaApp.sln` con Visual Studio; desde Windows puedes ejecutar `start AceriaApp.sln`.
2. Ejecuta `dotnet sln AceriaApp.sln list` y comprueba los 12 proyectos heredados; todos deben existir bajo esta etapa.

3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`; conserva la salida como comprobación inicial.
4. Si la API está en ejecución, detenla antes de compilar. Continúa cuando la compilación termine sin errores y las rutas no apunten a otra etapa.

Verificación concreta. `dotnet sln AceriaApp.sln list` muestra 12 proyectos y `dotnet build AceriaApp.sln -c Release` termina con `Build succeeded.`.

Diagnóstico si falla.

- Abrir la solución del punto 4.3 en lugar de la del punto 4.4: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener archivos DLL bloqueados por una API en ejecución: detén la aplicación antes de recompilar.

PASO 4.4.3 — Verificar que la carpeta `docs` contiene los documentos del punto 4.3

La etapa conserva los 14 documentos anteriores y añade únicamente su ficha de patrón y su ADR. Las modificaciones de código se escriben en la copia actual.

1. Expande `docs` dentro de la copia de `4.4-result-pattern`.
2. Comprueba los 14 documentos heredados: los ADR-010 a ADR-016 y las fichas de los puntos anteriores.
3. Compara sus nombres y contenido con `docs` de `4.3-mediator`. No modifiques ni elimines los documentos anteriores.
4. Confirma que todavía no existen `docs/patron-result.md` ni `docs/ADR-017-patron-result.md`; se crearán en este punto.

Verificación concreta. `docs` contiene 14 archivos heredados idénticos; los dos documentos de este punto todavía no existen.

Diagnóstico si falla.

- Modificar alguno de los ADR o documentos existentes: rompe la conservación de la etapa anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 4.3.
- Crear la carpeta `docs` si ya existe: `mkdir` falla.

PASO 4.4.4 — Crear el documento del patrón Result en `docs`

El documento del patrón Result es el artefacto de `docs` específico del punto 4.4. No modifica los 14 documentos existentes del punto 4.3.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `docs`.
2. Seleccionar `Add` → `New Item` → `Text File` y nombrar el archivo `patron-result.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que el archivo aparece en la carpeta `docs` sin subrayados rojos.

Archivo: `docs/patron-result.md`. **Operación:** Crear.

```
# Patrón Result — Punto 4.4

## Definición

Patrón que representa el resultado de una operación como un valor explícito en lugar de como una excepción.

## Errores esperados y excepcionales
```

```
- Errores esperados: Result.
- Errores excepcionales: excepción.

## Estructura

- `IsSuccess`: indicador del tipo básico que se estudia y prueba en `AceriaApp.Api.ResultPattern`. En el
contrato ejecutable de aplicación se denomina `EsExito`.
- `Value`: valor del tipo básico. El contrato ejecutable utiliza `Valor` y rechaza leerlo en un fallo.
- `Error`: información del error; el contrato ejecutable incorpora `Codigo`, `Descripcion` y `Tipo`, y
rechaza leerlo en un éxito.

## Variantes

- `Result<T>`: valor de éxito y mensaje de error.
- `Result<T, TError>`: valor de éxito y tipo de error específico.
- `Result`: sin valor de éxito.
- `Result<T>` con lista de errores.

## Result Pattern en los casos de uso

Los casos de uso devuelven `Result<T>` en lugar de lanzar excepciones para errores esperados.

## Result Pattern en los adaptadores de entrada

Los adaptadores traducen el resultado en respuestas HTTP.

## Composición de resultados

Las variantes más completas pueden ofrecer `Map`, `Bind` y `Match`. El contrato construido en este punto
utiliza comprobaciones explícitas de `EsExito`; no expone esos tres métodos.

## Result Pattern y validación

La validación de esta ruta se realiza en `OrdenesConResultado`, que devuelve un error de categoría
`Validacion` antes de preparar escrituras. Integrar FluentValidation en un pipeline es otra variante; el
comportamiento registrado aquí es `LoggingBehavior`.

## Cuándo usar el patrón

- Errores esperados: sí.
- Operaciones frecuentes: sí.
- Firma explícita: sí.
- Errores excepcionales: no.
- Operación sin fallo esperado: no.

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Caso de uso ejecutable | OrdenesConResultado devuelve `Result<OrdenHexagonal>` |
| Errores esperados | `Result<OrdenHexagonal>.Fallo` con Error tipado |
| Éxito | `Result<OrdenHexagonal>.Exito` |
| Adaptadores entrada | Traducen el resultado a HTTP |
| Validación ejecutable | OrdenesConResultado; 400 antes de persistir |
| Validación del patrón | Test de coherencia |

## Correspondencia con el código ejecutable

El catálogo de este punto describe reglas de diseño y contiene ejemplos de contratos. Sus nombres no implican
servicios registrados. La implementación utiliza los proyectos `AceriaApp.Hexagonal.Aplicacion`,
`AceriaApp.Hexagonal.Dominio` y `AceriaApp.Hexagonal.Infraestructura`, compuestos desde `AceriaApp.Api`.
Comprueba los archivos y los tests funcionales que incorpora el paso de compilación de este punto.
```

Verificación concreta. El archivo `docs/patron-result.md` existe y contiene las 11 secciones mostradas; abre su vista previa y comprueba todos los apartados.

Diagnóstico si falla.

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: rompe la conservación de la etapa anterior.
- Guardar el archivo en la raíz del punto en lugar de `docs`: rompe la estructura del repositorio.

PASO 4.4.5 — Crear la carpeta `ResultPattern` en el proyecto API

La carpeta `ResultPattern` albergará las clases de validación del patrón Result. Se crea como carpeta nueva sin tocar las carpetas existentes.

1. En el panel Solution Explorer, hacer clic derecho sobre el proyecto `AceriaApp.Api`.
2. Seleccionar `Add` → `New Folder` y nombrar la carpeta `ResultPattern`.
3. Verificar que la carpeta `ResultPattern` aparece dentro de `AceriaApp.Api` sin afectar a las carpetas existentes.
4. Verificar que no se ha modificado ningún archivo existente del punto 4.3.

Verificación concreta. la carpeta `ResultPattern` existe dentro de `AceriaApp.Api` y las carpetas existentes permanecen intactas.

Diagnóstico si falla.

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: rompe la conservación de la etapa anterior.
- Crear la carpeta con otro nombre: rompe la convención del curso.

PASO 4.4.6 — Crear la clase `Result`

`Result` expresa un resultado explícito que el consumidor puede inspeccionar sin recurrir a excepciones de control de flujo. Observa cómo se distingue éxito de error y qué información queda disponible para la traducción HTTP.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `ResultPattern`.
2. Seleccionar `Add` → `Class` y nombrar la clase `Result.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/ResultPattern/Result.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.ResultPattern;

public class Result<T>
{
    public bool IsSuccess { get; }
    public T? Value { get; }
    public string? Error { get; }

    private Result(bool isSuccess, T? value, string? error)
    {
        IsSuccess = isSuccess;
        Value = value;
        Error = error;
    }

    public static Result<T> Success(T value) => new(true, value, null);

    public static Result<T> Failure(string error) => new(false, default, error);
}
```

Verificación concreta. la clase `Result<T>` compila sin errores y expone tres propiedades y dos métodos estáticos.

Diagnóstico si falla.

- Olvidar el `namespace`: la clase no se resuelve desde otros archivos.

- Confundir este tipo básico con el contrato ejecutable de aplicación: comprueba el namespace y utiliza el Result con fábricas y acceso de ramas protegidos en el flujo real.
- Usar `set` en lugar de solo getter: rompe la inmutabilidad.

PASO 4.4.7 — Crear la clase `ResultadoOperacion`

`ResultadoOperacion` ofrece una representación docente de estados y errores. La API no debería devolver un 200 ante un fallo funcional solo porque el objeto se creó correctamente; su adaptador decide el código según el contrato.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `ResultPattern`.
2. Seleccionar `Add` → `Class` y nombrar la clase `ResultadoOperacion.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/ResultPattern/ResultadoOperacion.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.ResultPattern;

public class ResultadoOperacion
{
    public string Nombre { get; init; } = string.Empty;
    public bool UsaResult { get; init; }
    public bool UsaExcepcionesParaErroresEsperados { get; init; }
    public int NumeroDeErroresEsperados { get; init; }
}
```

Verificación concreta. la clase `ResultadoOperacion` compila sin errores y expone cuatro propiedades.

Diagnóstico si falla.

- Olvidar el `namespace`: la clase no se resuelve desde otros archivos.
- No inicializar `Nombre` con `string.Empty`: el compilador emite advertencia de nulabilidad.
- Usar `set` en lugar de `init`: permite modificar la instancia después de crearla.

PASO 4.4.8 — Crear la clase `ValidadorResultados`

El validador permite contrastar combinaciones de estado y significado definidas en el catálogo. Debe diferenciarse de un middleware que captura excepciones inesperadas: los dos mecanismos resuelven problemas distintos.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `ResultPattern`.
2. Seleccionar `Add` → `Class` y nombrar la clase `ValidadorResultados.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/ResultPattern/ValidadorResultados.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.ResultPattern;

public class ValidadorResultados
{
    public ResultadoResultados Validar(IEnumerable<ResultadoOperacion> operaciones)
    {
        var errores = new List<string>();
        var lista = operaciones.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay operaciones declaradas.");
        }
```

```
        foreach (var operacion in lista)
        {
            if (string.IsNullOrWhiteSpace(operacion.Nombre))
            {
                errores.Add("Una operación no declara el nombre.");
            }

            if (!operacion.UsaResult && operacion.NumeroDeErroresEsperados > 0)
            {
                errores.Add($"La operación '{operacion.Nombre}' tiene errores esperados pero no usa el patrón
Result.");
            }

            if (operacion.UsaExcepcionesParaErroresEsperados)
            {
                errores.Add($"La operación '{operacion.Nombre}' usa excepciones para errores esperados.");
            }
        }

        return new ResultadoResultados(errores.Count == 0, errores);
    }
}

public record ResultadoResultados(bool EsValido, IReadOnlyList<string> Errores);
```

Verificación concreta. la clase `ValidadorResultados` compila sin errores y expone un método y un `record`.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<string>` e `IReadOnlyList<string>` no se resuelven.
- Olvidar el `using System.Linq;`: `ToList` no se resuelve.
- No comprobar el caso de lista vacía: el validador devuelve válido sin operaciones.

PASO 4.4.9 — Añadir los `using` necesarios en `ValidadorResultados`

Los tipos `List<string>` e `IReadOnlyList<string>` pertenecen a `System.Collections.Generic`. El método `ToList` pertenece a `System.Linq`. En proyectos con `ImplicitUsings` desactivado, hay que añadir los usings explícitos.

1. En el panel Solution Explorer, hacer doble clic en `ValidadorResultados.cs`.
2. Inserta las directivas indicadas antes del namespace del archivo; la indicación del archivo identifica la línea exacta.
3. Verificar que no hay subrayados rojos en `List<string>` ni `ToList`.
4. Pulsar `Ctrl+S` para guardar el archivo.

Archivo: `src/AceriaApp.Api/ResultPattern/ValidadorResultados.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
namespace AceriaApp.Api.ResultPattern;
```

```
using System.Collections.Generic;
using System.Linq;
```

Verificación concreta. `ValidadorResultados.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir solo `System.Collections.Generic` y olvidar `System.Linq`: `ToList` no se resuelve.
- Mezclar directivas using con declaraciones de tipos: coloca las directivas antes del namespace para mantener el orden del archivo.
- Olvidar los usings y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 4.4.10 — Crear la clase `CatalogoResultados`

La clase `CatalogoResultados` contiene las operaciones por defecto del proyecto. Se usan en los tests y en la validación automática.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `ResultPattern`.
2. Seleccionar `Add` → `Class` y nombrar la clase `CatalogoResultados.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/ResultPattern/CatalogoResultados.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.ResultPattern;

public class CatalogoResultados
{
    public static List<ResultadoOperacion> OperacionesPorDefecto()
    {
        return new List<ResultadoOperacion>
        {
            new() { Nombre = "CrearOrden", UsaResult = true, UsaExcepcionesParaErroresEsperados = false,
NumeroDeErroresEsperados = 3 },
            new() { Nombre = "CancelarOrden", UsaResult = true, UsaExcepcionesParaErroresEsperados = false,
NumeroDeErroresEsperados = 2 },
            new() { Nombre = "ObtenerOrden", UsaResult = true, UsaExcepcionesParaErroresEsperados = false,
NumeroDeErroresEsperados = 1 },
            new() { Nombre = "ListarOrdenes", UsaResult = true, UsaExcepcionesParaErroresEsperados = false,
NumeroDeErroresEsperados = 0 }
        };
    }
}
```

Verificación concreta. la clase `CatalogoResultados` compila sin errores y expone un método estático.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<T>` no se resuelve.
- No inicializar las propiedades de los objetos: el compilador emite advertencias.
- Reutilizar una lista estática mutable: cada llamada del catálogo debe construir una lista independiente.

PASO 4.4.11 — Añadir el `using` necesario en `CatalogoResultados`

El tipo `List<T>` pertenece a `System.Collections.Generic`. En proyectos con `ImplicitUsings` desactivado, hay que añadir el using explícito.

1. En el panel Solution Explorer, hacer doble clic en `CatalogoResultados.cs`.
2. Inserta las directivas indicadas antes del namespace del archivo; la indicación del archivo identifica la línea exacta.
3. Verificar que no hay subrayados rojos en `List<T>`.
4. Pulsar `Ctrl+S` para guardar el archivo.

Archivo: `src/AceriaApp.Api/ResultPattern/CatalogoResultados.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
namespace AceriaApp.Api.ResultPattern;
```

```
using System.Collections.Generic;
```

Verificación concreta. `CatalogoResultados.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir `using System.Linq;` en lugar de `System.Collections.Generic`: no resuelve `List<T>`.
- Colocar el using después de una clase: las directivas deben preceder las declaraciones de tipos.
- Olvidar el using y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 4.4.12 — Crear el proyecto de tests del punto 4.4

El punto 4.4 añade un proyecto de tests específico para las clases del patrón Result. Se crea como proyecto nuevo y conserva las pruebas previas; las actualizaciones explícitas de integración se muestran en su paso correspondiente.

1. En `tests` crea la carpeta `AceriaApp.ResultPattern.Tests` y, dentro, el archivo `AceriaApp.ResultPattern.Tests.csproj` con el XML completo de este paso. En Visual Studio puedes elegir la plantilla xUnit y ajustar el archivo generado.
2. Elimina el archivo de ejemplo `UnitTest1.cs` si lo creó la plantilla; los archivos de pruebas completos se escriben en sus pasos correspondientes.
3. Ejecuta `dotnet sln AceriaApp.sln add tests/AceriaApp.ResultPattern.Tests/AceriaApp.ResultPattern.Tests.csproj` incluso si la solución ya existía y después `dotnet sln AceriaApp.sln list`.
4. Comprueba que el proyecto aparece bajo `tests` y que `dotnet restore tests/AceriaApp.ResultPattern.Tests/AceriaApp.ResultPattern.Tests.csproj` termina correctamente.

Archivo: `tests/AceriaApp.ResultPattern.Tests/AceriaApp.ResultPattern.Tests.csproj`. **Operación:** Crear.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. La solución muestra 13 proyectos, incluido `AceriaApp.ResultPattern.Tests`; el nuevo csproj fija .NET 8, SDK de tests 17.12.0, xUnit 2.9.2 y runner 2.8.2.

Diagnóstico si falla.

- Crear el proyecto dentro de `src` en lugar de `tests`: rompe la estructura.
- Modificar los proyectos de tests del punto 4.3: rompe la conservación de la etapa anterior.
- Usar `dotnet new mstest`: el curso usa xUnit.

PASO 4.4.13 — Añadir la referencia del proyecto de tests a la API

La referencia permite que el proyecto de tests acceda a las clases de `ResultPattern` del proyecto API.

1. Ejecuta `dotnet add tests/AceriaApp.ResultPattern.Tests/AceriaApp.ResultPattern.Tests.csproj reference src/AceriaApp.Api/AceriaApp.Api.csproj` o deja el archivo exactamente como el bloque XML de este paso.
2. Comprueba que la referencia va de tests a API; la API nunca debe referenciar el proyecto de tests.

3. Abre `tests/AceriaApp.ResultPattern.Tests/AceriaApp.ResultPattern.Tests.csproj` y comprueba el único `ProjectReference` a `../../src/AceriaApp.Api/AceriaApp.Api.csproj`.
4. Ejecuta `dotnet build tests/AceriaApp.ResultPattern.Tests/AceriaApp.ResultPattern.Tests.csproj -c Release` y verifica que el proyecto resuelve la API y sus dependencias.

Archivo: `tests/AceriaApp.ResultPattern.Tests/AceriaApp.ResultPattern.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. el archivo `AceriaApp.ResultPattern.Tests.csproj` contiene un `ProjectReference` al proyecto API.

Diagnóstico si falla.

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo `.csproj`: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

PASO 4.4.14 — Añadir el paquete `FluentAssertions` al proyecto de tests

`FluentAssertions` proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

1. Ejecuta `dotnet add tests/AceriaApp.ResultPattern.Tests/AceriaApp.ResultPattern.Tests.csproj package FluentAssertions --version 6.12.2` o reemplaza el csproj por el XML completo indicado.
2. Ejecuta `dotnet list tests/AceriaApp.ResultPattern.Tests/AceriaApp.ResultPattern.Tests.csproj package` y localiza `FluentAssertions 6.12.2`.
3. Comprueba que el paquete se añade al nuevo proyecto de tests y que se mantienen la referencia a la API y las versiones de xUnit.
4. Guarda el archivo y ejecuta `dotnet build tests/AceriaApp.ResultPattern.Tests/AceriaApp.ResultPattern.Tests.csproj -c Release`; las aserciones `Should()` estarán disponibles cuando escribas las pruebas.

Archivo: `tests/AceriaApp.ResultPattern.Tests/AceriaApp.ResultPattern.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
```

```
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. El archivo de proyecto y `dotnet list package` muestran `FluentAssertions 6.12.2`; la compilación finaliza correctamente.

Diagnóstico si falla.

- Instalar una versión preliminar: el curso usa la versión fijada en el archivo de proyecto.
- Olvidar el paquete: `Should()` no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

PASO 4.4.15 — Crear el archivo `ResultTests.cs`

Los tests verifican que la clase `Result<T>` produce los valores correctos en los estados de éxito y de fallo.

1. En el panel Solution Explorer, expandir `tests` → `AceriaApp.ResultPattern.Tests`.
2. Crear el archivo `ResultTests.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `tests/AceriaApp.ResultPattern.Tests/ResultTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.ResultPattern;
using FluentAssertions;
using Xunit;

namespace AceriaApp.ResultPattern.Tests;

public class ResultTests
{
    [Fact]
    public void Success_ConValor_DevuelveIsSuccessTrue()
    {
        var result = Result<int>.Success(42);

        result.IsSuccess.Should().BeTrue();
        result.Value.Should().Be(42);
        result.Error.Should().BeNull();
    }

    [Fact]
    public void Failure_ConError_DevuelveIsSuccessFalse()
    {
        var result = Result<int>.Failure("Error de prueba");

        result.IsSuccess.Should().BeFalse();
        result.Value.Should().Be(default);
        result.Error.Should().Be("Error de prueba");
    }

    [Fact]
    public void Success_ConTipoComplejo_DevuelveValorCorrecto()
    {
        var result = Result<string>.Success("AceriaApp");

        result.IsSuccess.Should().BeTrue();
        result.Value.Should().Be("AceriaApp");
    }
}
```

Verificación concreta. el archivo `ResultTests.cs` compila y contiene tres métodos de test.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.ResultPattern;`: la clase no se resuelve.
- Usar `Assert.True` en lugar de `Should().BeTrue()`: el curso usa FluentAssertions.
- No cubrir el caso de fallo: el estado de fallo no se verifica.

PASO 4.4.16 — Crear el archivo `ValidadorResultadosTests.cs`

Los tests verifican que el validador acepta el catálogo por defecto y rechaza las operaciones que violan las reglas del patrón.

1. En el panel Solution Explorer, hacer clic derecho sobre el proyecto `AceriaApp.ResultPattern.Tests`.
2. Seleccionar `Add` → `Class` y nombrar la clase `ValidadorResultadosTests.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `tests/AceriaApp.ResultPattern.Tests/ValidadorResultadosTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.ResultPattern;
using FluentAssertions;
using Xunit;

namespace AceriaApp.ResultPattern.Tests;

public class ValidadorResultadosTests
{
    private readonly ValidadorResultados _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoResultados.OperacionesPorDefecto());

        resultado.EsValido.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinOperaciones_DevuelveError()
    {
        var resultado = _validador.Validar(new List<ResultadoOperacion>());

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay operaciones"));
    }

    [Fact]
    public void Validar_SinNombre_DevuelveError()
    {
        var operaciones = new List<ResultadoOperacion>
        {
            new() { Nombre = string.Empty, UsaResult = true }
        };

        var resultado = _validador.Validar(operaciones);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara el nombre"));
    }

    [Fact]
    public void Validar_ErroresEsperadosSinResult_DevuelveError()
    {
        var operaciones = new List<ResultadoOperacion>
        {
            new() { Nombre = "OperacionX", UsaResult = false, NumeroDeErroresEsperados = 2 }
        };

        var resultado = _validador.Validar(operaciones);
```

```
        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no usa el patrón Result"));
    }

    [Fact]
    public void Validar_ExcepcionesParaErroresEsperados_DevuelveError()
    {
        var operaciones = new List<ResultadoOperacion>
        {
            new() { Nombre = "OperacionX", UsaResult = true, UsaExcepcionesParaErroresEsperados = true }
        };

        var resultado = _validador.Validar(operaciones);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("usa excepciones"));
    }
}
```

Verificación concreta. el archivo `ValidadorResultadosTests.cs` compila y contiene cinco métodos de test.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.ResultPattern;`: las clases no se resuelven.
- Usar `Assert.True` en lugar de `Should().BeTrue()`: el curso usa FluentAssertions.
- No cubrir el caso de lista vacía: el mensaje de error no se verifica.

PASO 4.4.17 — Integrar Result y sus respuestas HTTP y compilar la solución

El recorrido HTTP convierte un resultado de aplicación en códigos estables, como ausencia o conflicto, y confirma que una operación exitosa sí modifica el estado previsto. Probar únicamente que existe un tipo `Result` dejaría sin verificar la integración.

1. Abre cada archivo de integración funcional indicado en este paso. Crea los nuevos y reemplaza únicamente los que se señalen; copia todo el código mostrado.
2. Recorre las referencias: el dominio define sus entidades, la aplicación usa puertos y la infraestructura implementa persistencia; la API compone el flujo.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`; comprueba que todos los proyectos nuevos estén incluidos en la solución.
4. Verifica `Build succeeded.` y `0 Error(s)`. Si hay advertencias, revisa su causa; en el paso siguiente ejecutarás las pruebas del comportamiento completo.

Integración funcional. Expone /api/v1/patrones/ordenes sin alterar los endpoints heredados. 4.2 usa el caso transaccional; 4.3 enruta por ISender; 4.4 traduce Result a 201/200 y ProblemDetails 400/404/409 con código estable.

Archivo: `src/AceriaApp.Api/Patrones/PatronesOrdenesEndpoints.cs`. **Operación:** Reemplazar el contenido completo.

```
using MediatR;
using AceriaApp.Api.Hexagonal;
using AceriaApp.Hexagonal.Aplicacion.Patrones;
namespace AceriaApp.Api.Patrones;

public static class PatronesOrdenesEndpoints
{
    public static IEndpointRouteBuilder MapPatronesOrdenes(this IEndpointRouteBuilder app)
    {
        var group = app.MapGroup("/api/v1/patrones/ordenes").WithTags("Patrones");
        group.MapGet("/{id:guid}", async (Guid id, ISender sender, CancellationToken ct) =>
        {
            var resultado = await sender.Send(new ObtenerOrdenQuery(id), ct);
            return resultado.EsExito ? Results.Ok(OrdenHexagonalDto.Desde(resultado.Valor)) :
TraducirError(resultado.Error);
```

```
        }).WithName("ObtenerOrdenPatrones").Produces<OrdenHexagonalDto>(200).ProducesProblem(404);
        group.MapPost("", async (CrearOrdenHexagonalRequest body, ISender sender, CancellationToken ct) =>
        {
            var resultado = await sender.Send(new CrearOrdenCommand(body.Cliente, body.Material,
body.Cantidad), ct);
            if (!resultado.EsExito) return TraducirError(resultado.Error);
            var orden = resultado.Valor;
            return Results.CreatedAtRoute("ObtenerOrdenPatrones", new { id = orden.Id },
OrdenHexagonalDto.Desde(orden));
        }).WithName("CrearOrdenPatrones").Produces<OrdenHexagonalDto>(201).ProducesProblem(400);
        group.MapPost("/{id:guid}/finalizar", async (Guid id, ISender sender, CancellationToken ct) =>
        {
            var resultado = await sender.Send(new FinalizarOrdenCommand(id), ct);
            return resultado.EsExito ? Results.Ok(OrdenHexagonalDto.Desde(resultado.Valor)) :
TraducirError(resultado.Error);
        }).WithName("FinalizarOrdenPatrones").Produces<OrdenHexagonalDto>(200).ProducesProblem(404).ProducesP
roblem(409);
        return app;
    }

    private static IResult TraducirError(Error error) => Results.Problem(
        title: error.Descripcion,
        statusCode: error.Tipo switch
        {
            TipoError.Validacion => 400,
            TipoError.NoEncontrado => 404,
            TipoError.Conflicto => 409,
            _ => throw new ArgumentOutOfRangeException(nameof(error))
        },
        extensions: new Dictionary<string, object?> { ["codigo"] = error.Codigo });
}
```

Aísla la ruta de este módulo y conserva los endpoints existentes de M02 y M03. La API solo construye la consulta y la envía; el contenedor selecciona su manejador. Traduce la petición HTTP en un comando de aplicación y propaga la cancelación.

Selecciona la rama de error antes de leer el valor de éxito. Devuelve 201 y una cabecera Location calculada con la ruta de lectura nombrada y el identificador creado. Solicita la transición mediante el mediador y mantiene la lógica de negocio fuera del endpoint.

Centraliza la traducción del error de aplicación al cuerpo Problem Details de HTTP. Los datos inválidos producen 400; la categoría se mantiene separada del texto del mensaje. La ausencia del recurso produce 404.

Una transición incompatible con el estado del recurso produce 409. Añade el código estable del error como extensión de Problem Details para que el cliente no dependa del texto.

Integración funcional. Integra únicamente registros y mapeo del endpoint de patrones. Conserva íntegros los endpoints, documentación y arranque de M2/M3; en 4.5 reemplaza registros repetidos por AddPatronesOrdenes().

Archivo: `src/AceriaApp.Api/Program.cs`. **Operación:** Reemplazar el contenido completo.

```
using AceriaApp.Api.Patrones;
using AceriaApp.Hexagonal.Aplicacion.Patrones;
using AceriaApp.Hexagonal.Infraestructura.Patrones;
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Validacion;
using FluentValidation;
using AceriaApp.Api.Documentacion;

using AceriaApp.Api.Hexagonal;
using AceriaApp.Hexagonal.Infraestructura;
using AceriaApp.Hexagonal.Infraestructura.Datos;
var builder=WebApplication.CreateBuilder(args);
// INICIO PASO 2.1.12: SQL Server Express LocalDB, no SQLite.
```

```
builder.Services.AddDbContext<AceriaDbContext>(o=>
    o.UseSqlServer(builder.Configuration.GetConnectionString("AceriaApp")));
// PASO 2.3.14: registrar validadores en el contenedor.
builder.Services.AddValidatorsFromAssemblyContaining<CrearOrdenValidator>(); // PASO 2.3.14
// PASO 2.4.6: registrar generador antes de Build.
builder.Services.AddDocumentacion(); // PASO 2.4.6

builder.Services.AddServiciosHexagonales(builder.Configuration);
builder.Services.AddScoped<IOrdenTrabajoRepositorio, OrdenTrabajoRepositorio>();
builder.Services.AddScoped<IUnitOfWork, EfUnitOfWork>();
builder.Services.AddScoped<OrdenesTransaccionales>();
builder.Services.AddScoped<OrdenesConResultado>();
builder.Services.AddMediatR(cfg =>
{
    cfg.RegisterServicesFromAssemblyContaining<CrearOrdenCommand>();
    cfg.Lifetime = ServiceLifetime.Scoped;
    cfg.AddOpenBehavior(typeof(LoggingBehavior<,>));
});
var app=builder.Build();
// EnsureCreated se utiliza SOLO como ayuda didáctica en Development.
// Para un sistema productivo se utilizarían migraciones controladas.
// PASO 2.1.13: inicializar LocalDB solo en Development.
if(app.Environment.IsDevelopment()&&!app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope=app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<AceriaDbContext>().Database.EnsureCreatedAsync();
}

// INICIO PASO 2.1.14: colección de recursos, consulta real con EF Core.
app.MapGet("/api/v1/ordenes",async (AceriaDbContext db,CancellationToken ct)=>
{
    // PASO 2.1.15: lectura sin tracking y proyeccion a DTO.
    var ordenes=await db.Ordenes.AsNoTracking().OrderBy(x=>x.FechaCreacion).ToListAsync(ct);
    return Results.Ok(ordenes.Select(OrdenDto.Desde));
})
.WithName("ListarOrdenes").WithTags("Ordenes")
.WithSummary("Lista las órdenes de producción").Produces<IEnumerable<OrdenDto>>(200)
;
// INICIO PASO 2.1.17: recurso individual.
app.MapGet("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(x=>x.Id==id);
    // PASO 2.1.18: 200 para orden presente; 404 si no existe.
    IResult response=orden is null ? Results.NotFound() : Results.Ok(OrdenDto.Desde(orden));
    return response;
})
.WithName("ObtenerOrdenPorId").WithTags("Ordenes")
.WithSummary("Obtiene una orden por ID").Produces<OrdenDto>(200).Produces(404)
;


// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    // PASO 2.2.6: crear entidad y persistirla.
    var orden=new Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    // PASO 2.2.7: devolver 201 con Location.
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")
// PASO 2.3.15: filtro de POST.
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Crea una orden").Produces<OrdenDto>(201).ProducesValidationProblem(400).ProducesProblem(422)
;


// INICIO PASO 2.2.11: sustitución completa de los campos editables.
app.MapPut("/api/v1/ordenes/{id:guid}",async (Guid id,CrearOrdenRequest request,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
```

```
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cliente=request.Cliente;orden.Material=request.Material;
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("ReemplazarOrden").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Sustituye campos
editables").Produces(204).Produces(404).ProducesProblem(409).ProducesValidationProblem(400).ProducesProblem(4
22)
;


// INICIO PASO 2.2.14: cambiar SOLO la cantidad.
app.MapPatch("/api/v1/ordenes/{id:guid}/cantidad",
    async (Guid id,ActualizarCantidadRequest request,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("CambiarCantidad").WithTags("Ordenes")
// PASO 2.3.17: filtro de PATCH.
.AddEndpointFilter<ValidationFilter<ActualizarCantidadRequest>>()
.WithSummary("Modifica
cantidad").Produces<OrdenDto>(200).Produces(404).ProducesValidationProblem(400).ProducesProblem(409).Produces
Problem(422)
;


// INICIO PASO 2.2.16: eliminar recurso persistido.
app.MapDelete("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede eliminar una orden finalizada.",statusCode:409);
    db.Ordenes.Remove(orden);
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("EliminarOrden").WithTags("Ordenes")
.WithSummary("Elimina una orden pendiente").Produces(204).Produces(404).ProducesProblem(409)
;
// INICIO PASO 2.2.18: transición explícita del dominio.
app.MapPost("/api/v1/ordenes/{id:guid}/finalizar",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"La orden ya estaba finalizada.",statusCode:409);
    orden.Estado="Finalizada";
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("FinalizarOrden").WithTags("Ordenes")
.WithSummary("Finaliza una orden").Produces<OrdenDto>(200).Produces(404).ProducesProblem(409)
;

// PASO 2.4.8: habilitar Swagger solo en Development.
app.UseDocumentacion(); // PASO 2.4.8
// El contexto hexagonal usa su propia base para que EnsureCreated no dependa de
// las tablas heredadas del curso M02.
if (app.Environment.IsDevelopment() && !app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope = app.Services.CreateScope();
```

```
    await scope.ServiceProvider.GetRequiredService<HexagonalDbContext>()
        .Database.EnsureCreatedAsync();
}
app.MapHexagonalOrdenes();
app.MapPatronesOrdenes();
app.Run();
public partial class Program {} // permite WebApplicationFactory en tests
```

Permite mapear los endpoints del módulo desde la raíz de composición. Permite registrar las implementaciones de los puertos desde la API. Mantiene el registro SQL Server y los servicios que ya utilizaba M03.

Durante esta etapa se registra explícitamente el puerto de escritura con su adaptador. La unidad de trabajo comparte el contexto Scoped de la petición. Permite a los manejadores recibir el caso de uso que devuelve Result.

Conecta MediatR con los manejadores reales del ensamblado de aplicación. Registra el logging como comportamiento transversal del mediador. Publica las rutas del nuevo flujo después de construir la aplicación y antes de ejecutarla.

Integración funcional. Define command y query tipados y handlers que invocan el caso de uso real. En 4.4 los mensajes pasan a `Result<OrdenHexagonal>` y se añade FinalizarOrdenCommand, conservando el mismo flujo MediatR.

Archivo: `src/AceriaApp.Hexagonal.Aplicacion/Patrones/MensajesOrden.cs`. **Operación:** Reemplazar el contenido completo.

```
using AceriaApp.Hexagonal.Dominio;
using MediatR;
namespace AceriaApp.Hexagonal.Aplicacion.Patrones;

public sealed record CrearOrdenCommand(string Cliente, string Material, int Cantidad) :
IRequest<Result<OrdenHexagonal>>;
public sealed record ObtenerOrdenQuery(Guid Id) : IRequest<Result<OrdenHexagonal>>;
public sealed record FinalizarOrdenCommand(Guid Id) : IRequest<Result<OrdenHexagonal>>;

public sealed class CrearOrdenHandler(OrdenesConResultado caso) : IRequestHandler<CrearOrdenCommand,
Result<OrdenHexagonal>>
{
    public Task<Result<OrdenHexagonal>> Handle(CrearOrdenCommand request, CancellationToken ct) =>
        caso.CrearAsync(request.Cliente, request.Material, request.Cantidad, ct);
}
public sealed class ObtenerOrdenHandler(OrdenesConResultado caso) : IRequestHandler<ObtenerOrdenQuery,
Result<OrdenHexagonal>>
{
    public Task<Result<OrdenHexagonal>> Handle(ObtenerOrdenQuery request, CancellationToken ct) =>
caso.ObtenerAsync(request.Id, ct);
}
public sealed class FinalizarOrdenHandler(OrdenesConResultado caso) : IRequestHandler<FinalizarOrdenCommand,
Result<OrdenHexagonal>>
{
    public Task<Result<OrdenHexagonal>> Handle(FinalizarOrdenCommand request, CancellationToken ct) =>
caso.FinalizarAsync(request.Id, ct);
}
```

El comando transporta los datos de entrada e implementa IRequest con su tipo de respuesta; MediatR utiliza ese contrato para seleccionar el manejador. La consulta solicita una orden por identificador y declara el resultado que espera el emisor. El comando de transición se mantiene separado de la consulta y conserva el identificador del recurso.

El manejador recibe el caso de uso por constructor; no recibe DbContext ni construye infraestructura. Delega la operación completa y transmite la cancelación. La regla de negocio y la confirmación siguen en el caso de uso. MediatR asocia esta clase al contrato de consulta; su único trabajo es delegar la lectura.

El manejador de finalización devuelve el mismo resultado explícito que produce el caso de uso.

Integración funcional. Integra errores esperados de validación, ausencia y conflicto sin excepciones de negocio. Solo hace commit en creación válida o transición válida. Las excepciones operacionales y la cancelación siguen propagándose.

Archivo: `src/AceriaApp.Hexagonal.Aplicacion/Patrones/OrdenesConResultado.cs`. **Operación:** Crear.

```
using AceriaApp.Hexagonal.Dominio;
namespace AceriaApp.Hexagonal.Aplicacion.Patrones;

public sealed class OrdenesConResultado(IOrdenTrabajoRepositorio repositorio, IUnitOfWork trabajo)
{
    public async Task<Result<OrdenHexagonal>> CrearAsync(string cliente, string material, int cantidad,
CancellationToken ct = default)
    {
        if (string.IsNullOrWhiteSpace(cliente) || string.IsNullOrWhiteSpace(material)
            || cliente.Trim().Length > 100 || material.Trim().Length > 100 || cantidad <= 0 || cantidad >
500)
            return Result<OrdenHexagonal>.Fallo(new Error("orden.validacion",
                "Cliente y material de 1 a 100 caracteres; cantidad entre 1 y 500.", TipoError.Validacion));
        var orden = new OrdenHexagonal(cliente, material, cantidad);
        await repositorio.AgregarAsync(orden, ct);
        await trabajo.CommitAsync(ct);
        return Result<OrdenHexagonal>.Exito(orden);
    }

    public async Task<Result<OrdenHexagonal>> ObtenerAsync(Guid id, CancellationToken ct = default)
    {
        var orden = await repositorio.ObtenerAsync(id, ct);
        return orden is null
            ? Result<OrdenHexagonal>.Fallo(new Error("orden.ausente", "No existe la orden.",
TipoError.NoEncontrado))
            : Result<OrdenHexagonal>.Exito(orden);
    }

    public async Task<Result<OrdenHexagonal>> FinalizarAsync(Guid id, CancellationToken ct = default)
    {
        var resultado = await ObtenerAsync(id, ct);
        if (!resultado.EsExito) return resultado;
        var orden = resultado.Valor;
        if (orden.Estado == "Finalizada")
            return Result<OrdenHexagonal>.Fallo(new Error("orden.finalizada", "La orden ya está finalizada.",
TipoError.Conflicto));
        orden.Finalizar();
        await trabajo.CommitAsync(ct);
        return Result<OrdenHexagonal>.Exito(orden);
    }
}
```

El caso de uso conserva la dependencia de los puertos de repositorio y unidad de trabajo; no conoce HTTP. Comprueba los datos antes de crear una entidad o preparar escrituras. Representa un rechazo esperado con un código estable y una categoría de validación.

Prepara la nueva entidad solo después de superar la validación. Confirma el cambio una vez. Los fallos de infraestructura se propagan como excepciones y no se disfrazan de errores esperados. Expresa la ausencia como un fallo de resultado, permitiendo al adaptador elegir la respuesta HTTP.

Devuelve el fallo de lectura sin acceder al valor inexistente. Una segunda finalización produce conflicto y no vuelve a modificar ni confirmar la orden. El dominio realiza la transición válida. El contexto conserva el seguimiento de la entidad recuperada.

Integración funcional. Implementa un `Result<T>` inmutable con constructor privado y factories que rechazan null. El éxito contiene valor y el fallo un error tipado; acceder a la rama incorrecta lanza. Error valida código, descripción y tipo. No colisiona con el ejemplo del catálogo en API.

Archivo: `src/AceriaApp.Hexagonal.Aplicacion/Patrones/Result.cs`. **Operación:** Crear.

```
namespace AceriaApp.Hexagonal.Aplicacion.Patrones;

public enum TipoError { Validacion, NoEncontrado, Conflicto }

public sealed record Error
{
    public string Codigo { get; }
    public string Descripcion { get; }
    public TipoError Tipo { get; }
    public Error(string codigo, string descripcion, TipoError tipo)
    {
        if (string.IsNullOrWhiteSpace(codigo)) throw new ArgumentException("Código requerido.",
nameof(codigo));
        if (string.IsNullOrWhiteSpace(descripcion)) throw new ArgumentException("Descripción requerida.",
nameof(descripcion));
        if (!Enum.IsDefined(tipo)) throw new ArgumentOutOfRangeException(nameof(tipo));
        Codigo = codigo; Descripcion = descripcion; Tipo = tipo;
    }
}

/// <summary>Solo puede construirse un éxito con valor o un fallo con error.</summary>
public sealed class Result<T> where T : notnull
{
    private readonly T? valor;
    private readonly Error? error;
    public bool EsExito { get; }
    public T Valor => EsExito ? valor! : throw new InvalidOperationException("Un fallo no contiene valor.");
    public Error Error => !EsExito ? error! : throw new InvalidOperationException("Un éxito no contiene
error.");
    private Result(T valor) { this.valor = valor; EsExito = true; }
    private Result(Error error) { this.error = error; EsExito = false; }
    public static Result<T> Exito(T valor)
    { ArgumentNullException.ThrowIfNull(valor); return new Result<T>(valor); }
    public static Result<T> Fallo(Error error)
    { ArgumentNullException.ThrowIfNull(error); return new Result<T>(error); }
}
```

Limita las categorías esperadas a validación, ausencia y conflicto, que se traducirán en el adaptador HTTP. Agrupa código estable, descripción y categoría en un contrato inmutable. Rechaza contratos de error sin código. Es una defensa ante un uso incorrecto del tipo, no un error de negocio del usuario.

Rechaza categorías inválidas incluso si se convierten desde un entero. El genérico exige un valor no nulo en la rama de éxito. Permite leer el valor únicamente en un éxito; una lectura de la rama equivocada es un error de programación.

Permite leer el error únicamente en un fallo, evitando estados ambiguos. El constructor privado establece solo la rama de éxito; no acepta simultáneamente valor y error. El constructor privado establece solo la rama de fallo.

La fábrica pública controla la creación del éxito y rechaza un valor nulo. La fábrica pública controla la creación del fallo y rechaza un error nulo.

Integración funcional. Verifica persistencia y consulta mediante ISender, ejecución observable del pipeline sin payloads y cancelación sin efectos persistidos. En 4.4 adapta la lectura a Result.Valor y registra OrdenesConResultado.

Archivo: `tests/AceriaApp.Mediator.Tests/MediatorFuncionalesTests.cs`. **Operación:** Reemplazar el contenido completo.

```
using AceriaApp.Hexagonal.Aplicacion.Patrones;
using AceriaApp.Hexagonal.Infraestructura.Datos;
using AceriaApp.Hexagonal.Infraestructura.Patrones;
using MediatR;
using AceriaApp.PruebasSqlServer;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;
using Xunit;
```

```
namespace AceriaApp.Mediator.Tests;

public sealed class MediatorFuncionalesTests
{
    [Fact]
    public async Task Command_y_query_ejecutan_el_caso_de_uso_y_devuelven_datos_persistidos()
    {
        await using var fixture = await Entorno.CrearAsync();
        using var scope = fixture.Services.CreateScope();
        var sender = scope.ServiceProvider.GetRequiredService<ISender>();
        await sender.Send(new CrearOrdenCommand("Axia", "Acero", 20));
        await using var db = fixture.Contexto();
        var id = (await db.Ordenes.SingleAsync()).Id;
        var recuperada = await sender.Send(new ObtenerOrdenQuery(id));
        Assert.Equal("Axia", recuperada.Valor.Cliente);
        Assert.Equal(20, recuperada.Valor.Cantidad);
    }

    [Fact]
    public async Task Pipeline_registra_inicio_y_fin_sin_datos_de_cliente()
    {
        await using var fixture = await Entorno.CrearAsync();
        using var scope = fixture.Services.CreateScope();
        await scope.ServiceProvider.GetRequiredService<ISender>()
            .Send(new CrearOrdenCommand("Cliente confidencial", "Acero", 20));
        Assert.Contains(fixture.Logs.Mensajes, m => m.Contains("Inicio CrearOrdenCommand"));
        Assert.Contains(fixture.Logs.Mensajes, m => m.Contains("Fin CrearOrdenCommand"));
        Assert.DoesNotContain(fixture.Logs.Mensajes, m => m.Contains("Cliente confidencial"));
    }

    [Fact]
    public async Task Cancelacion_del_command_no_persiste_la_orden()
    {
        await using var fixture = await Entorno.CrearAsync();
        using var scope = fixture.Services.CreateScope();
        using var cancelacion = new CancellationTokenSource();
        cancelacion.Cancel();
        await Assert.ThrowsAnyAsync<OperationCanceledException>(() => scope.ServiceProvider
            .GetRequiredService<ISender>().Send(new CrearOrdenCommand("Axia", "Acero", 20),
cancelacion.Token));
        await using var db = fixture.Contexto();
        Assert.Empty(await db.Ordenes.ToListAsync());
    }

    private sealed class Entorno(BaseSqlServer store, ServiceProvider services, RegistroLogs logs) :
IAsyncDisposable
    {
        public ServiceProvider Services => services;
        public RegistroLogs Logs => logs;
        public HexagonalDbContext Contexto() => new(new
DbContextOptionsBuilder<HexagonalDbContext>().UseSqlServer(store.ConnectionString).Options);
        public static async Task<Entorno> CrearAsync()
        {
            var store = BaseSqlServer.Crear();
            var logs = new RegistroLogs();
            var services = new ServiceCollection();
            services.AddLogging(builder => builder.AddProvider(logs));
            services.AddDbContext<HexagonalDbContext>(o => o.UseSqlServer(store.ConnectionString));
            services.AddScoped<IOrdenTrabajoRepositorio, OrdenTrabajoRepositorio>();
            services.AddScoped<IUnitOfWork, EfUnitOfWork>();
            services.AddScoped<OrdenesTransaccionales>();
            services.AddScoped<OrdenesConResultado>();
            services.AddMediatR(cfg => {
                cfg.RegisterServicesFromAssemblyContaining<CrearOrdenCommand>();
                cfg.Lifetime = ServiceLifetime.Scoped;
                cfg.AddOpenBehavior(typeof(LoggingBehavior<,>));
            });
            var fixture = new Entorno(store, services.BuildServiceProvider(new ServiceProviderOptions
{ ValidateScopes = true }), logs);
            try
            {
                await using var db = fixture.Contexto();
                Assert.Equal("Microsoft.EntityFrameworkCore.SqlServer", db.Database.ProviderName);
                await db.Database.EnsureCreatedAsync();
            }
            catch { await fixture.DisposeAsync(); throw; }
```

```
            return fixture;
        }
        public async ValueTask DisposeAsync() { await services.DisposeAsync(); await store.DisposeAsync(); }
    }

    private sealed class RegistroLogs : ILoggerProvider
    {
        public List<string> Mensajes { get; } = new();
        public ILogger CreateLogger(string name) => new Captura(Mensajes);
        public void Dispose() { }
        private sealed class Captura(List<string> mensajes) : ILogger
        {
            public IDisposable? BeginScope<TState>(TState state) where TState : notnull => null;
            public bool IsEnabled(LogLevel level) => true;
            public void Log<TState>(LogLevel level, EventId id, TState state, Exception? error, Func<TState,
Exception?, string> formatter) =>
                mensajes.Add(formatter(state, error));
        }
    }
}
```

El mediador ejecuta el caso de uso y una lectura independiente verifica la orden persistida. El pipeline registra inicio y fin sin incluir el nombre del cliente. Una cancelación previa impide persistir una orden.

En `var store = BaseSqlServer.Crear();` se asigna una base SQL Server aislada al entorno de cada prueba. Los ámbitos de DI reciben contextos SQL Server sobre la misma base de esa prueba. Comprueba el proveedor antes de crear las tablas.

En `catch { await fixture.DisposeAsync(); throw; }` se si falla la inicialización, libera también la base temporal. Libera los servicios antes de eliminar la base que les pertenece.

Integración funcional. Conserva las dependencias de los tests de catálogo y añade Testcontainers.MsSql, el helper compartido y WebApplicationFactory para las pruebas funcionales.

Archivo: `tests/AceriaApp.ResultPattern.Tests/AceriaApp.ResultPattern.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings><IsPackable>false</IsPackable><IsTestProject>true</IsTestProject></PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="Testcontainers.MsSql" Version="3.10.0" />
    <PackageReference Include="Microsoft.AspNetCore.Mvc.Testing" Version="8.0.21" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <Compile Include="../Compartido/SqlServerPruebas.cs" Link="SqlServerPruebas.cs" />
  </ItemGroup>
</Project>
```

Compila para .NET 8, igual que los proyectos heredados. Fija el cliente de contenedores para SQL Server; el proveedor EF Core SQL Server llega mediante la referencia a la API. Permite ejecutar peticiones HTTP contra el host real de la API.

Compila el helper heredado que crea y elimina únicamente la base aislada de cada prueba, sin añadir otro proyecto a la solución.

Integración funcional. Prueba invariantes de Result y flujo HTTP real con WebApplicationFactory y SQL Server: 201 con Location, 400 sin persistencia, 404 de lectura y finalización, y 409 al repetir la transición.

Archivo: `tests/AceriaApp.ResultPattern.Tests/ResultFuncionalesTests.cs`. **Operación:** Crear.

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Api.Hexagonal;
using AceriaApp.Hexagonal.Aplicacion.Patrones;
using AceriaApp.Hexagonal.Infraestructura.Datos;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.AspNetCore.TestHost;
using AceriaApp.PruebasSqlServer;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.DependencyInjection.Extensions;
using Xunit;
namespace AceriaApp.ResultPattern.Tests;

public sealed class ResultFuncionalesTests
{
    [Fact]
    public void Factories_impiden_resultados_o_errores_incompletos()
    {
        Assert.Throws<ArgumentNullException>(() => Result<string>.Exito(null!));
        Assert.Throws<ArgumentNullException>(() => Result<string>.Fallo(null!));
        Assert.Throws<ArgumentException>(() => new Error(" ", "Detalle", TipoError.Validacion));
        Assert.Throws<ArgumentException>(() => new Error("codigo", " ", TipoError.Validacion));
        Assert.Throws<ArgumentOutOfRangeException>(() => new Error("codigo", "Detalle", (TipoError)99));
    }

    [Fact]
    public void Un_resultado_no_expone_la_rama_que_no_le_corresponde()
    {
        var exito = Result<int>.Exito(5);
        var fallo = Result<int>.Fallo(new Error("orden.ausente", "No existe.", TipoError.NoEncontrado));
        Assert.True(exito.EsExito); Assert.Equal(5, exito.Valor);
        Assert.False(fallo.EsExito); Assert.Equal(TipoError.NoEncontrado, fallo.Error.Tipo);
        Assert.Throws<InvalidOperationException>(() => exito.Error);
        Assert.Throws<InvalidOperationException>(() => fallo.Valor);
    }

    [Fact]
    public async Task Crear_devuelve_201_location_y_lectura_del_recurso_persistido()
    {
        using var factory = new PatronesFactory(); using var client = factory.CreateClient();
        var response = await client.PostAsJsonAsync("/api/v1/patrones/ordenes", new { Cliente = "Axia",
Material = "Acero", Cantidad = 20 });
        Assert.Equal(HttpStatusCode.Created, response.StatusCode);
        Assert.NotNull(response.Headers.Location);
        var lectura = await client.GetAsync(response.Headers.Location);
        Assert.Equal(HttpStatusCode.OK, lectura.StatusCode);
        var dto = await lectura.Content.ReadFromJsonAsync<OrdenHexagonalDto>();
        Assert.Equal("Axia", dto!.Cliente); Assert.Equal(20, dto.Cantidad);
    }

    [Fact]
    public async Task Datos_invalidos_devuelven_400_sin_crear_ordenes()
    {
        using var factory = new PatronesFactory(); using var client = factory.CreateClient();
        var casos = new[] { (" ", 20), ("Axia", 0), ("Axia", 501), (new string('A', 101), 20) };
        foreach (var (cliente, cantidad) in casos)
        {
            var response = await client.PostAsJsonAsync("/api/v1/patrones/ordenes", new { Cliente = cliente,
Material = "Acero", Cantidad = cantidad });
            Assert.Equal(HttpStatusCode.BadRequest, response.StatusCode);
            Assert.Contains("orden.validacion", await response.Content.ReadAsStringAsync());
        }
        using var scope = factory.Services.CreateScope();
        Assert.Equal(0, await
scope.ServiceProvider.GetRequiredService<HexagonalDbContext>().Ordenes.CountAsync());
    }

    [Fact]
    public async Task Consultar_una_orden_ausente_devuelve_404()
    {
        using var factory = new PatronesFactory(); using var client = factory.CreateClient();
        var response = await client.GetAsync($"/api/v1/patrones/ordenes/{Guid.NewGuid()}");
        Assert.Equal(HttpStatusCode.NotFound, response.StatusCode);
```

```
        Assert.Contains("orden.ausente", await response.Content.ReadAsStringAsync());
    }

    [Fact]
    public async Task Finalizar_una_orden_ausente_devuelve_404()
    {
        using var factory = new PatronesFactory(); using var client = factory.CreateClient();
        var response = await client.PostAsync($"/api/v1/patrones/ordenes/{Guid.NewGuid()}/finalizar", null);
        Assert.Equal(HttpStatusCode.NotFound, response.StatusCode);
    }

    [Fact]
    public async Task Finalizar_dos_veces_devuelve_409_y_conserva_la_primera_transicion()
    {
        using var factory = new PatronesFactory(); using var client = factory.CreateClient();
        var creada = await client.PostAsJsonAsync("/api/v1/patrones/ordenes", new { Cliente = "Axia",
Material = "Acero", Cantidad = 20 });
        var ruta = creada.Headers.Location + "/finalizar";
        Assert.Equal(HttpStatusCode.OK, (await client.PostAsync(ruta, null)).StatusCode);
        var repetida = await client.PostAsync(ruta, null);
        Assert.Equal(HttpStatusCode.Conflict, repetida.StatusCode);
        Assert.Contains("orden.finalizada", await repetida.Content.ReadAsStringAsync());
        var dto = await client.GetFromJsonAsync<OrdenHexagonalDto>(creada.Headers.Location);
        Assert.Equal("Finalizada", dto!.Estado);
    }

    private sealed class PatronesFactory : WebApplicationFactory<Program>
    {
        private readonly BaseSqlServer store = BaseSqlServer.Crear();
        public PatronesFactory()
        {
            try
            {
                using var db = new HexagonalDbContext(new
DbContextOptionsBuilder<HexagonalDbContext>().UseSqlServer(store.ConnectionString).Options);
                Assert.Equal("Microsoft.EntityFrameworkCore.SqlServer", db.Database.ProviderName);
                db.Database.EnsureCreated();
            }
            catch { store.Dispose(); throw; }
        }
        protected override void ConfigureWebHost(IWebHostBuilder builder)
        {
            builder.UseEnvironment("Development"); builder.UseSetting("SkipDatabaseInitialization", "true");
            builder.ConfigureTestServices(services => {
                services.RemoveAll<DbContextOptions<HexagonalDbContext>>();
                services.AddDbContext<HexagonalDbContext>(o => o.UseSqlServer(store.ConnectionString));
            });
        }
        protected override void Dispose(bool disposing)
        { base.Dispose(disposing); if (disposing) store.Dispose(); }
    }
}
```

Comprueba las invariantes de los resultados y sus errores. La creación HTTP devuelve un recurso persistido con Location. Los datos inválidos producen 400 y ninguna escritura.

La segunda finalización devuelve conflicto y conserva el estado de la primera. En `private readonly BaseSqlServer store = BaseSqlServer.Crear();` se cada host de prueba es dueño de una base SQL Server distinta. La inicialización falla si no está activo el proveedor SQL Server.

Sustituye la conexión del contexto por la base aislada de esta prueba. En `{ base.Dispose(disposing); if (disposing) store.Dispose(); }` se cierra el host antes de eliminar su base de prueba.

Verificación concreta. La solución completa compila, incluidos el código de integración y sus pruebas. Las siguientes ejecuciones de tests comprobarán el comportamiento de los patrones.

Diagnóstico si falla.

- Tener la API en ejecución: el archivo `.dll` está bloqueado y la compilación falla.

- No restaurar paquetes: `dotnet build` restaura por defecto, pero si hay un error de red, hay que ejecutar `dotnet restore` primero.
- Usar una versión distinta del SDK: `dotnet --version` debe devolver `8.x.x`.

PASO 4.4.18 — Ejecutar todos los tests del punto 4.4

Ejecutar los tests del punto 4.4 verifica que las clases del patrón Result funcionan correctamente.

1. Ejecuta `dotnet test tests/AceriaApp.ResultPattern.Tests/AceriaApp.ResultPattern.Tests.csproj -c Release` desde la raíz de esta etapa.
2. Comprueba un total de 15 pruebas: 8 de catálogo y 7 funcionales, con cero fallos y cero omisiones.
3. Si falla una prueba, abre su mensaje de aserción y el archivo indicado; corrige el comportamiento antes de continuar.
4. Repite el mismo comando tras la corrección y conserva el resultado de la ejecución válida.

Verificación concreta. El proyecto `AceriaApp.ResultPattern.Tests` termina con `Failed: 0, Passed: 15, Skipped: 0, Total: 15`.

Diagnóstico si falla.

- Ejecutar otro proyecto: comprueba la ruta y el nombre de la asamblea de pruebas antes de interpretar los resultados.
- No compilar antes: `dotnet test` compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

PASO 4.4.19 — Ejecutar todos los tests de la solución

Ejecutar todos los tests de la solución verifica que los tests de los puntos anteriores y del punto 4.4 pasan conjuntamente.

1. Ejecuta `dotnet build AceriaApp.sln -c Release` para producir todas las asambleas de esta etapa.
2. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build --no-restore` para ejecutar todas las pruebas acumuladas.
3. Suma los resultados de los 9 proyectos de tests: deben ser 105 pruebas aprobadas, sin fallos ni omisiones.
4. Comprueba que las pruebas heredadas de M03 y los puntos anteriores de M04 siguen presentes; no borres tests para obtener un resultado verde.

Verificación concreta. La suma de las líneas de resultados es `Failed: 0`, `Passed: 105`, `Skipped: 0` y `Total: 105`. Dotnet muestra un resumen por proyecto, no necesariamente una única línea global.

Diagnóstico si falla.

- No haber compilado antes: `--no-build` falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
- Eliminar pruebas anteriores o cambiarlas sin una instrucción explícita: se pierde la cobertura de los contratos heredados.

PASO 4.4.20 — Crear el README del punto 4.4

El README del punto 4.4 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 4.3.

1. En la carpeta del punto `4.4-result-pattern`, hacer clic derecho y seleccionar `Add` → `New Item` → `Text File`.
2. Nombrar el archivo `README.md`.

3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que el README del punto 4.3 no se ha modificado.

Archivo: `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 4.4 — Result Pattern

## Objetivo
Comprender el patrón Result y añadir al proyecto las clases de validación de resultados.

La práctica conecta además el patrón con los puertos, adaptadores y pruebas de integración que aparecen en
este manual.

## Estado del proyecto
- Solución `AceriaApp.sln` con 13 proyectos.
- Documento `docs/patron-result.md` con la definición y las reglas.
- Clases `Result<T>`, `ResultadoOperacion`, `ValidadorResultados` y `CatalogoResultados` en `src/
AceriaApp.Api/ResultPattern/`.
- 8 tests nuevos de catálogo y 7 funcionales en `tests/AceriaApp.ResultPattern.Tests`.
- Los tests acumulados suman 105 pruebas: 8 de catálogo nuevas y 7 funcionales nuevas en este punto.

## Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio 2026.
2. Verificar que SQL Server Express está arrancado.
3. Configurar las conexiones SQL Server `AceriaApp` y `AceriaHexagonal` en el entorno local.
4. Ejecutar `dotnet run --project src/AceriaApp.Api/AceriaApp.Api.csproj --urls http://localhost:5000` desde
la raíz de esta etapa.
5. Navegar a `http://localhost:5000/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

## Qué se ha añadido respecto al punto anterior
- Documento `docs/patron-result.md`.
- Carpeta `ResultPattern` en el proyecto API con cuatro clases nuevas.
- Proyecto `tests/AceriaApp.ResultPattern.Tests` con 8 tests de catálogo y 7 funcionales nuevos.
- Se conserva intacta la carpeta del punto 4.3.

## Documentación
- [ADR-010](docs/ADR-010-capas-aplicacion.md) a [ADR-016](docs/ADR-016-patron-mediator.md) (heredados).
- [Patrón Result](docs/patron-result.md).

## Verificación ejecutable

Los catálogos describen ejemplos de diseño. El comportamiento de las clases reales se comprueba además con
los tests funcionales del punto. Ejecuta `dotnet test AceriaApp.sln -c Release` y exige cero fallos y cero
omisiones.
```

Verificación concreta. El archivo `README.md` existe y contiene las 6 secciones mostradas; abre su vista previa y comprueba todos los apartados.

Diagnóstico si falla.

- Modificar el README del punto 4.3: rompe la conservación de la etapa anterior.
- Omitir la sección `Qué se ha añadido respecto al punto anterior`: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 4.3: rompe la separación.

PASO 4.4.21 — Crear el ADR-017 del punto 4.4

El ADR-017 documenta la decisión de adoptar el patrón Result. Es un artefacto nuevo del punto 4.4 que no modifica los ADR existentes.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `docs`.
2. Seleccionar `Add` → `New Item` → `Text File` y nombrar el archivo `ADR-017-patron-result.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que los ADR-010 a ADR-016 no se han modificado.

Archivo: `docs/ADR-017-patron-result.md`. **Operación:** Crear.

```
# ADR-017: Adopción del patrón Result

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha adoptado los patrones Repository, Unit of Work y Mediator. Es necesario definir cómo se representan los errores en los casos de uso.

## Decisión
Se adopta `AceriaApp.Hexagonal.Aplicacion.Patrones.Result<T>` para los errores esperados del nuevo flujo. `OrdenesConResultado` valida las entradas y devuelve `Exito` o `Fallo` con `Error` tipado; los errores de infraestructura y de programación siguen propagándose como excepciones. Los manejadores de MediatR conservan el resultado explícito y `PatronesOrdenesEndpoints` lo traduce a 201 con Location, 200, 400, 404 o 409 y Problem Details. `LoggingBehavior` permanece como pipeline de observación; no se incorpora un comportamiento de FluentValidation. El tipo básico `AceriaApp.Api.ResultPattern.Result<T>` se mantiene para el ejercicio de catálogo y sus pruebas. Siete pruebas funcionales verifican las fábricas, las ramas del resultado y las respuestas HTTP con persistencia real de prueba.

## Alternativas consideradas
- Uso exclusivo de excepciones: descartado por coste y falta de expresividad.
- Uso de códigos de error numéricos: descartado por menor legibilidad.
- Uso de una jerarquía de excepciones personalizadas: descartado por mezclar errores esperados y excepcionales.

## Consecuencias
- Positivas: errores explícitos, contrato de respuesta verificable, flujo de errores controlado.
- Negativas: la estructura del resultado añade código adicional.
- Mitigación: la validación automática y el uso consistente del patrón.
```

Verificación concreta. El archivo `docs/ADR-017-patron-result.md` existe y contiene las 5 secciones mostradas; abre su vista previa y comprueba todos los apartados.

Diagnóstico si falla.

- Modificar los ADR anteriores: rompe la conservación de la etapa anterior.
- Omitir la sección `Alternativas consideradas`: el ADR pierde valor.
- Nombrar el archivo sin el prefijo `ADR-017`: rompe la convención.

PASO 4.4.22 — Verificar que los documentos heredados y la etapa 4.3 permanecen intactos

Las decisiones previas se conservan como registros históricos. Los nuevos ADR explican la evolución y las modificaciones de integración se aplican solo a la copia de la etapa actual.

1. Vuelve a la raíz del repositorio; las rutas siguientes se interpretan desde allí.
2. En CMD ejecuta `for %f in (M04_PATRONES_ARQUITECTURA\PROYECTO\4.3-mediator\docs\*.md) do fc /L "%f" "M04_PATRONES_ARQUITECTURA\PROYECTO\4.4-result-pattern\docs\%~nxf"`. En PowerShell puedes comparar `Get-FileHash` para cada par de documentos.
3. Comprueba 14 comparaciones y exige igualdad en todas; se conserva exactamente el contenido de los documentos heredados.
4. Si una comparación falla, restaura el documento de la etapa actual desde `4.3-mediator`. Verifica también que la carpeta del punto anterior no tiene modificaciones.

Verificación concreta. Los 14 documentos heredados coinciden byte a byte y el punto anterior permanece intacto.

Diagnóstico si falla.

- Usar `comp` en lugar de `fc`: `comp` compara binarios y no muestra diferencias de texto.
- Omitir `/L` cuando se desea una comparación de texto: indica el modo explícitamente y revisa cada resultado.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

PASO 4.4.23 — Verificar la estructura del punto 4.4

La verificación estructural confirma que el punto 4.4 añade archivos nuevos conservando los documentos heredados.

1. Escribir `tree /F M04_PATRONES_ARQUITECTURA\PROYECTO\4.4-result-pattern` y pulsar Enter.
2. Verificar que la salida muestra `AceriaApp.sln`, `README.md`, `src\AceriaApp.Api\ResultPattern` con las cuatro clases nuevas, `tests\AceriaApp.ResultPattern.Tests` con los dos archivos de catálogo y el archivo de tests funcionales y `docs` con el documento del patrón y el ADR-017. Comprueba además todos los archivos de integración de las carpetas Patrones y los cambios de composición indicados en este punto.
3. Verificar que las carpetas existentes del punto 4.3 permanecen intactas.
4. Verificar que no existe ninguna carpeta `labs` ni ningún archivo `metaprompt.md` ni `verificacion.md`.

Verificación concreta. la estructura contiene los archivos nuevos y conserva los del punto 4.3.

Diagnóstico si falla.

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 4.3: viola la regla de no modificar.
- Omitir la carpeta `ResultPattern`: los tests no encuentran las clases.

PASO 4.4.24 — Hacer el commit del punto 4.4 y verificar el estado final

El commit permite comparar esta etapa con la anterior. Si has realizado un commit por punto desde 1.1, el recorrido suma 17 commits de puntos; otros commits legítimos pueden aumentar el historial.

1. Desde la raíz del repositorio ejecuta `git status` y revisa que solo aparezcan los archivos de esta etapa y los entregables que has trabajado.
2. Ejecuta `git add .` después de verificar que `bin/`, `obj/`, `TestResults/` y credenciales están excluidos.
3. Ejecuta `git commit -m "Punto 4.4: Result Pattern"` con `user.name` y `user.email` configurados.
4. Ejecuta `git log --oneline` y `git status`; comprueba que se registró el commit de este punto y que no quedan cambios pendientes de esta actividad.

Verificación concreta. El historial incluye el commit de 4.4 y `git status` no muestra cambios pendientes de esta actividad; no se exige un número rígido de commits a un repositorio con más trabajo.

Diagnóstico si falla.

- No tener configurado `user.name` y `user.email`: `git commit` falla.
- Añadir la carpeta `bin/` y `obj/` al commit: el `.gitignore` lo evita.
- Hacer commit antes de `git add .`: no hay cambios en el índice.

Reto de ampliación (optativo)

Trabaja en una copia independiente de la etapa obligatoria, por ejemplo `4.4-result-pattern-reto`. Mantén intacta la carpeta obligatoria y todos sus documentos. Los tres fragmentos siguientes añaden un contrato, un cálculo y una prueba al proyecto del punto.

Reto: añadir un método `Resumen` a `ValidadorResultados` que devuelva el número total de operaciones, el número de operaciones que usan Result y el número total de errores esperados, y un test que verifique el cálculo con el catálogo por defecto.

Paso 1: En `ValidadorResultados.cs`, añadir el siguiente `record` al final del archivo:

Archivo: `src/AceriaApp.Api/ResultPattern/ValidadorResultados.cs`. **Operación:** Modificar.

Punto de inserción: después de este texto exacto, que debe aparecer una sola vez:

```
public record ResultadoResultados(bool EsValido, IReadOnlyList<string> Errores);
```

```
public record ResumenOperaciones(
    int Total,
    int UsanResult,
    int TotalErroresEsperados);
```

Paso 2: Explicar el propósito del fragmento y las decisiones que contiene:

La declaración `public record ResumenOperaciones(` declara un contrato inmutable. Número total de operaciones. Número de operaciones que usan Result. Número total de errores esperados.

Paso 3: En `ValidadorResultados.cs`, añadir el siguiente método estático dentro de la clase:

Archivo: `src/AceriaApp.Api/ResultPattern/ValidadorResultados.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
}

public record ResultadoResultados(bool EsValido, IReadOnlyList<string> Errores);
```

Indenta el fragmento con cuatro espacios dentro de la clase; conserva el cierre de clase que aparece en el texto de inserción.

```
public static ResumenOperaciones Resumen(IEnumerable<ResultadoOperacion> operaciones)
{
    var lista = operaciones.ToList();
    var usanResult = lista.Count(o => o.UsaResult);
    var totalErrores = lista.Sum(o => o.NumeroDeErroresEsperados);
    return new ResumenOperaciones(lista.Count, usanResult, totalErrores);
}
```

Paso 4: Explicar el propósito del fragmento y las decisiones que contiene:

La declaración `public static ResumenOperaciones Resumen(IEnumerable<ResultadoOperacion>` declara el método. Materializa la lista. Cuenta las operaciones que usan Result. Suma los errores esperados. Devuelve el resumen.

Paso 5: Guardar el archivo con `Ctrl+S`.

Paso 6: En `ValidadorResultadosTests.cs`, añadir el siguiente test:

Archivo: `tests/AceriaApp.ResultPattern.Tests/ValidadorResultadosTests.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
}
```

Indenta el fragmento con cuatro espacios dentro de la clase; conserva el cierre de clase que aparece en el texto de inserción.

```
[Fact]
public void Resumen_ConCatalogoPorDefecto_CalculaTotales()
{
    var resumen = ValidadorResultados.Resumen(CatalogoResultados.OperacionesPorDefecto());

    resumen.Total.Should().Be(4);
    resumen.UsanResult.Should().Be(4);
    resumen.TotalErroresEsperados.Should().Be(6);
}
```

Paso 7: Interpretar el test y explicar qué comportamiento acredita:

Marca el método como test de xUnit. Declara el test. Invoca el método. Verifica el total. Verifica las operaciones que usan Result. Verifica el total de errores.

Paso 8: Ejecuta `dotnet test tests/AceriaApp.ResultPattern.Tests/AceriaApp.ResultPattern.Tests.csproj -c Release` en la copia optativa. Deben pasar 16 tests del proyecto, incluido el nuevo resumen. Ejecuta después `dotnet test AceriaApp.sln -c Release` y exige 106 pruebas aprobadas.

Paso 9: Comprueba que solo cambian `ValidadorResultados.cs` y `ValidadorResultadosTests.cs` en la copia optativa; la etapa obligatoria `4.4-result-pattern` y el punto anterior deben permanecer idénticos.

Paso 10: Hacer commit con `git commit -m "Reto 4.4: método Resumen en ValidadorResultados"`.

Verificación: `Passed: 16` en el proyecto del punto y 106 pruebas aprobadas en la solución de la copia optativa, sin fallos ni omisiones.

Resultado verificable

Al finalizar este punto, el alumno debe tener:

- La carpeta `M04_PATRONES_ARQUITECTURA\PROYECTO/4.4-result-pattern/` con la solución `AceriaApp.sln` que contiene 13 proyectos.
- El documento `docs/patron-result.md` con la definición y las reglas.
- El documento `docs/ADR-017-patron-result.md` con la decisión sobre el patrón Result.
- Las clases `Result<T>`, `ResultadoOperacion`, `ValidadorResultados` y `CatalogoResultados` en `src/AceriaApp.Api/ResultPattern/`.
- El proyecto `tests/AceriaApp.ResultPattern.Tests` añade 8 tests de catálogo y 7 funcionales; el reto añade uno más.
- `dotnet test AceriaApp.sln -c Release` ejecuta 105 pruebas en la etapa obligatoria y 106 en una copia con su reto; ninguna falla ni se omite.
- Los ADR-010 a ADR-016 del punto 4.3 sin modificar.
- Un commit revisado para el punto 4.4, con el historial previo conservado.

Continuidad del aprendizaje

El punto 4.4 ha definido el patrón Result como mecanismo para representar errores esperados como valores en lugar de excepciones. El proyecto AceriaApp dispone ahora de un validador automático de resultados que comprueba el uso correcto del patrón, y 8 tests nuevos de catálogo más 7 funcionales. Esta base permite avanzar hacia el punto 4.5, donde se estudiará la inyección de dependencias como mecanismo para componer los puertos y los adaptadores en el contenedor, y se cerrará el módulo 4 con el resumen del estado del proyecto.

Punto 4.5 — Inyección de dependencias

Componer los adaptadores con el ciclo de vida correcto

La inversión de dependencias solo funciona en ejecución si ASP.NET Core puede resolver cada puerto y entregar al caso de uso el adaptador adecuado. El contenedor administra además ciclos de vida: un DbContext suele ser Scoped por petición y no debería capturarse en un singleton que prolongue su existencia. El último punto de M04 explica estas decisiones en el punto de composición y comprueba que las instancias se utilizan en el ámbito previsto.

La práctica parte de la solución acumulada del punto 4.4. Conservaremos los ejercicios de catálogo para estudiar registros válidos e inválidos, pero los contrastaremos con servicios realmente registrados. La compilación y las pruebas de integración nos permitirán cerrar M04 con un flujo funcional desde HTTP hasta SQL Server, además de un ADR que justifica por qué la API depende de abstracciones mientras infraestructura contiene las implementaciones.

Carpeta de trabajo: `M04_PATRONES_ARQUITECTURA/PROYECTO/4.5-inyeccion-dependencias`.

Objetivo: completar los veinticuatro pasos, conservar los documentos heredados y comprobar el patrón con código y tests ejecutables. La etapa suma 118 pruebas en 14 proyectos.

PASO 4.5.1 — Crear la carpeta del punto 4.5 a partir del punto 4.4

La copia preserva una etapa autónoma y reproducible. 4.4-result-pattern aporta 105 tests aprobados y 13 proyectos antes de añadir este punto.

1. Abre la terminal integrada de Visual Studio y localiza `M04_PATRONES_ARQUITECTURA/PROYECTO/4.4-result-pattern`; esa es la etapa completa que vas a continuar.
2. Crea `M04_PATRONES_ARQUITECTURA/PROYECTO/4.5-inyeccion-dependencias` y copia en ella el contenido de `4.4-result-pattern`; en Windows puedes usar `xcopy` con `/E /I /H /Y`.
3. Excluye `bin`, `obj` y `TestResults`; abre únicamente la copia de `4.5-inyeccion-dependencias` para que el punto anterior permanezca intacto.
4. Ejecuta `dir` dentro de `4.5-inyeccion-dependencias` y verifica `AceriaApp.sln`, `README.md`, `src`, `tests`, `docs` y `global.json`.

Verificación concreta. La copia contiene la solución y los 13 proyectos de la etapa anterior; no contiene todavía archivos nuevos de 4.5.

Diagnóstico si falla.

- Usar `copy` en lugar de `xcopy`: `copy` no copia carpetas de forma recursiva.
- Olvidar el flag `/E`: no se copian las subcarpetas vacías.
- Olvidar el flag `/I`: `xcopy` pregunta si el destino es un directorio cuando no existe.
- Olvidar el flag `/Y`: `xcopy` pide confirmación para sobrescribir.

PASO 4.5.2 — Abrir la solución copiada en Visual Studio 2026

Abrir la solución copiada permite trabajar en el punto 4.5 sobre la base del punto 4.4. La compilación verifica que la copia es correcta.

1. Entra en `4.5-inyeccion-dependencias` y abre `AceriaApp.sln` con Visual Studio; desde Windows puedes ejecutar `start AceriaApp.sln`.
2. Ejecuta `dotnet sln AceriaApp.sln list` y comprueba los 13 proyectos heredados; todos deben existir bajo esta etapa.

3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`; conserva la salida como comprobación inicial.
4. Si la API está en ejecución, detenla antes de compilar. Continúa cuando la compilación termine sin errores y las rutas no apunten a otra etapa.

Verificación concreta. `dotnet sln AceriaApp.sln list` muestra 13 proyectos y `dotnet build AceriaApp.sln -c Release` termina con `Build succeeded.`.

Diagnóstico si falla.

- Abrir la solución del punto 4.4 en lugar de la del punto 4.5: los cambios se aplican al punto equivocado.
- No compilar antes de empezar: si la copia tiene errores, los cambios posteriores fallan.
- Tener archivos DLL bloqueados por una API en ejecución: detén la aplicación antes de recompilar.

PASO 4.5.3 — Verificar que la carpeta `docs` contiene los documentos del punto 4.4

La etapa conserva los 16 documentos anteriores y añade únicamente su ficha de patrón y su ADR. Las modificaciones de código se escriben en la copia actual.

1. Expande `docs` dentro de la copia de `4.5-inyeccion-dependencias`.
2. Comprueba los 16 documentos heredados: los ADR-010 a ADR-017 y las fichas de los puntos anteriores.
3. Compara sus nombres y contenido con `docs` de `4.4-result-pattern`. No modifiques ni elimines los documentos anteriores.
4. Confirma que todavía no existen `docs/inyeccion-dependencias.md` ni `docs/ADR-018-inyeccion-dependencias.md`; se crearán en este punto.

Verificación concreta. `docs` contiene 16 archivos heredados idénticos; los dos documentos de este punto todavía no existen.

Diagnóstico si falla.

- Modificar alguno de los ADR o documentos existentes: rompe la conservación de la etapa anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 4.4.
- Crear la carpeta `docs` si ya existe: `mkdir` falla.

PASO 4.5.4 — Crear el documento de inyección de dependencias en `docs`

El documento de inyección de dependencias es el artefacto de `docs` específico del punto 4.5. Incluye el cierre del módulo 4 con el resumen del estado del proyecto. No modifica los 16 documentos heredados del punto 4.4.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `docs`.
2. Seleccionar `Add` → `New Item` → `Text File` y nombrar el archivo `inyeccion-dependencias.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que el archivo aparece en la carpeta `docs` sin subrayados rojos.

Archivo: `docs/inyeccion-dependencias.md`. **Operación:** Crear.

```
# Inyección de dependencias — Punto 4.5

## Principio de inversión de dependencias

Los módulos de alto nivel y de bajo nivel dependen de abstracciones. Las abstracciones no dependen de los detalles.
```

```
## Inyección de dependencias

Un objeto no crea sus dependencias: las recibe desde el exterior, normalmente por constructor.

## Contenedor de ASP.NET Core

Registra y resuelve las dependencias. Gestiona el ciclo de vida de las instancias.

## Ciclos de vida

- Transient: nueva instancia en cada resolución.
- Scoped: una instancia por ámbito de petición.
- Singleton: una instancia por aplicación.

## Registro por convención

Agrupar registros por capa es una convención de organización. `AddPatronesOrdenes` contiene registros
explícitos de cada puerto, adaptador y caso de uso; no adivina implementaciones por su nombre.

## Registro por ensamblado

`RegisterServicesFromAssemblyContaining<CrearOrdenCommand>()` descubre los manejadores de MediatR por los
contratos que implementan en ese ensamblado. Los demás servicios se registran explícitamente; no se utiliza
el nombre del tipo como regla de descubrimiento.

## Reglas de registro

1. En este flujo se elige una implementación por puerto; el contenedor permite varias cuando se solicita una
colección de servicios.
2. Ciclo adecuado al estado.
3. Inyección por constructor.
4. Sin resolución manual.
5. Sin capturar el contenedor.

## Prohibiciones

1. No convertir los casos de uso en Service Locator; la raíz de composición y los tests sí pueden resolver
servicios para construir o verificar el flujo.
2. No capturar el contenedor.
3. No registrar en tiempo de ejecución.
4. No depender de ciclos más cortos.

## Inyección de dependencias y tests

Las dependencias se sustituyen por dobles en los tests.

## Aplicación al proyecto del curso

| Elemento | Aplicación |
|----------|------------|
| Registro de puertos | Interfaces con adaptadores |
| Ciclo de vida | Scoped para la mayoría |
| Configuración | Métodos de extensión |
| Prohibiciones | Respetadas |
| Validación | Test de coherencia |

## Cierre del módulo 4

El proyecto AceriaApp dispone de:

- Patrón Repository con repositorios específicos.
- Patrón Unit of Work con coordinación de repositorios.
- Patrón Mediator con MediatR.
- Patrón Result para errores esperados.
- Inyección de dependencias con métodos de extensión.
- 118 pruebas acumuladas en 10 proyectos de tests.
- Dieciocho puntos completados (1.1 a 1.5, 2.1 a 2.4, 3.1 a 3.4 y 4.1 a 4.5).

## Correspondencia con el código ejecutable

El catálogo de este punto describe reglas de diseño y contiene ejemplos de contratos. Sus nombres no implican
servicios registrados. La implementación utiliza los proyectos `AceriaApp.Hexagonal.Aplicacion`,
`AceriaApp.Hexagonal.Dominio` y `AceriaApp.Hexagonal.Infraestructura`, compuestos desde `AceriaApp.Api`.
Comprueba los archivos y los tests funcionales que incorpora el paso de compilación de este punto.
```

Verificación concreta. El archivo `docs/inyeccion-dependencias.md` existe y contiene las 12 secciones mostradas; abre su vista previa y comprueba todos los apartados.

Diagnóstico si falla.

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: rompe la conservación de la etapa anterior.
- Guardar el archivo en la raíz del punto en lugar de `docs`: rompe la estructura del repositorio.

PASO 4.5.5 — Crear la carpeta `InyeccionDependencias` en el proyecto API

La carpeta `InyeccionDependencias` albergará las clases de validación del contenedor. Se crea como carpeta nueva sin tocar las carpetas existentes.

1. En el panel Solution Explorer, hacer clic derecho sobre el proyecto `AceriaApp.Api`.
2. Seleccionar `Add` → `New Folder` y nombrar la carpeta `InyeccionDependencias`.
3. Verificar que la carpeta `InyeccionDependencias` aparece dentro de `AceriaApp.Api` sin afectar a las carpetas existentes.
4. Verificar que no se ha modificado ningún archivo existente del punto 4.4.

Verificación concreta. la carpeta `InyeccionDependencias` existe dentro de `AceriaApp.Api` y las carpetas existentes permanecen intactas.

Diagnóstico si falla.

- Crear la carpeta dentro de alguna de las carpetas existentes: rompe la separación de responsabilidades.
- Modificar las carpetas existentes: rompe la conservación de la etapa anterior.
- Crear la carpeta con otro nombre: rompe la convención del curso.

PASO 4.5.6 — Crear la clase `RegistroInyeccion`

`RegistroInyeccion` describe un registro de DI para el catálogo. Sin una llamada real a `AddScoped`, `AddTransient` o `AddSingleton` en la composición, la descripción no hace que ASP.NET Core pueda resolver el servicio.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `InyeccionDependencias`.
2. Seleccionar `Add` → `Class` y nombrar la clase `RegistroInyeccion.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/InyeccionDependencias/RegistroInyeccion.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.InyeccionDependencias;

public class RegistroInyeccion
{
    public string Interfaz { get; init; } = string.Empty;
    public string Implementacion { get; init; } = string.Empty;
    public string Ciclo { get; init; } = string.Empty;
    public bool RegistradoPorConstructor { get; init; }
    public bool ResuelveManualmente { get; init; }
    public bool CapturaContenedor { get; init; }
}
```

Verificación concreta. la clase `RegistroInyeccion` compila sin errores y expone seis propiedades.

Diagnóstico si falla.

- Olvidar el `namespace`: la clase no se resuelve desde otros archivos.

- No inicializar las propiedades de tipo `string` con `string.Empty`: el compilador emite advertencias.
- Usar `set` en lugar de `init`: permite modificar la instancia después de crearla.

PASO 4.5.7 — Crear la clase `ValidadorInyeccionDependencias`

El validador del catálogo distingue vidas útiles y asociaciones potencialmente incompatibles. Esas reglas ayudan a razonar, pero conviene comprobar también la resolución de las implementaciones con el contenedor real.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `InyeccionDependencias`.
2. Seleccionar `Add` → `Class` y nombrar la clase `ValidadorInyeccionDependencias.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/InyeccionDependencias/ValidadorInyeccionDependencias.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.InyeccionDependencias;

public class ValidadorInyeccionDependencias
{
    private static readonly string[] CiclosPermitidos = { "Transient", "Scoped", "Singleton" };

    public ResultadoInyeccion Validar(IEnumerable<RegistroInyeccion> registros)
    {
        var errores = new List<string>();
        var lista = registros.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay registros declarados.");
        }

        foreach (var registro in lista)
        {
            if (string.IsNullOrWhiteSpace(registro.Interfaz))
            {
                errores.Add("Un registro no declara la interfaz.");
            }

            if (string.IsNullOrWhiteSpace(registro.Implementacion))
            {
                errores.Add($"El registro de '{registro.Interfaz}' no declara implementación.");
            }

            if (!CiclosPermitidos.Contains(registro.Ciclo))
            {
                errores.Add($"Ciclo no permitido en '{registro.Interfaz}': '{registro.Ciclo}'.");
            }

            if (!registro.RegistradoPorConstructor)
            {
                errores.Add($"El registro de '{registro.Interfaz}' no se resuelve por constructor.");
            }

            if (registro.ResuelveManualmente)
            {
                errores.Add($"El registro de '{registro.Interfaz}' resuelve manualmente.");
            }

            if (registro.CapturaContenedor)
            {
                errores.Add($"El registro de '{registro.Interfaz}' captura el contenedor.");
            }
        }

        return new ResultadoInyeccion(errores.Count == 0, errores);
    }
}
```

```
public record ResultadoInyeccion(bool EsValido, IReadOnlyList<string> Errores);
```

Verificación concreta. la clase `ValidadorInyeccionDependencias` compila sin errores y expone un método y un `record`.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<string>` e `IReadOnlyList<string>` no se resuelven.
- Olvidar el `using System.Linq;`: `Contains` y `ToList` no se resuelven.
- No comprobar el caso de lista vacía: el validador devuelve válido sin registros.

PASO 4.5.8 — Añadir los `using` necesarios en `ValidadorInyeccionDependencias`

Los tipos `List<string>` e `IReadOnlyList<string>` pertenecen a `System.Collections.Generic`. Los métodos `Contains` y `ToList` pertenecen a `System.Linq`. En proyectos con `ImplicitUsings` desactivado, hay que añadir los usings explícitos.

1. En el panel Solution Explorer, hacer doble clic en `ValidadorInyeccionDependencias.cs`.
2. Inserta las directivas indicadas antes del namespace del archivo; la indicación del archivo identifica la línea exacta.
3. Verificar que no hay subrayados rojos en `List<string>`, `Contains` ni `ToList`.
4. Pulsar `Ctrl+S` para guardar el archivo.

Archivo: `src/AceriaApp.Api/InyeccionDependencias/ValidadorInyeccionDependencias.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
namespace AceriaApp.Api.InyeccionDependencias;
```

```
using System.Collections.Generic;
using System.Linq;
```

Verificación concreta. `ValidadorInyeccionDependencias.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir solo `System.Collections.Generic` y olvidar `System.Linq`: `Contains` y `ToList` no se resuelven.
- Mezclar directivas using con declaraciones de tipos: coloca las directivas antes del namespace para mantener el orden del archivo.
- Olvidar los usings y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 4.5.9 — Crear la clase `CatalogoInyeccion`

La clase `CatalogoInyeccion` contiene los registros por defecto del proyecto. Se usan en los tests y en la validación automática.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `InyeccionDependencias`.
2. Seleccionar `Add` → `Class` y nombrar la clase `CatalogoInyeccion.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `src/AceriaApp.Api/InyeccionDependencias/CatalogoInyeccion.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.InyeccionDependencias;

public class CatalogoInyeccion
{
    public static List<RegistroInyeccion> RegistrosPorDefecto()
    {
        return new List<RegistroInyeccion>
        {
            new() { Interfaz = "IOrdenRepositorio", Implementacion = "OrdenRepositorioSqlServer", Ciclo =
"Scoped", RegistradoPorConstructor = true },
            new() { Interfaz = "IInventarioRepositorio", Implementacion = "InventarioRepositorioSqlServer",
Ciclo = "Scoped", RegistradoPorConstructor = true },
            new() { Interfaz = "IOrdenServicio", Implementacion = "OrdenServicio", Ciclo = "Scoped",
RegistradoPorConstructor = true },
            new() { Interfaz = "IPublicadorEventos", Implementacion = "PublicadorEventosRabbitMq", Ciclo =
"Singleton", RegistradoPorConstructor = true },
            new() { Interfaz = "IUnitOfWork", Implementacion = "UnitOfWorkEfCore", Ciclo = "Scoped",
RegistradoPorConstructor = true }
        };
    }
}
```

Verificación concreta. la clase `CatalogoInyeccion` compila sin errores y expone un método estático.

Diagnóstico si falla.

- Olvidar el `using System.Collections.Generic;`: `List<T>` no se resuelve.
- No inicializar las propiedades de los objetos: el compilador emite advertencias.
- Reutilizar una lista estática mutable: cada llamada del catálogo debe construir una lista independiente.

PASO 4.5.10 — Añadir el `using` necesario en `CatalogoInyeccion`

El tipo `List<T>` pertenece a `System.Collections.Generic`. En proyectos con `ImplicitUsings` desactivado, hay que añadir el using explícito.

1. En el panel Solution Explorer, hacer doble clic en `CatalogoInyeccion.cs`.
2. Inserta las directivas indicadas antes del namespace del archivo; la indicación del archivo identifica la línea exacta.
3. Verificar que no hay subrayados rojos en `List<T>`.
4. Pulsar `Ctrl+S` para guardar el archivo.

Archivo: `src/AceriaApp.Api/InyeccionDependencias/CatalogoInyeccion.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
namespace AceriaApp.Api.InyeccionDependencias;
```

```
using System.Collections.Generic;
```

Verificación concreta. `CatalogoInyeccion.cs` compila sin errores y no muestra subrayados rojos.

Diagnóstico si falla.

- Añadir `using System.Linq;` en lugar de `System.Collections.Generic`: no resuelve `List<T>`.
- Colocar el using después de una clase: las directivas deben preceder las declaraciones de tipos.
- Olvidar el using y confiar en `ImplicitUsings`: el proyecto puede tenerlo desactivado.

PASO 4.5.11 — Crear el proyecto de tests del punto 4.5

El punto 4.5 añade un proyecto de tests específico para las clases de inyección de dependencias. Se crea como proyecto nuevo y conserva las pruebas previas; las actualizaciones explícitas de integración se muestran en su paso correspondiente.

1. En `tests` crea la carpeta `AceriaApp.InyeccionDependencias.Tests` y, dentro, el archivo `AceriaApp.InyeccionDependencias.Tests.csproj` con el XML completo de este paso. En Visual Studio puedes elegir la plantilla xUnit y ajustar el archivo generado.
2. Elimina el archivo de ejemplo `UnitTest1.cs` si lo creó la plantilla; los archivos de pruebas completos se escriben en sus pasos correspondientes.
3. Ejecuta `dotnet sln AceriaApp.sln add tests/AceriaApp.InyeccionDependencias.Tests/AceriaApp.InyeccionDependencias.Tests.csproj` incluso si la solución ya existía y después `dotnet sln AceriaApp.sln list`.
4. Comprueba que el proyecto aparece bajo `tests` y que `dotnet restore tests/AceriaApp.InyeccionDependencias.Tests/AceriaApp.InyeccionDependencias.Tests.csproj` termina correctamente.

Archivo: `tests/AceriaApp.InyeccionDependencias.Tests/AceriaApp.InyeccionDependencias.Tests.csproj`.
Operación: Crear.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. La solución muestra 14 proyectos, incluido `AceriaApp.InyeccionDependencias.Tests`; el nuevo csproj fija .NET 8, SDK de tests 17.12.0, xUnit 2.9.2 y runner 2.8.2.

Diagnóstico si falla.

- Crear el proyecto dentro de `src` en lugar de `tests`: rompe la estructura.
- Modificar los proyectos de tests del punto 4.4: rompe la conservación de la etapa anterior.
- Usar `dotnet new mstest`: el curso usa xUnit.

PASO 4.5.12 — Añadir la referencia del proyecto de tests a la API

La referencia permite que el proyecto de tests acceda a las clases de `InyeccionDependencias` del proyecto API.

1. Ejecuta `dotnet add tests/AceriaApp.InyeccionDependencias.Tests/AceriaApp.InyeccionDependencias.Tests.csproj reference src/AceriaApp.Api/AceriaApp.Api.csproj` o deja el archivo exactamente como el bloque XML de este paso.
2. Comprueba que la referencia va de tests a API; la API nunca debe referenciar el proyecto de tests.
3. Abre `tests/AceriaApp.InyeccionDependencias.Tests/AceriaApp.InyeccionDependencias.Tests.csproj` y comprueba el único `ProjectReference` a `../../src/AceriaApp.Api/AceriaApp.Api.csproj`.

4. Ejecuta `dotnet build tests/AceriaApp.InyeccionDependencias.Tests/AceriaApp.InyeccionDependencias.Tests.csproj -c Release` y verifica que el proyecto resuelve la API y sus dependencias.

Archivo: `tests/AceriaApp.InyeccionDependencias.Tests/AceriaApp.InyeccionDependencias.Tests.csproj`.
Operación: Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. el archivo `AceriaApp.InyeccionDependencias.Tests.csproj` contiene un `ProjectReference` al proyecto API.

Diagnóstico si falla.

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo `.csproj`: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

PASO 4.5.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

`FluentAssertions` proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

1. Ejecuta `dotnet add tests/AceriaApp.InyeccionDependencias.Tests/AceriaApp.InyeccionDependencias.Tests.csproj package FluentAssertions --version 6.12.2` o reemplaza el csproj por el XML completo indicado.
2. Ejecuta `dotnet list tests/AceriaApp.InyeccionDependencias.Tests/AceriaApp.InyeccionDependencias.Tests.csproj package` y localiza `FluentAssertions 6.12.2`.
3. Comprueba que el paquete se añade al nuevo proyecto de tests y que se mantienen la referencia a la API y las versiones de xUnit.
4. Guarda el archivo y ejecuta `dotnet build tests/AceriaApp.InyeccionDependencias.Tests/AceriaApp.InyeccionDependencias.Tests.csproj -c Release`; las aserciones `Should()` estarán disponibles cuando escribas las pruebas.

Archivo: `tests/AceriaApp.InyeccionDependencias.Tests/AceriaApp.InyeccionDependencias.Tests.csproj`.
Operación: Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsTestProject>true</IsTestProject>
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
```

```
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. El archivo de proyecto y `dotnet list package` muestran `FluentAssertions 6.12.2`; la compilación finaliza correctamente.

Diagnóstico si falla.

- Instalar una versión preliminar: el curso usa la versión fijada en el archivo de proyecto.
- Olvidar el paquete: `Should()` no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

PASO 4.5.14 — Crear el archivo `ValidadorInyeccionDependenciasTests.cs`

Las pruebas de validador deben cubrir registros correctos y errores de ámbito. Una condición válida del catálogo no basta para certificar que el DbContext no ha sido capturado por un singleton en ejecución.

1. En el panel Solution Explorer, expandir `tests` → `AceriaApp.InyeccionDependencias.Tests`.
2. Crear el archivo `ValidadorInyeccionDependenciasTests.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `tests/AceriaApp.InyeccionDependencias.Tests/ValidadorInyeccionDependenciasTests.cs`.
Operación: Crear.

```
using AceriaApp.Api.InyeccionDependencias;
using FluentAssertions;
using Xunit;

namespace AceriaApp.InyeccionDependencias.Tests;

public class ValidadorInyeccionDependenciasTests
{
    private readonly ValidadorInyeccionDependencias _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoInyeccion.RegistrosPorDefecto());

        resultado.EsValido.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinRegistros_DevuelveError()
    {
        var resultado = _validador.Validar(new List<RegistroInyeccion>());

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay registros"));
    }

    [Fact]
    public void Validar_CicloNoPermitido_DevuelveError()
    {
        var registros = new List<RegistroInyeccion>
        {
            new() { Interfaz = "IX", Implementacion = "X", Ciclo = "CicloInvalido", RegistradoPorConstructor
= true }
        };

        var resultado = _validador.Validar(registros);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Ciclo no permitido"));
```

```
    }

    [Fact]
    public void Validar_SinRegistroPorConstructor_DevuelveError()
    {
        var registros = new List<RegistroInyeccion>
        {
            new() { Interfaz = "IX", Implementacion = "X", Ciclo = "Scoped", RegistradoPorConstructor =
false }
        };

        var resultado = _validador.Validar(registros);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no se resuelve por constructor"));
    }

    [Fact]
    public void Validar_ResolucionManual_DevuelveError()
    {
        var registros = new List<RegistroInyeccion>
        {
            new() { Interfaz = "IX", Implementacion = "X", Ciclo = "Scoped", RegistradoPorConstructor = true,
ResuelveManualmente = true }
        };

        var resultado = _validador.Validar(registros);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("resuelve manualmente"));
    }

    [Fact]
    public void Validar_CapturaContenedor_DevuelveError()
    {
        var registros = new List<RegistroInyeccion>
        {
            new() { Interfaz = "IX", Implementacion = "X", Ciclo = "Scoped", RegistradoPorConstructor = true,
CapturaContenedor = true }
        };

        var resultado = _validador.Validar(registros);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("captura el contenedor"));
    }
}
```

Verificación concreta. el archivo `ValidadorInyeccionDependenciasTests.cs` compila y contiene seis métodos de test.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.InyeccionDependencias;`: las clases no se resuelven.
- Usar `Assert.True` en lugar de `Should().BeTrue()`: el curso usa FluentAssertions.
- No cubrir el caso de resolución manual ni de captura del contenedor: las prohibiciones no se verifican.

PASO 4.5.15 — Crear el archivo `CatalogoInyeccionTests.cs`

Los tests verifican que el catálogo por defecto contiene los registros esperados y que ninguno viola las prohibiciones del contenedor.

1. En el panel Solution Explorer, hacer clic derecho sobre el proyecto `AceriaApp.InyeccionDependencias.Tests`.
2. Seleccionar `Add` → `Class` y nombrar la clase `CatalogoInyeccionTests.cs`.
3. Escribir exactamente el siguiente código:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que la clase compila sin errores.

Archivo: `tests/AceriaApp.InyeccionDependencias.Tests/CatalogoInyeccionTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.InyeccionDependencias;
using FluentAssertions;
using Xunit;

namespace AceriaApp.InyeccionDependencias.Tests;

public class CatalogoInyeccionTests
{
    [Fact]
    public void RegistrosPorDefecto_ContieneCincoRegistros()
    {
        var registros = CatalogoInyeccion.RegistrosPorDefecto();

        registros.Should().HaveCount(5);
    }

    [Fact]
    public void RegistrosPorDefecto_TodosResueltosPorConstructor()
    {
        var registros = CatalogoInyeccion.RegistrosPorDefecto();

        registros.Should().OnlyContain(r => r.RegistradoPorConstructor);
    }

    [Fact]
    public void RegistrosPorDefecto_NingunoResuelveManualmente()
    {
        var registros = CatalogoInyeccion.RegistrosPorDefecto();

        registros.Should().OnlyContain(r => !r.ResuelveManualmente);
    }

    [Fact]
    public void RegistrosPorDefecto_NingunoCapturaContenedor()
    {
        var registros = CatalogoInyeccion.RegistrosPorDefecto();

        registros.Should().OnlyContain(r => !r.CapturaContenedor);
    }
}
```

Verificación concreta. el archivo `CatalogoInyeccionTests.cs` compila y contiene cuatro métodos de test.

Diagnóstico si falla.

- Olvidar el `using AceriaApp.Api.InyeccionDependencias;`: las clases no se resuelven.
- Olvidar el `using FluentAssertions;`: `OnlyContain` no se resuelve.
- No verificar las prohibiciones: el catálogo podría violarlas.

PASO 4.5.16 — Componer los servicios y comprobar sus ámbitos antes de compilar

Componer los servicios exige asignar correctamente puertos, implementaciones y ámbitos. Comprueba la resolución de servicios en una petición real, la persistencia en SQL Server y que la instancia Scoped no se utilice fuera de su ciclo de vida.

1. Abre cada archivo de integración funcional indicado en este paso. Crea los nuevos y reemplaza únicamente los que se señalen; copia todo el código mostrado.
2. Recorre las referencias: el dominio define sus entidades, la aplicación usa puertos y la infraestructura implementa persistencia; la API compone el flujo.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`; comprueba que todos los proyectos nuevos estén incluidos en la solución.

4. Verifica `Build succeeded.` y `0 Error(s)`. Si hay advertencias, revisa su causa; en el paso siguiente ejecutarás las pruebas del comportamiento completo.

Integración funcional. Integra únicamente registros y mapeo del endpoint de patrones. Conserva íntegros los endpoints, documentación y arranque de M2/M3; en 4.5 reemplaza registros repetidos por AddPatronesOrdenes().

Archivo: `src/AceriaApp.Api/Program.cs`. **Operación:** Reemplazar el contenido completo.

```
using AceriaApp.Api.Patrones;
using AceriaApp.Hexagonal.Infraestructura.Patrones;
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Validacion;
using FluentValidation;
using AceriaApp.Api.Documentacion;

using AceriaApp.Api.Hexagonal;
using AceriaApp.Hexagonal.Infraestructura;
using AceriaApp.Hexagonal.Infraestructura.Datos;
var builder=WebApplication.CreateBuilder(args);
// INICIO PASO 2.1.12: SQL Server Express LocalDB, no SQLite.
builder.Services.AddDbContext<AceriaDbContext>(o=>
    o.UseSqlServer(builder.Configuration.GetConnectionString("AceriaApp")));
// PASO 2.3.14: registrar validadores en el contenedor.
builder.Services.AddValidatorsFromAssemblyContaining<CrearOrdenValidator>(); // PASO 2.3.14
// PASO 2.4.6: registrar generador antes de Build.
builder.Services.AddDocumentacion(); // PASO 2.4.6

builder.Services.AddServiciosHexagonales(builder.Configuration);
builder.Services.AddPatronesOrdenes();
var app=builder.Build();
// EnsureCreated se utiliza SOLO como ayuda didáctica en Development.
// Para un sistema productivo se utilizarían migraciones controladas.
// PASO 2.1.13: inicializar LocalDB solo en Development.
if(app.Environment.IsDevelopment()&&!app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope=app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<AceriaDbContext>().Database.EnsureCreatedAsync();
}

// INICIO PASO 2.1.14: colección de recursos, consulta real con EF Core.
app.MapGet("/api/v1/ordenes",async (AceriaDbContext db,CancellationToken ct)=>
{
    // PASO 2.1.15: lectura sin tracking y proyeccion a DTO.
    var ordenes=await db.Ordenes.AsNoTracking().OrderBy(x=>x.FechaCreacion).ToListAsync(ct);
    return Results.Ok(ordenes.Select(OrdenDto.Desde));
})
.WithName("ListarOrdenes").WithTags("Ordenes")
.WithSummary("Lista las órdenes de producción").Produces<IEnumerable<OrdenDto>>(200)
;
// INICIO PASO 2.1.17: recurso individual.
app.MapGet("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(x=>x.Id==id);
    // PASO 2.1.18: 200 para orden presente; 404 si no existe.
    IResult response=orden is null ? Results.NotFound() : Results.Ok(OrdenDto.Desde(orden));
    return response;
})
.WithName("ObtenerOrdenPorId").WithTags("Ordenes")
.WithSummary("Obtiene una orden por ID").Produces<OrdenDto>(200).Produces(404)
;


// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    // PASO 2.2.6: crear entidad y persistirla.
```

```
    var orden=new Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    // PASO 2.2.7: devolver 201 con Location.
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")
// PASO 2.3.15: filtro de POST.
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Crea una orden").Produces<OrdenDto>(201).ProducesValidationProblem(400).ProducesProblem(422)
;


// INICIO PASO 2.2.11: sustitución completa de los campos editables.
app.MapPut("/api/v1/ordenes/{id:guid}",async (Guid id,CrearOrdenRequest request,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cliente=request.Cliente;orden.Material=request.Material;
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("ReemplazarOrden").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Sustituye campos
editables").Produces(204).Produces(404).ProducesProblem(409).ProducesValidationProblem(400).ProducesProblem(4
22)
;


// INICIO PASO 2.2.14: cambiar SOLO la cantidad.
app.MapPatch("/api/v1/ordenes/{id:guid}/cantidad",
    async (Guid id,ActualizarCantidadRequest request,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("CambiarCantidad").WithTags("Ordenes")
// PASO 2.3.17: filtro de PATCH.
.AddEndpointFilter<ValidationFilter<ActualizarCantidadRequest>>()
.WithSummary("Modifica
cantidad").Produces<OrdenDto>(200).Produces(404).ProducesValidationProblem(400).ProducesProblem(409).Produces
Problem(422)
;


// INICIO PASO 2.2.16: eliminar recurso persistido.
app.MapDelete("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede eliminar una orden finalizada.",statusCode:409);
    db.Ordenes.Remove(orden);
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("EliminarOrden").WithTags("Ordenes")
.WithSummary("Elimina una orden pendiente").Produces(204).Produces(404).ProducesProblem(409)
;
// INICIO PASO 2.2.18: transición explícita del dominio.
app.MapPost("/api/v1/ordenes/{id:guid}/finalizar",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
```

```
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"La orden ya estaba finalizada.",statusCode:409);
    orden.Estado="Finalizada";
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("FinalizarOrden").WithTags("Ordenes")
.WithSummary("Finaliza una orden").Produces<OrdenDto>(200).Produces(404).ProducesProblem(409)
;

// PASO 2.4.8: habilitar Swagger solo en Development.
app.UseDocumentacion(); // PASO 2.4.8
// El contexto hexagonal usa su propia base para que EnsureCreated no dependa de
// las tablas heredadas del curso M02.
if (app.Environment.IsDevelopment() && !app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope = app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<HexagonalDbContext>()
        .Database.EnsureCreatedAsync();
}
app.MapHexagonalOrdenes();
app.MapPatronesOrdenes();
app.Run();
public partial class Program {} // permite WebApplicationFactory en tests
```

Permite mapear los endpoints del módulo desde la raíz de composición. Permite registrar las implementaciones de los puertos desde la API. Mantiene el registro SQL Server y los servicios que ya utilizaba M03.

Sustituye los registros repetidos del flujo por un método de extensión; se conserva el registro previo del contexto. Publica las rutas del nuevo flujo después de construir la aplicación y antes de ejecutarla.

Integración funcional. Centraliza registros del flujo real. Requiere HexagonalDbContext registrado por AddServiciosHexagonales; repositorio, UoW, casos de uso y servicios MediatR se resuelven dentro del mismo ámbito. El pipeline abierto no captura dependencias scoped en un singleton.

Archivo: `src/AceriaApp.Hexagonal.Infraestructura/Patrones/ServiciosPatrones.cs`. **Operación:** Crear.

```
using AceriaApp.Hexagonal.Aplicacion.Patrones;
using Microsoft.Extensions.DependencyInjection;
namespace AceriaApp.Hexagonal.Infraestructura.Patrones;

public static class ServiciosPatrones
{
    /// <summary>Requiere HexagonalDbContext registrado; cada petición comparte contexto, repositorio y
UoW.</summary>
    public static IServiceCollection AddPatronesOrdenes(this IServiceCollection services)
    {
        services.AddLogging();
        services.AddScoped<IOrdenTrabajoRepositorio, OrdenTrabajoRepositorio>();
        services.AddScoped<IUnitOfWork, EfUnitOfWork>();
        services.AddScoped<OrdenesTransaccionales>();
        services.AddScoped<OrdenesConResultado>();
        services.AddMediatR(cfg =>
        {
            cfg.RegisterServicesFromAssemblyContaining<CrearOrdenCommand>();
            cfg.Lifetime = ServiceLifetime.Scoped;
            cfg.AddOpenBehavior(typeof(LoggingBehavior<,>));
        });
        return services;
    }
}
```

Agrupa los registros del flujo. Antes debe registrarse HexagonalDbContext mediante AddServiciosHexagonales. Proporciona ILogger al pipeline de MediatR. Reutiliza el repositorio dentro del ámbito de petición y permite sustituir el puerto en tests.

Registra la unidad de trabajo con el mismo ciclo Scoped del contexto; las resoluciones del ámbito comparten la persistencia. Registra el caso de uso que separa preparación y confirmación. Registra el caso de uso que devuelve éxito o errores esperados.

Descubre los manejadores en el ensamblado de la capa de aplicación. Evita capturar un servicio del ámbito desde un manejador de vida más larga. Añade el comportamiento genérico de logging alrededor de todos los manejadores compatibles.

Integración funcional. Conserva las dependencias de los tests de catálogo y añade Testcontainers.MsSql, el helper compartido y WebApplicationFactory para las pruebas funcionales.

Archivo: `tests/AceriaApp.InyeccionDependencias.Tests/AceriaApp.InyeccionDependencias.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings><IsPackable>false</IsPackable><IsTestProject>true</IsTestProject></PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="Testcontainers.MsSql" Version="3.10.0" />
    <PackageReference Include="Microsoft.AspNetCore.Mvc.Testing" Version="8.0.21" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <Compile Include="../Compartido/SqlServerPruebas.cs" Link="SqlServerPruebas.cs" />
  </ItemGroup>
</Project>
```

Compila para .NET 8, igual que los proyectos heredados. Fija el cliente de contenedores para SQL Server; el proveedor EF Core SQL Server llega mediante la referencia a la API. Permite ejecutar peticiones HTTP contra el host real de la API.

Compila el helper heredado que crea y elimina únicamente la base aislada de cada prueba, sin añadir otro proyecto a la solución.

Integración funcional. Comprueba aislamiento entre ámbitos, ejecución del flujo con la extensión real y rechazo de resolución scoped desde raíz. Añade un singleton cautivo solo en la prueba y verifica que ValidateScopes junto a ValidateOnBuild lo detectan.

Archivo: `tests/AceriaApp.InyeccionDependencias.Tests/InyeccionFuncionalesTests.cs`. **Operación:** Crear.

```
using AceriaApp.Hexagonal.Aplicacion.Patrones;
using AceriaApp.Hexagonal.Dominio;
using AceriaApp.Hexagonal.Infraestructura.Datos;
using AceriaApp.Hexagonal.Infraestructura.Patrones;
using MediatR;
using AceriaApp.PruebasSqlServer;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Xunit;
namespace AceriaApp.InyeccionDependencias.Tests;

public sealed class InyeccionFuncionalesTests
{
    [Fact]
    public async Task Ambitos_distintos_no_confirman_cambios_pendientes_del_otro()
    {
        await using var store = BaseSqlServer.Crear();
        await using var provider = CrearServicios(store.ConnectionString).BuildServiceProvider(Opciones());
        using var primero = provider.CreateScope(); using var segundo = provider.CreateScope();
        var db = primero.ServiceProvider.GetRequiredService<HexagonalDbContext>();
        Assert.Equal("Microsoft.EntityFrameworkCore.SqlServer", db.Database.ProviderName);
        await db.Database.EnsureCreatedAsync();
        var repo = primero.ServiceProvider.GetRequiredService<IOrdenTrabajoRepositorio>();
        Assert.Same(repo, primero.ServiceProvider.GetRequiredService<IOrdenTrabajoRepositorio>());
```

```
        Assert.NotSame(repo, segundo.ServiceProvider.GetRequiredService<IOrdenTrabajoRepositorio>());
        await repo.AgregarAsync(new OrdenHexagonal("Axia", "Acero", 20));
        Assert.Equal(0, await segundo.ServiceProvider.GetRequiredService<IUnitOfWork>().CommitAsync());
        Assert.Equal(1, await primero.ServiceProvider.GetRequiredService<IUnitOfWork>().CommitAsync());
        Assert.Equal(1, await
segundo.ServiceProvider.GetRequiredService<HexagonalDbContext>().Ordenes.CountAsync());
    }

    [Fact]
    public async Task Extension_resuelve_flujo_completo_en_un_ambito_valido()
    {
        await using var store = BaseSqlServer.Crear();
        await using var provider = CrearServicios(store.ConnectionString).BuildServiceProvider(Opciones());
        using var scope = provider.CreateScope();
        var db = scope.ServiceProvider.GetRequiredService<HexagonalDbContext>();
        Assert.Equal("Microsoft.EntityFrameworkCore.SqlServer", db.Database.ProviderName);
        await db.Database.EnsureCreatedAsync();
        var sender = scope.ServiceProvider.GetRequiredService<ISender>();
        var creada = await sender.Send(new CrearOrdenCommand("Axia", "Acero", 20));
        Assert.True(creada.EsExito);
        var resultado = await sender.Send(new FinalizarOrdenCommand(creada.Valor.Id));
        Assert.True(resultado.EsExito); Assert.Equal("Finalizada", resultado.Valor.Estado);
        Assert.Throws<InvalidOperationException>(() => provider.GetRequiredService<IUnitOfWork>());
    }

    [Fact]
    public void ValidateScopes_detecta_singleton_que_captura_unidad_de_trabajo_scoped()
    {
        var services = CrearServicios("Server=localhost;Database=ValidacionDi;Integrated
Security=True;TrustServerCertificate=True");
        services.AddSingleton<ConsumidorCautivo>(); // Registro incorrecto solo en esta prueba.
        var error = Assert.Throws<AggregateException>(() => services.BuildServiceProvider(Opciones()));
        Assert.Contains(nameof(IUnitOfWork), error.ToString());
    }

    private static ServiceProviderOptions Opciones() => new() { ValidateOnBuild = true, ValidateScopes =
true };
    private static IServiceCollection CrearServicios(string connectionString)
    {
        var services = new ServiceCollection();
        services.AddDbContext<HexagonalDbContext>(o => o.UseSqlServer(connectionString));
        services.AddPatronesOrdenes();
        return services;
    }
    private sealed class ConsumidorCautivo(IUnitOfWork trabajo)
    {
        public IUnitOfWork Trabajo { get; } = trabajo;
    }
}
```

Dos ámbitos comparten la base de la prueba, pero tienen contextos y cambios pendientes independientes. Exige el proveedor SQL Server antes de crear y consultar tablas. Confirmar el segundo ámbito no escribe los cambios preparados en el primero.

Comprueba el flujo completo del mediador dentro de un ámbito válido. La validación de DI no abre conexiones: la cadena solo permite construir las opciones del contexto. Introduce deliberadamente una dependencia Scoped cautiva para verificar el diagnóstico del contenedor.

Los contextos mantienen ciclo Scoped y el proveedor SQL Server.

Verificación concreta. La solución completa compila, incluidos el código de integración y sus pruebas. Las siguientes ejecuciones de tests comprobarán el comportamiento de los patrones.

Diagnóstico si falla.

- Tener la API en ejecución: el archivo `.dll` está bloqueado y la compilación falla.
- No restaurar paquetes: `dotnet build` restaura por defecto, pero si hay un error de red, hay que ejecutar `dotnet restore` primero.
- Usar una versión distinta del SDK: `dotnet --version` debe devolver `8.x.x`.

PASO 4.5.17 — Ejecutar todos los tests del punto 4.5

Ejecutar los tests del punto 4.5 verifica que las clases de inyección de dependencias funcionan correctamente.

1. Ejecuta `dotnet test tests/AceriaApp.InyeccionDependencias.Tests/AceriaApp.InyeccionDependencias.Tests.csproj -c Release` desde la raíz de esta etapa.
2. Comprueba un total de 13 pruebas: 10 de catálogo y 3 funcionales, con cero fallos y cero omisiones.
3. Si falla una prueba, abre su mensaje de aserción y el archivo indicado; corrige el comportamiento antes de continuar.
4. Repite el mismo comando tras la corrección y conserva el resultado de la ejecución válida.

Verificación concreta. El proyecto `AceriaApp.InyeccionDependencias.Tests` termina con `Failed: 0, Passed: 13, Skipped: 0, Total: 13`.

Diagnóstico si falla.

- Ejecutar otro proyecto: comprueba la ruta y el nombre de la asamblea de pruebas antes de interpretar los resultados.
- No compilar antes: `dotnet test` compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

PASO 4.5.18 — Ejecutar todos los tests de la solución

Ejecutar todos los tests de la solución verifica que los tests de los puntos anteriores y del punto 4.5 pasan conjuntamente.

1. Ejecuta `dotnet build AceriaApp.sln -c Release` para producir todas las asambleas de esta etapa.
2. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build --no-restore` para ejecutar todas las pruebas acumuladas.
3. Suma los resultados de los 10 proyectos de tests: deben ser 118 pruebas aprobadas, sin fallos ni omisiones.
4. Comprueba que las pruebas heredadas de M03 y los puntos anteriores de M04 siguen presentes; no borres tests para obtener un resultado verde.

Verificación concreta. La suma de las líneas de resultados es `Failed: 0`, `Passed: 118`, `Skipped: 0` y `Total: 118`. Dotnet muestra un resumen por proyecto, no necesariamente una única línea global.

Diagnóstico si falla.

- No haber compilado antes: `--no-build` falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
- Eliminar pruebas anteriores o cambiarlas sin una instrucción explícita: se pierde la cobertura de los contratos heredados.

PASO 4.5.19 — Crear el README del punto 4.5

El README del punto 4.5 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 4.4.

1. En la carpeta del punto `4.5-inyeccion-dependencias`, hacer clic derecho y seleccionar `Add` → `New Item` → `Text File`.
2. Nombrar el archivo `README.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que el README del punto 4.4 no se ha modificado.

Archivo: `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 4.5 — Inyección de dependencias

## Objetivo
Comprender la inyección de dependencias y añadir al proyecto las clases de validación de registros del
contenedor. Cerrar el módulo 4 con el resumen del estado del proyecto.

La práctica conecta además el patrón con los puertos, adaptadores y pruebas de integración que aparecen en
este manual.

## Estado del proyecto
- Solución `AceriaApp.sln` con 14 proyectos.
- Documento `docs/inyeccion-dependencias.md` con las reglas y el cierre del módulo 4.
- Clases `RegistroInyeccion`, `ValidadorInyeccionDependencias` y `CatalogoInyeccion` en `src/AceriaApp.Api/
InyeccionDependencias/`.
- 10 tests nuevos de catálogo y 3 funcionales en `tests/AceriaApp.InyeccionDependencias.Tests`.
- Los tests acumulados suman 118 pruebas: 10 de catálogo nuevas y 3 funcionales nuevas en este punto.

## Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio 2026.
2. Verificar que SQL Server Express está arrancado.
3. Configurar las conexiones SQL Server `AceriaApp` y `AceriaHexagonal` en el entorno local.
4. Ejecutar `dotnet run --project src/AceriaApp.Api/AceriaApp.Api.csproj --urls http://localhost:5000` desde
la raíz de esta etapa.
5. Navegar a `http://localhost:5000/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

## Qué se ha añadido respecto al punto anterior
- Documento `docs/inyeccion-dependencias.md`.
- Carpeta `InyeccionDependencias` en el proyecto API con tres clases nuevas.
- Proyecto `tests/AceriaApp.InyeccionDependencias.Tests` con 10 tests de catálogo y 3 funcionales nuevos.
- Cierre del módulo 4 con el resumen del estado del proyecto.
- Se conserva intacta la carpeta del punto 4.4.

## Documentación
- [ADR-010](docs/ADR-010-capas-aplicacion.md) a [ADR-017](docs/ADR-017-patron-result.md) (heredados).
- [Inyección de dependencias](docs/inyeccion-dependencias.md).

## Verificación ejecutable

Los catálogos describen ejemplos de diseño. El comportamiento de las clases reales se comprueba además con
los tests funcionales del punto. Ejecuta `dotnet test AceriaApp.sln -c Release` y exige cero fallos y cero
omisiones.
```

Verificación concreta. El archivo `README.md` existe y contiene las 6 secciones mostradas; abre su vista previa y comprueba todos los apartados.

Diagnóstico si falla.

- Modificar el README del punto 4.4: rompe la conservación de la etapa anterior.
- Omitir la sección `Qué se ha añadido respecto al punto anterior`: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 4.4: rompe la separación.

PASO 4.5.20 — Crear el ADR-018 del punto 4.5

El ADR-018 documenta la decisión de adoptar la inyección de dependencias y de cerrar el módulo 4. Es un artefacto nuevo del punto 4.5 que no modifica los ADR existentes.

1. En el panel Solution Explorer, hacer clic derecho sobre la carpeta `docs`.
2. Seleccionar `Add` → `New Item` → `Text File` y nombrar el archivo `ADR-018-inyeccion-dependencias.md`.
3. Escribir exactamente el siguiente contenido:
4. Pulsar `Ctrl+S` para guardar el archivo. Verificar que los ADR-010 a ADR-017 no se han modificado.

Archivo: `docs/ADR-018-inyeccion-dependencias.md`. **Operación:** Crear.

```
# ADR-018: Inyección de dependencias y cierre del módulo 4

## Estado
Aceptado
```

```
## Contexto
El proyecto AceriaApp ha adoptado los patrones Repository, Unit of Work, Mediator y Result. Es necesario
definir cómo se componen los servicios en el contenedor de dependencias y cerrar el módulo 4.

## Decisión
Se adopta el contenedor de ASP.NET Core. `AddServiciosHexagonales(configuration)` registra el contexto SQL
Server heredado y `AddPatronesOrdenes()` agrupa registros explícitos de `IOrdenTrabajoRepositorio`,
`IUnitOfWork`, los casos de uso y el pipeline. Los manejadores se descubren por sus interfaces con
`RegisterServicesFromAssemblyContaining`. Los componentes que comparten `HexagonalDbContext` tienen ciclo
Scoped; MediatR también se configura con ese ciclo. Tres pruebas verifican la identidad dentro del ámbito, el
aislamiento entre ámbitos, el flujo completo y el rechazo de una dependencia Scoped capturada por un
Singleton. El catálogo de cinco registros sigue siendo un ejercicio de diseño y no afirma que exista un
publicador RabbitMQ en la aplicación.

## Alternativas consideradas
- Uso de un contenedor externo: descartado por simplicidad del proyecto.
- Registro directo en `Program.cs`: descartado por crecimiento del archivo.
- Uso de Service Locator: descartado por acoplamiento.

## Consecuencias
- Positivas: composición declarativa, ciclo de vida gestionado, servicios testeables.
- Negativas: el contenedor añade indirección en la resolución.
- Mitigación: validación automática y uso consistente del patrón.
```

Verificación concreta. El archivo `docs/ADR-018-inyeccion-dependencias.md` existe y contiene las 5 secciones mostradas; abre su vista previa y comprueba todos los apartados.

Diagnóstico si falla.

- Modificar los ADR anteriores: rompe la conservación de la etapa anterior.
- Omitir la sección `Alternativas consideradas`: el ADR pierde valor.
- Nombrar el archivo sin el prefijo `ADR-018`: rompe la convención.

PASO 4.5.21 — Verificar que los documentos heredados y la etapa 4.4 permanecen intactos

Las decisiones previas se conservan como registros históricos. Los nuevos ADR explican la evolución y las modificaciones de integración se aplican solo a la copia de la etapa actual.

1. Vuelve a la raíz del repositorio; las rutas siguientes se interpretan desde allí.
2. En CMD ejecuta `for %f in (M04_PATRONES_ARQUITECTURA\PROYECTO\4.4-result-pattern\docs\*.md) do fc /L "%f" "M04_PATRONES_ARQUITECTURA\PROYECTO\4.5-inyeccion-dependencias\docs\%~nxf"`. En PowerShell puedes comparar `Get-FileHash` para cada par de documentos.
3. Comprueba 16 comparaciones y exige igualdad en todas; se conserva exactamente el contenido de los documentos heredados.
4. Si una comparación falla, restaura el documento de la etapa actual desde `4.4-result-pattern`. Verifica también que la carpeta del punto anterior no tiene modificaciones.

Verificación concreta. Los 16 documentos heredados coinciden byte a byte y el punto anterior permanece intacto.

Diagnóstico si falla.

- Usar `comp` en lugar de `fc`: `comp` compara binarios y no muestra diferencias de texto.
- Omitir `/L` cuando se desea una comparación de texto: indica el modo explícitamente y revisa cada resultado.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

PASO 4.5.22 — Verificar la estructura del punto 4.5

La verificación estructural confirma que el punto 4.5 añade archivos nuevos conservando los documentos heredados.

1. Escribir `tree /F M04_PATRONES_ARQUITECTURA\PROYECTO\4.5-inyeccion-dependencias` y pulsar Enter.
2. Verificar que la salida muestra `AceriaApp.sln`, `README.md`, `src\AceriaApp.Api\InyeccionDependencias` con las tres clases nuevas, `tests\AceriaApp.InyeccionDependencias.Tests` con los dos archivos de catálogo y el archivo de tests funcionales y `docs` con el documento de inyección y el ADR-018. Comprueba además todos los archivos de integración de las carpetas Patrones y los cambios de composición indicados en este punto.
3. Verificar que las carpetas existentes del punto 4.4 permanecen intactas.
4. Verificar que no existe ninguna carpeta `labs` ni ningún archivo `metaprompt.md` ni `verificacion.md`.

Verificación concreta. la estructura contiene los archivos nuevos y conserva los del punto 4.4.

Diagnóstico si falla.

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 4.4: viola la regla de no modificar.
- Omitir la carpeta `InyeccionDependencias`: los tests no encuentran las clases.

PASO 4.5.23 — Verificar el `.gitignore` del punto 4.5

El `.gitignore` se hereda del punto 4.4 sin modificaciones. La verificación confirma que no se ha tocado.

1. Verificar que el archivo `.gitignore` existe en la carpeta del punto 4.5 (copiado del punto 4.4).
2. Abrir el archivo `.gitignore` con Visual Studio 2026.
3. Verificar que contiene las líneas `**/bin/`, `**/obj/`, `.vs/`, `*.user`, `TestResults/` y `*.trx`.
4. Verificar que no se ha modificado respecto al punto 4.4. Pulsar `Ctrl+S` si no hay cambios y cerrar el archivo.

Verificación concreta. el `.gitignore` existe y contiene las seis líneas sin modificaciones.

Diagnóstico si falla.

- Modificar el `.gitignore` copiado del punto 4.4: rompe la conservación de la etapa anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.
- Crear el `.gitignore` con otro nombre: el archivo no se reconoce.

PASO 4.5.24 — Hacer el commit del punto 4.5 y verificar el estado final del módulo 4

El commit permite comparar esta etapa con la anterior. Si has realizado un commit por punto desde 1.1, el recorrido suma 18 commits de puntos; otros commits legítimos pueden aumentar el historial.

1. Desde la raíz del repositorio ejecuta `git status` y revisa que solo aparezcan los archivos de esta etapa y los entregables que has trabajado.
2. Ejecuta `git add .` después de verificar que `bin/`, `obj/`, `TestResults/` y credenciales están excluidos.
3. Ejecuta `git commit -m "Punto 4.5: Inyección de dependencias"` con `user.name` y `user.email` configurados.
4. Ejecuta `git log --oneline` y `git status`; comprueba que se registró el commit de este punto y que no quedan cambios pendientes de esta actividad.

Verificación concreta. El historial incluye el commit de 4.5 y `git status` no muestra cambios pendientes de esta actividad; no se exige un número rígido de commits a un repositorio con más trabajo.

Diagnóstico si falla.

- No tener configurado `user.name` y `user.email`: `git commit` falla.
- Añadir la carpeta `bin/` y `obj/` al commit: el `.gitignore` lo evita.
- Hacer commit antes de `git add .`: no hay cambios en el índice.

Reto de ampliación (optativo)

Trabaja en una copia independiente de la etapa obligatoria, por ejemplo `4.5-inyeccion-dependencias-reto`. Mantén intacta la carpeta obligatoria y todos sus documentos. Los tres fragmentos siguientes añaden un contrato, un cálculo y una prueba al proyecto del punto.

Reto: añadir un método `Resumen` a `ValidadorInyeccionDependencias` que devuelva el número total de registros, el número de registros con ciclo `Scoped`, el número con ciclo `Singleton` y el número con ciclo `Transient`, y un test que verifique el cálculo con el catálogo por defecto.

Paso 1: En `ValidadorInyeccionDependencias.cs`, añadir el siguiente `record` al final del archivo:

Archivo: `src/AceriaApp.Api/InyeccionDependencias/ValidadorInyeccionDependencias.cs`. **Operación:** Modificar.

Punto de inserción: después de este texto exacto, que debe aparecer una sola vez:

```
public record ResultadoInyeccion(bool EsValido, IReadOnlyList<string> Errores);
```

```
public record ResumenInyeccion(
    int Total,
    int Scoped,
    int Singleton,
    int Transient);
```

Paso 2: Explicar el propósito del fragmento y las decisiones que contiene:

La declaración `public record ResumenInyeccion(` declara un contrato inmutable. Número total de registros. Número de registros con ciclo Scoped. Número de registros con ciclo Singleton. Número de registros con ciclo Transient.

Paso 3: En `ValidadorInyeccionDependencias.cs`, añadir el siguiente método estático dentro de la clase:

Archivo: `src/AceriaApp.Api/InyeccionDependencias/ValidadorInyeccionDependencias.cs`. **Operación:** Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
}

public record ResultadoInyeccion(bool EsValido, IReadOnlyList<string> Errores);
```

Indenta el fragmento con cuatro espacios dentro de la clase; conserva el cierre de clase que aparece en el texto de inserción.

```
public static ResumenInyeccion Resumen(IEnumerable<RegistroInyeccion> registros)
{
    var lista = registros.ToList();
    var scoped = lista.Count(r => r.Ciclo == "Scoped");
    var singleton = lista.Count(r => r.Ciclo == "Singleton");
    var transient = lista.Count(r => r.Ciclo == "Transient");
    return new ResumenInyeccion(lista.Count, scoped, singleton, transient);
}
```

Paso 4: Explicar el propósito del fragmento y las decisiones que contiene:

La declaración `public static ResumenInyeccion Resumen(IEnumerable<RegistroInyeccion> re` declara el método. Materializa la lista. Cuenta los Scoped. Cuenta los Singleton. Cuenta los Transient. Devuelve el resumen.

Paso 5: Guardar el archivo con `Ctrl+S`.

Paso 6: En `ValidadorInyeccionDependenciasTests.cs`, añadir el siguiente test:

Archivo: `tests/AceriaApp.InyeccionDependencias.Tests/ValidadorInyeccionDependenciasTests.cs`.
Operación: Modificar.

Punto de inserción: antes de este texto exacto, que debe aparecer una sola vez:

```
}
```

Indenta el fragmento con cuatro espacios dentro de la clase; conserva el cierre de clase que aparece en el texto de inserción.

```
[Fact]
public void Resumen_ConCatalogoPorDefecto_CuentaPorCiclo()
{
    var resumen = ValidadorInyeccionDependencias.Resumen(CatalogoInyeccion.RegistrosPorDefecto());

    resumen.Total.Should().Be(5);
    resumen.Scoped.Should().Be(4);
    resumen.Singleton.Should().Be(1);
    resumen.Transient.Should().Be(0);
}
```

Paso 7: Interpretar el test y explicar qué comportamiento acredita:

Marca el método como test de xUnit. Declara el test. Invoca el método. Verifica el total. Verifica los Scoped. Verifica los Singleton. Verifica los Transient.

Paso 8: Ejecuta `dotnet test tests/AceriaApp.InyeccionDependencias.Tests/AceriaApp.InyeccionDependencias.Tests.csproj -c Release` en la copia optativa. Deben pasar 14 tests del proyecto, incluido el nuevo resumen. Ejecuta después `dotnet test AceriaApp.sln -c Release` y exige 119 pruebas aprobadas.

Paso 9: Comprueba que solo cambian `ValidadorInyeccionDependencias.cs` y `ValidadorInyeccionDependenciasTests.cs` en la copia optativa; la etapa obligatoria `4.5-inyeccion-dependencias` y el punto anterior deben permanecer idénticos.

Paso 10: Hacer commit con `git commit -m "Reto 4.5: método Resumen en ValidadorInyeccionDependencias"`.

Verificación: `Passed: 14` en el proyecto del punto y 119 pruebas aprobadas en la solución de la copia optativa, sin fallos ni omisiones.

Resultado verificable

Al finalizar este punto, el alumno debe tener:

- La carpeta `M04_PATRONES_ARQUITECTURA\PROYECTO/4.5-inyeccion-dependencias/` con la solución `AceriaApp.sln` que contiene 14 proyectos.
- El documento `docs/inyeccion-dependencias.md` con las reglas, las prohibiciones y el cierre del módulo 4.

- El documento `docs/ADR-018-inyeccion-dependencias.md` con la decisión sobre la inyección de dependencias.
- Las clases `RegistroInyeccion`, `ValidadorInyeccionDependencias` y `CatalogoInyeccion` en `src/AceriaApp.Api/InyeccionDependencias/`.
- El proyecto `tests/AceriaApp.InyeccionDependencias.Tests` añade 10 tests de catálogo y 3 funcionales; el reto añade uno más.
- `dotnet test AceriaApp.sln -c Release` ejecuta 118 pruebas en la etapa obligatoria y 119 en una copia con su reto; ninguna falla ni se omite.
- Los ADR-010 a ADR-017 del punto 4.4 sin modificar.
- Un commit revisado para el punto 4.5, con el historial previo conservado.
- El módulo 4 cerrado con el resumen del estado del proyecto en `docs/inyeccion-dependencias.md`.

Continuidad del aprendizaje

El punto 4.5 ha cerrado el módulo 4 con la adopción de la inyección de dependencias como mecanismo para aplicar el principio de inversión de dependencias. El proyecto AceriaApp dispone ahora de los cuatro patrones del módulo 4 —Repository, Unit of Work, Mediator y Result— además de la inyección de dependencias que los compone. El módulo 4 queda cerrado con el resumen del estado del proyecto en `docs/inyeccion-dependencias.md`. Esta base permite avanzar hacia el módulo 5, donde se estudiará la introducción a DDD y CQRS: conceptos básicos de Domain-Driven Design, entidades y value objects, casos de uso, separación entre commands y queries y aplicación al dominio de la acería.
